

Introduction to Algorithm Design



INTRODUCTION TO ALGORITHM DESIGN



| BIBLIOTEX
Digital Library

www.bibliotex.com

INTRODUCTION TO ALGORITHM DESIGN



www.bibliotex.com
email: info@bibliotex.com

e-book Edition 2018

ISBN: 978-1-98461-187-1 (e-book)

This book contains information obtained from highly regarded resources. A Wide variety of references are listed. Reasonable efforts have been made to publish reliable data and information, but the authors, editors, and the publisher cannot assume responsibility for the legality of all materials or the consequences of their use. The authors, editors, and the publisher have attempted to trace the copyright holders of all material in this publication and express regret to copyright holders if permission to publish has not been obtained. If any copyright material has not been acknowledged, let us know so we may rectify in any future reprint. Registered trademark of products or corporate names are used only for explanation and identification without intent to infringe.

Notice: Registered trademark of products or corporate names are used only for explanation and identification without intent of infringement. Case Studies and/or Images presented in the book are the proprietary information of the respective organizations, and have been used here specifically and only for educational purposes. Although care has been taken to check accuracy of formulas and procedures, the detailed methods should be tested further on a small scale before being adopted commercially.

© 2017 3G E-Learning LLC

In Collaboration with 3G E-Learning LLC. Originally Published in printed book format by 3G E-Learning LLC with ISBN 978-1-68095-573-6

EDITORIAL BOARD



Aleksandar Mratinković is born on May 5, 1988. in Arandjelovac, Serbia. He has graduated on Economic high school (2007), The College of Tourism in Belgrade (2013), and also has a master degree of Psychology (Faculty of Philosophy, University of Novi Sad). He has been engaged in different fields of psychology (Developmental Psychology, Clinical Psychology, Educational Psychology and Industrial Psychology) and has published several scientific works



Dan Piestun (PhD) is currently a startup entrepreneur in Israel working on the interface of Agriculture and Biomedical Sciences and was formerly president-CEO of the National Institute of Agricultural Research (INIA) in Uruguay. Dan is a widely published scientist who has received many honours during his career including being a two-time recipient of the Amit Golda Meir Prize from the Hebrew University of Jerusalem, his areas of expertise includes stem cell molecular biology, plant and animal genetics and bioinformatics. Dan's passion for applied science and technological solutions did not stop him from pursuing a deep connection to the farmer, his family and nature. Among some of his interest and practices counts enjoying working as a beekeeper and onboard fishing.



Hazem Shawky Fouda has a PhD. in Agriculture Sciences, obtained his PhD. From the Faculty of Agriculture, Alexandria University in 2008, He is working in Cotton Arbitration & Testing General Organization (CATGO).



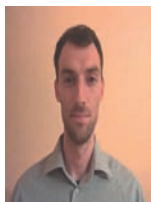
Felecia Killings is the Founder and CEO of LiyahAmore Publishing, a publishing company committed to providing technical and educational services and products to Christian Authors. She operates as the Senior Editor and Writer, the Senior Writing Coach, the Content Marketing Specialist, Editor-in-Chief to the company's quarterly magazine, the Executive and Host of an international virtual network, and the Executive Director of the company's online school for Authors. She is a former high-school English instructor and professional development professor. She possesses a Master of Arts degree in Education and a Bachelor's degree in English and African American studies.



Dr. Sandra El Hajj, Ph.D. in Health Sciences from Nova Southeastern University, Florida, USA is a health professional specialized in Preventive and Global Health. With her 12 years of education obtained from one of the most prominent universities in Beirut, in addition to two leading universities in the State of Florida (USA), Dr. Sandra made sure to incorporate interdisciplinary and multicultural approaches in her work. Her long years of studies helped her create her own miniature world of knowledge linking together the healthcare field with Medical Research, Statistics, Food Technology, Environmental & Occupational Health, Preventive Health and most noteworthy her precious last degree of Global Health. Till today, she is the first and only doctor specialized in Global Health in the Middle East area.



Fozia Parveen has a Dphil in Sustainable Water Engineering from the University of Oxford. Prior to this she has received MS in Environmental Sciences from National University of Science and Technology (NUST), Islamabad Pakistan and BS in Environmental Sciences from Fatima Jinnah Women University (FJWU), Rawalpindi.



Igor Kronic 2003-2007 in the School of Economics. After graduating in 2007, he went on to study at The College of Tourism, at the University of Belgrade where he got his bachelor degree in 2010. He was active as a third-year student representative in the student parliament. Then he went on the Faculty of science, at the University of Novi Sad where he successfully defended his master's thesis in 2013. The crown of his study was the work titled Opportunities for development of cultural tourism in Cacak". Later on, he became part of a multinational company where he got promoted to a deputy director of logistic. Nowadays he is a consultant and writer of academic subjects in the field of tourism.



Dr. Jovan Pehceviski obtained his PhD in Computer Science from RMIT University in Melbourne, Australia in 2007. His research interests include big data, business intelligence and predictive analytics, data and information science, information retrieval, XML, web services and service-oriented architectures, and relational and NoSQL database systems. He has published over 30 journal and conference papers and he also serves as a journal and conference reviewer. He is currently working as a Dean and Associate Professor at European University in Skopje, Macedonia.



Dr. Tanjina Nur finished her PhD in Civil and Environmental Engineering in 2014 from University of Technology Sydney (UTS). Now she is working as Post-Doctoral Researcher in the Centre for Technology in Water and Wastewater (CTWW) and published about eight International journal papers with 80 citations. Her research interest is wastewater treatment technology using adsorption process.



Stephen obtained his PhD from the University of North Carolina at Charlotte in 2013 where his graduate research focused on cancer immunology and the tumor microenvironment. He received postdoctoral training in regenerative and translational medicine, specifically gastrointestinal tissue engineering, at the Wake Forest Institute of Regenerative Medicine. Currently, Stephen is an instructor for anatomy and physiology and biology at Forsyth Technical Community College.



Michelle holds a Masters of Business Administration from the University of Phoenix, with a concentration in Human Resources Management. She is a professional author and has had numerous articles published in the Henry County Times and has written and revised several employee handbooks for various YMCA organizations throughout the United States.

TABLE OF CONTENTS

Preface

xi

Chapter 1 Fundamentals of Algorithms

Introduction.....	1
Learning Objectives.....	1
1.1 Basic Concepts of Algorithms.....	2
1.1.1 Formalization	3
1.1.2 Termination	4
1.1.3 Characteristics of an Algorithm	4
1.1.4 Writing an Algorithm.....	5
1.2 Expressing Algorithms	7
1.2.1 Computer Algorithms	7
1.3 Analysis of Algorithms	8
1.3.1 Masters Theorem	15
1.4 Designing of Algorithms	16
1.4.1 Data Analysis and Design.....	17
1.4.2 Making Choices	21
1.4.3 Process	24
1.5 Sorting of Algorithms	25
1.5.1 Exchange (Bubble) Sort	26
1.5.2 Insertion Sort	26
1.5.3 Selection Sort	27
1.5.4 Quick Sort	27
Summary.....	28
Knowledge Check.....	28
Review Questions	29
Check your Result	29
References	29

Chapter 2 Mathematical Aspects and Analysis of Algorithms

Introduction.....	31
Learning Objectives.....	31
2.1 Mathematical Analysis of Non Recursive Algorithm	32
2.1.1 General Plan for Analyzing Efficiency of Non Recursive Algorithms ..	32
2.1.2 Mathematical Analysis	34
2.1.3 Analysis of Non-recursive Algorithms	35
2.2 Mathematical Analysis of Recursive Algorithm	37
2.2.1 General Plan for Analyzing Efficiency of Recursive	37
2.2.2 Mathematical Analysis	38
2.2.3 Analysis of Recursive Algorithms.....	40
2.2.4 Explicit Formula for n^{th} Fibonacci number	43
2.3 Empirical Analysis of Algorithms.....	46
2.3.1 General Plan for Empirical Analysis of Algorithm	47
2.3.2 Algorithm Visualization.....	48
Summary.....	51
Knowledge Check.....	51
Review Questions	52
Check Your Result	52
References	53

Chapter 3 Advance Data Structure

Introduction.....	55
Learning Objectives.....	55
3.1 Basic Concepts of Data Structure and Algorithms	56
3.1.1 Basic Concepts	58
3.1.2 Classification of Data Structures	59
3.2 Arrays.....	62
3.2.1 Array Representation.....	63
3.2.2 Basic Operations	63
3.3 Linked Lists	80
3.3.1 Singly Linked List.....	81
3.3.2 Doubly Linked List	85
3.4 Binary Search Tree	88
3.4.1 Insertion.....	89
3.4.2 Searching	90
3.4.3 Deletion.....	91

3.4.4	Finding the parent of a given node.....	93
3.4.5	Attaining a reference to a node	94
3.4.6	Finding the smallest and largest values in the binary search tree	94
3.4.7	Tree Traversals	95
3.5	Stack and Queue	99
3.5.1	Stack	100
3.5.2	Queue	104
	Summary	115
	Knowledge Check.....	115
	Check Your Result	116
	Review Questions	116
	References	116

Chapter 4 Searching and Sorting Techniques

	Introduction.....	119
	Learning Objectives	119
4.1	Searching Algorithms.....	120
4.1.1	Linear Search.....	120
4.1.2	Binary Search	121
4.1.3	Interpolation Search.....	124
4.1.4	Hash Table	127
4.2	Sorting Algorithm.....	132
4.2.1	Bubble Sort Algorithm.....	135
4.2.2	Insertion Sort.....	139
4.2.3	Selection Sort.....	142
4.2.4	Merge Sort Algorithm.....	145
4.2.5	Shell Sort	148
4.2.6	Quick Sort.....	152
	Summary	157
	Knowledge Check.....	158
	Review Questions	158
	Check Your Results.....	159
	References	159

Chapter 5 Algorithm Design Methods

	Introduction.....	161
	Learning Objectives	161
5.1	Backtracking Algorithm.....	162
5.1.1	n-Queen's Problem	166

5.1.2	Hamiltonian Circuit problem	169
5.1.3	Subset-Sum problem.....	171
5.2	Branch and bound	174
5.2.1	Knapsack problem.....	175
5.2.2	Assignment problem.....	178
5.2.3	Traveling salesman problem.....	182
	Summary.....	185
	Knowledge Check.....	185
	Review Questions	186
	Check Your Result	187
	References	187

Chapter 6 Algorithms Techniques and Analysis

	Introduction.....	189
	Learning Objectives.....	189
6.1	Greedy Algorithms.....	190
6.1.1	Elements of the Greedy Strategy.....	193
6.1.2	Huffman Codes.....	198
6.1.3	Matroids and Greedy Methods	204
6.1.4	A Task-Scheduling Problem as a Matroid.....	206
6.2	Dynamic Programming	209
6.2.1	Overlapping Sub Problems.....	210
6.2.2	Optimal Substructure	212
6.2.3	Steps in Dynamic Programming	214
6.2.4	Knapsack Problem	224
6.3	Amortized Analysis.....	226
6.3.1	Aggregate Method	227
6.3.2	Accounting (Banker's) Method.....	228
6.3.3	Potential (Physicist's) Method	229
	Summary.....	231
	Knowledge Check.....	231
	Review Questions	232
	Check Your Result	233
	References	233

Chapter 7 Divide and Conquer

	Introduction.....	235
	Learning Objectives.....	235

7.1	Analysis of Divide and Conquer Algorithms.....	237
7.1.1	Multiplication.....	238
7.1.2	Recurrence Relations.....	240
7.1.3	Merge sort.....	242
7.1.4	Median	246
7.2	Strassen's Algorithm.....	248
7.2.1	Karatsuba Fast Multiplication	250
7.2.2	Finding the Closest Pair of Points	255
7.2.3	Strassen's algorithm:Matrix multiplication	259
	Summary	264
	Knowledge Check.....	264
	Review Questions	265
	Check Your Result	265
	References	265

Index

235

PREFACE

Problem solving is an essential part of every scientific discipline. It has two components; problem identification and formulation, and solution of the formulated problem. One can solve a problem on its own using ad hoc techniques or follow those techniques that have produced efficient solutions to similar problems. This requires the understanding of various algorithm design techniques, how and when to use them to formulate solutions and the context appropriate for each of them. Algorithm design is identified and incorporated into many solution theories of operation research, such as dynamic programming and divide-and-conquer. Techniques for designing and implementing algorithm designs are algorithm design patterns, such as template method pattern and decorator pattern, and uses of data structures, and name and sort lists. Some current day uses of algorithm design can be found in internet retrieval processes of web crawling, packet routing and caching.

Content Coverage

Chapter One starts with the basic concepts of algorithms. Furthermore, it illustrates how to express algorithms and algorithm analysis, design, and sorting. *Chapter Two* presents the coverage of mathematical analysis of recursive and non-recursive algorithm. *Chapter Three* will give you a great understanding on data structures needed to understand the complexity of enterprise level applications and need of algorithms, and data structures. *Chapter Four* is about searching and sorting techniques. Because searching and sorting are common computer tasks, or recipes, for doing searching and sorting. *Chapter Five* explores the algorithm design techniques to solve problems algorithmically that is applicable to a variety of problems from different areas of computing. *Chapter Six* focuses on algorithm analysis dedicated to understanding the complexity of algorithms. *Chapter Seven* reveals on the divide and conquer technique, which is the basis of efficient algorithms for all kinds of problems, such as sorting, multiplying large numbers, finding the closest pair of points, syntactic analysis, and computing the discrete Fourier transform (FFTs).

The following are the salient features of the book:

- The book sponsors the study of algorithm design techniques by presenting most of the useful algorithm design techniques and illustrating them through numerous examples.

- Topics are logically arranged according to the course outline of the subject.
- The use of simple language to facilitate a better understanding of each chapter.
- Illustrative examples of varying difficulties, wherever needed, are presented to enhance the interest of students.
- The provision of knowledge check test for each chapter is given at the end of chapter to strengthen the learning process of students.
- Case Study, Role Model, Keywords, and Examples are also provided to enrich the knowledge of students.

FUNDAMENTALS OF ALGORITHMS

CHAPTER

1

LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Discuss about basic concepts of algorithms
2. Perform analysis of algorithms
3. Explain the designing of algorithms
4. Understand sorting of algorithms

INTRODUCTION

An algorithm is a procedure or formula for solving a problem, based on conducting a sequence of specified actions. A computer program can be viewed as an elaborate algorithm. In mathematics and computer science, an algorithm usually means a small procedure that solves a recurrent problem.

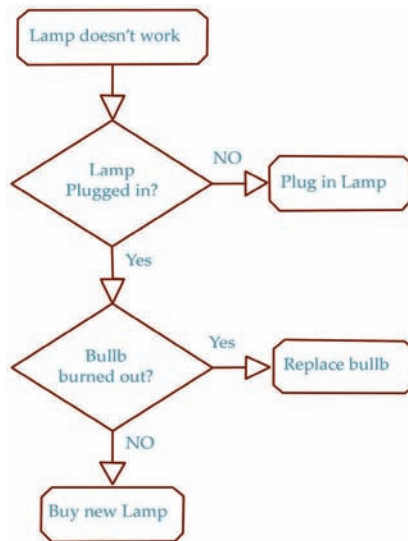
Algorithms are widely used throughout all areas of IT (information technology). A search engine algorithm, for example, takes search strings of keywords and operators as input, searches its associated database for relevant web pages, and returns results. An encryption algorithm transforms data according to specified actions to protect it. A

secret key algorithm such as the U.S. Department of Defense's Data Encryption Standard (DES), for example, uses the same key to encrypt and decrypt data. As long as the algorithm is sufficiently sophisticated, no one lacking the key can decrypt the data.

1.1. BASIC CONCEPTS OF ALGORITHMS

In mathematics, computing, and related subjects, an algorithm is an effective method for solving a problem using a finite sequence of instructions.

Each algorithm is a list of well-defined instructions for completing a task. Starting from an initial state, the instructions describe a computation that proceeds through a well-defined series of successive states, eventually terminating in a final ending state. The transition from one state to the next is not necessarily deterministic; some algorithms, known as randomized algorithms, incorporate randomness.



While there is no generally accepted *formal* definition of “algorithm”, an informal definition could be “a process that performs some sequence of operations.” For some people, a Programme is only an algorithm if it stops eventually. For others, a Programme is only an algorithm if it stops before a given number of calculation steps.

A prototypical example of an algorithm is Euclid’s algorithm to determine the maximum common divisor of two integers. We can derive clues to the issues involved and an informal meaning of the word from the following quotation from Boolos & Jeffrey:

No human being can write fast enough, or long enough, or small enough (†”smaller and smaller without limit...you’d be trying to write on molecules, on atoms, on

electrons”) to list all members of an enumerably infinite set by writing out their names, one after another, in some notation.

But humans can do something equally useful, in the case of certain enumerably infinite sets: They can give explicit instructions for determining the n th member of the set, for arbitrary finite n .

Such instructions are to be given quite explicitly, in a form in which they could be followed by a computing machine, or by a human who is capable of carrying out only very elementary operations on symbols

1.1.1. Formalization

Algorithms are essential to the way computers process information.

Thus, an algorithm can be considered to be any sequence of operations that can be simulated by a Turing-complete system.

Typically, when an algorithm is associated with processing information, data is read from an input source, written to an output device, and/or stored for further processing. Stored data is regarded as part of the internal state of the entity performing the algorithm. In practice, the state is stored in one or more data structures.

That is, any conditional steps must be systematically dealt with, case-by-case; the criteria for each case must be clear (and computable).

Because an algorithm is a precise list of precise steps, the order of computation will always be critical to the functioning of the algorithm. Instructions are usually assumed to be listed explicitly, and are described as starting “from the top” and going “down to the bottom”, an idea that is described more formally by *flow of control*. So far, this discussion of the formalization of an algorithm has assumed the premises of imperative Programming. This is the most common conception, and it attempts to describe a task in discrete, “mechanical” means. Unique to this conception of formalized algorithms is the assignment, setting the value of a variable. It derives from the intuition of “memory” as a scratchpad. There is an example below of such an assignment.



REMEMBER

Many computer programmes contain algorithms that specify the specific instructions a computer should perform (in a specific order) to carry out a specified task, such as calculating employees’ pay checks or printing students’ report cards.

**REMEMBER**

Any such computational process, the algorithm must be rigorously defined: specified in the way it applies in all possible circumstances that could arise.

1.1.2. Termination

But if the length of the process is not known in advance, then “trying” it may not be decisive, because if the process does go on forever — then at no time will we ever be sure of the answer.

As it happens, no other method can do any better, as was shown by Alan Turing with his celebrated result on the undesirability of the so-called halting. There is no algorithmic procedure for determining of arbitrary algorithms whether or not they terminate from given starting states. The analysis of algorithms for their likelihood of termination is called termination analysis.

The examples of (im-)“proper” subtraction at partial function for more about what can happen when an algorithm fails for certain of its input numbers — e.g.,

- Non-termination,
- Production of “junk” (output in the wrong format to be considered a number) or no number(s) at all (halt ends the computation with no output),
- Wrong number(s),
- A combination of these.

Kleene proposed that the production of “junk” or failure to produce a number is solved by having the algorithm detect these instances and produce e.g., an error message (he suggested “0”), or preferably, force the algorithm into an endless loop.

1.1.3. Characteristics of an Algorithm

Not all procedures can be called an algorithm. An algorithm should have the following characteristics –

- *Unambiguous* – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- *Input* – An algorithm should have 0 or more well-defined inputs.
- *Output* – An algorithm should have 1 or more well-defined outputs, and should match the desired output.



- *Finiteness* – Algorithms must terminate after a finite number of steps.
- *Feasibility* – Should be feasible with the available resources.
- *Independent* – An algorithm should have step-by-step directions, which should be independent of any programming code.

1.1.4. Writing an Algorithm

There are no well-defined standards for writing algorithms. Rather, it is problem and resource dependent. Algorithms are never written to support a particular programming code.

As we know that all programming languages share basic code constructs like loops (do, for, while), flow-control (if-else), etc. These common constructs can be used to write an algorithm.

We write algorithms in a step-by-step manner, but it is not always the case. Algorithm writing is a process and is executed after the problem domain is well-defined. That is, we should know the problem domain, for which we are designing a solution.

Example

Let us try to learn algorithm-writing by using an example.

Problem – Design an algorithm to add two numbers and display the result.

- step 1** – START
- step 2** – declare three integers **a**, **b** & **c**
- step 3** – define values of **a** & **b**
- step 4** – add values of **a** & **b**
- step 5** – store output of step 4 to **c**
- step 6** – print **c**
- step 7** – STOP



The word algorithm derives from the name of the mathematician, Mohammed ibn-Musa al-Khwarizmi, who was part of the royal court in Baghdad and who lived from about 780 to 850. Al-Khwarizmi's work is the likely source for the word algebra as well.

Algorithms tell the programmers how to code the program. Alternatively, the algorithm can be written as –

step 1 – START ADD

step 2 – get values of **a** & **b**

step 3 – $c \leftarrow a + b$

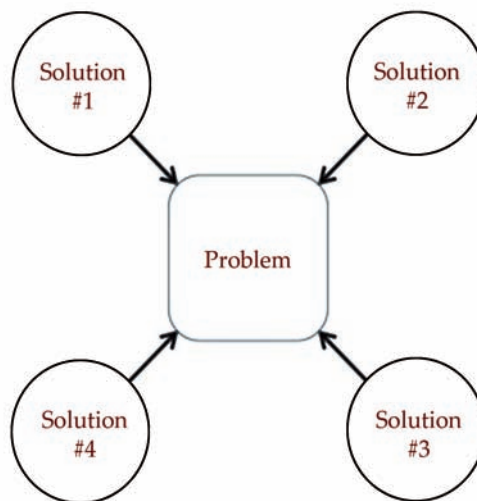
step 4 – display **c**

step 5 – STOP

In design and analysis of algorithms, usually the second method is used to describe an algorithm. It makes it easy for the analyst to analyze the algorithm ignoring all unwanted definitions. He can observe what operations are being used and how the process is flowing.

Writing step numbers, is optional.

We design an algorithm to get a solution of a given problem. A problem can be solved in more than one ways.



Hence, many solution algorithms can be derived for a given problem. The next step is to analyze those proposed solution algorithms and implement the best suitable solution.

1.2. EXPRESSING ALGORITHMS

Algorithms can be expressed in many kinds of notation, including natural languages, pseudo code, flowcharts, Programming languages or control tables (processed by interpreters). Natural language expressions of algorithms tend to be verbose and ambiguous, and are rarely used for complex or technical algorithms. Pseudocode, flowcharts and control tables are structured ways to express algorithms that avoid many of the ambiguities common in natural language statements, while remaining independent of a particular implementation language. Programming languages are primarily intended for expressing algorithms in a form that can be executed by a computer, but are often used as a way to define or document algorithms.

There is a wide variety of representations possible and one can express a given Turing machine Programme as a sequence of machine tables (see more at finite state machine and state transition table), as flowcharts (see more at state diagram), or as a form of rudimentary machine code or assembly code called “sets of quadruples”..

Sometimes it is helpful in the description of an algorithm to supplement small “flow charts” (state diagrams) with natural-language and/or arithmetic expressions written inside “block diagrams” to summarize what the “flow charts” are accomplishing.

Representations of algorithms are generally classed into three accepted levels of Turing machine description:

- *High-level description:* “...prose to describe an algorithm, ignoring the implementation details. At this level we do not need to mention how the machine manages its tape or head.”
- *Implementation description:* “...prose used to define the way the Turing machine uses its head and the way that it stores data on its tape. At this level we do not give details of states or transition function.”
- *Formal description:* Most detailed, “lowest level”, gives the Turing machine’s “state table”.

1.2.1. Computer Algorithms

In computer systems, an algorithm is basically an instance of logic written in software by software developers to be effective for the intended “target” computer(s), in order for the software on the target machines to *do something*. For instance, if a person is writing software that is supposed to print out a PDF document located at the operating system folder “/My Documents” at computer drive “D:” every Friday, they will write an algorithm that specifies the following actions: “If today’s date (computer time) is ‘Friday,’ open the document at ‘D:/My Documents’ and call the ‘print’ function”.

While this simple algorithm does not look into whether the printer has enough paper or whether the document has been moved into a different location, one can make

this algorithm more robust and anticipate these problems by rewriting it as a formal CASE statement or as a (carefully crafted) sequence of IF-THEN-ELSE statements. For example the CASE statement might appear as follows (there are other possibilities):

- *Case 1: IF today's date is NOT Friday THEN exit this CASE instruction ELSE*
- *Case 2: IF today's date is Friday AND the document is located at 'D:/My Documents' AND there is paper in the printer THEN print the document (and exit this CASE instruction) ELSE*
- *Case 3: IF today's date is Friday AND the document is NOT located at 'D:/My Documents' THEN display 'document not found' error message (and exit this CASE instruction) ELSE*
- *Case 4: IF today's date is Friday AND the document is located at 'D:/My Documents' AND there is NO paper in the printer THEN, (i) display 'out of paper' error message (ii) exit. Note that CASE 3 includes two possibilities: (i) the document is NOT located at 'D:/My Documents' AND there's paper in the printer OR (ii) the document is NOT located at 'D:/My Documents' AND there's NO paper in the printer.*

The sequence of IF-THEN-ELSE tests might look like this:

- *Test 1: IF today's date is NOT Friday THEN done ELSE TEST 2:*
- *Test 2: IF the document is NOT located at 'D:/My Documents' THEN display 'document not found' error message ELSE TEST 3:*
- *Test 3: IF there is NO paper in the printer THEN display 'out of paper' error message ELSE print the document.*

These examples' logic grants precedence to the instance of "NO document at 'D:/My Documents' ".



Efficiency is the ability to avoid wasting materials, energy, efforts, money, and time in doing something or in producing a desired result.

1.3. ANALYSIS OF ALGORITHMS

When analysing a Programme in terms of efficiency, we want to look at questions such as, "How long does it take for the

Programme to run?” and “Is there another approach that will get the answer more quickly?” Efficiency will always be less important than correctness; if you do not care whether a Programme works correctly, you can make it run very quickly indeed, but no one will think it’s much of an achievement! On the other hand, a Programme that gives a correct answer after ten thousand years is not very useful either, so **efficiency** is often an important issue.

The term “efficiency” can refer to efficient use of almost any resource, including time, computer memory, disk space, or network bandwidth. However, we will deal exclusively with time efficiency, and the major question that we want to ask about a Programme is, how long does it take to perform its task? It really makes little sense to classify an individual Programme as being “efficient” or “inefficient.” It makes more sense to compare two (correct) Programmes that perform the same task and ask which one of the two is “more efficient,” that is, which one performs the task more quickly. However, even here there are difficulties.

The running time of a Programme is not well-defined. The run time can be different depending on the number and speed of the processors in the computer on which it is run and, in the case of Java, on the design of the Java Virtual Machine which is used to interpret the Programme.

It can depend on details of the **compiler** which is used to translate the Programme from high-level language to machine language. Furthermore, the run time of a Programme depends on the size of the problem which the Programme has to solve. It takes a sorting Programme longer to sort 10000 items than it takes it to sort 100 items.

When the run times of two Programmes are compared, it often happens that Programme A solves small problems faster than Programme B, while Programme B solves large problems faster than Programme A, so that it is simply not the case that one Programme is faster than the other in all cases.

In spite of these difficulties, there is a field of computer science dedicated to analysing the efficiency of Programmes. The field is known as Analysis of Algorithms. The focus is on algorithms, rather than on Programmes as such, to avoid having to deal with multiple implementations of the same algorithm written in different languages, compiled



Compiler is a computer program that transforms source code written in a programming language into another computer language, with the latter often having a binary form known as object code.

with different compilers, and running on different computers. Still, even though it is a theoretical field, every working Programmer should be aware of some of its techniques and results. Because this is not a mathematics book, the treatment will be rather informal. One of the main techniques of analysis of algorithms is asymptotic analysis. The term “asymptotic” here means basically “the tendency in the long run.” An asymptotic analysis of an algorithm’s run time looks at the question of how the run time depends on the size of the problem.

The analysis is asymptotic because it only considers what happens to the run time as the size of the problem increases without limit; it is not concerned with what happens for problems of small size or, in fact, for problems of any fixed finite size. Only what happens in the long run, as the problem size increases without limit, is important.

Showing that Algorithm A is asymptotically faster than Algorithm B does not necessarily mean that Algorithm A will run faster than Algorithm B for problems of size 10 or size 1000 or even size 1000000 — it only means that if you keep increasing the problem size, you will eventually come to a point where Algorithm A is faster than Algorithm B. An asymptotic analysis is only a first approximation, but in practice it often gives important and useful information.

Using this notation, we might say, for example, that an algorithm has a running time that is $O(n^2)$ or $O(n)$ or $O(\log(n))$. These notations are read “Big-Oh of n squared,” “Big-Oh of n ,” and “Big-Oh of $\log n$ ” (where $\log$ is a logarithm function). More generally, we can refer to $O(f(n))$ (“Big-Oh of f of n ”), where $f(n)$ is some function that assigns a positive real number to every positive integer n . The “ n ” in this notation refers to the size of the problem.

Before you can even begin an asymptotic analysis, you need some way to measure problem size. Usually, this is not a big issue. *For example*, if the problem is to sort a list of items, then the problem size can be taken to be the number of items in the list. When the input to an algorithm is an integer, as in the case of an algorithm that checks whether a given positive integer is prime, the usual measure of the size of a problem is the number of bits in the input integer rather than the integer itself. More generally, the number of bits in the input to a problem is often a good measure of the size of the problem.

To say that the running time of an algorithm is $O(f(n))$ means that for large values of the problem size, n , the running time of the algorithm is no bigger than some constant times $f(n)$. (More rigorously, there is a number C and a positive integer M such that whenever n is greater than M , the run time is less than or equal to $C \cdot f(n)$.) The constant takes into account details such as the speed of the computer on which the algorithm is run; if you use a slower computer, you might have to use a bigger constant in the formula, but changing the constant would not change the basic fact that the run time is $O(f(n))$.



The constant also makes it unnecessary to say whether we are measuring time in seconds, years, CPU cycles, or any other unit of measure; a change from one unit of measure to another is just multiplication by a constant. Note also that $O(f(n))$ does not depend at all on what happens for small problem sizes, only on what happens in the long run as the problem size increases without limit.

To look at a simple example, consider the problem of adding up all the numbers in an array. The problem size, n , is the length of the array. Using A as the name of the **array**, the algorithm can be expressed in Java as:

```
total = 0;
for (int i = 0; i < n; i++)
    total = total + A[i];
```

This algorithm performs the same operation, $\text{total} = \text{total} + A[i]$, n times. The total time spent on this operation is $a \cdot n$, where a is the time it takes to perform the operation once. Now, this is not the only thing that is done in the algorithm. The value of i is incremented and is compared to n each time through the loop.

This adds an additional time of $b \cdot n$ to the run time, for some constant b . Furthermore, i and total both have to be initialized to zero; this adds some constant amount c to the running time.

The exact running time would then be $(a+b) \cdot n + c$, where the constants a , b , and c depend on factors such as how the code is compiled and what computer it is run on. Using the fact that c is less than or equal to $c \cdot n$ for any positive integer n , we can say that the run time is less than or equal to $(a+b+c) \cdot n$. That is, the run time is less than or equal to a constant times n . By definition, this means that the run time for this algorithm is $O(n)$.

If this explanation is too mathematical for you, we can just note that for large values of n , the c in the formula $(a+b) \cdot n + c$ is insignificant compared to the other term, $(a+b) \cdot n$. We say that c is a “lower order term.” When doing asymptotic analysis, lower order terms can be discarded. A rough, but correct, asymptotic analysis of the algorithm would go something like this: Each iteration of the for loop takes a certain constant amount of time. There are n iterations of the loop, so the total run time is a constant times n , plus lower order terms



Array is a systematic arrangement of similar objects, usually in rows and columns.

(to account for the initialization). Disregarding lower order terms, we see that the run time is $O(n)$. Note that to say that an algorithm has run time $O(f(n))$ is to say that its run time is no bigger than some constant times $f(n)$ (for large values of n). $O(f(n))$ puts an upper limit on the run time. However, the run time could be smaller, even much smaller. *For example*, if the run time is $O(n)$, it would also be correct to say that the run time is $O(n^2)$ or even $O(n^{10})$. If the run time is less than a constant times n , then it is certainly less than the same constant times n^2 or n^{10} .

Of course, sometimes it's useful to have a lower limit on the run time. That is, we want to be able to say that the run time is greater than or equal to some constant times $f(n)$ (for large values of n). The notation for this is $\Omega(f(n))$, read "Omega of f of n ." "Omega" is the name of a letter in the Greek alphabet, and Ω is the upper case version of that letter. (To be technical, saying that the run time of an algorithm is $\Omega(f(n))$ means that there is a positive number C and a positive integer M such that whenever n is greater than M , the run time is greater than or equal to $C \cdot f(n)$.) $O(f(n))$ tells you something about the maximum amount of time that you might have to wait for an algorithm to finish; $\Omega(f(n))$ tells you something about the minimum time.

The algorithm for adding up the numbers in an array has a run time that is $\Omega(n)$ as well as $O(n)$. When an algorithm has a run time that is both $\Omega(f(n))$ and $O(f(n))$, its run time is said to be $\Theta(f(n))$, read "Theta of f of n ." (Theta is another letter from the Greek alphabet.) To say that the run time of an algorithm is $\Theta(f(n))$ means that for large values of n , the run time is between $a \cdot f(n)$ and $b \cdot f(n)$, where a and b are constants (with b greater than a , and both greater than 0).

Let's look at another example. Consider the algorithm that can be expressed in Java in the following method:

```
/**
 * Sorts the n array elements A[0], A[1]...,
 * A[n-1] into increasing order.
 */
public static simpleBubbleSort(int[] A, int
n) {
    for (int i = 0; i < n; i++) {
        // Do n passes through the array...
        for (int j = 0; j < n-1; j++) {
            if (A[j] > A[j+1]) {
                // A[j] and A[j+1] are out of order, so swap
                them
                int temp = A[j];
                A[j] = A[j+1];
                A[j+1] = temp;
            }
        }
    }
}
```



Here, the parameter n represents the problem size. The outer for loop in the method is executed n times. Each time the outer for loop is executed, the inner for loop is executed $n-1$ times, so the if statement is executed $n*(n-1)$ times. This is n^2-n , but since lower order terms are not significant in an asymptotic analysis, it is good enough to say that the if statement is executed about n^2 times.

In particular, the test $A[j] > A[j+1]$ is executed about n^2 times, and this fact by itself is enough to say that the run time of the algorithm is $W(n^2)$, that is, the run time is at least some constant times n^2 . Furthermore, if we look at other operations — the assignment statements, incrementing i and j , etc. — none of them are executed more than n^2 times, so the run time is also $O(n^2)$, that is, the run time is no more than some constant times n^2 . Since it is both $W(n^2)$ and $O(n^2)$, the run time of the simpleBubbleSort algorithm is $Q(n^2)$.

You should be aware that some people use the notation $O(f(n))$ as if it meant $\Theta(f(n))$. That is, when they say that the run time of an algorithm is $O(f(n))$, they mean to say that the run time is about equal to a constant times $f(n)$. For that, they should use $\Theta(f(n))$. Properly speaking, $O(f(n))$ means that the run time is less than a constant times $f(n)$, possibly much less. So far, the analysis has ignored an important detail. We have looked at how run time depends on the problem size, but in fact the run time usually depends not just on the size of the problem but on the specific data that has to be processed.

The run time of a sorting algorithm can depend on the initial order of the items that are to be sorted, and not just on the number of items.



To account for this dependency, we can consider either the worst case run time analysis or the average case run time analysis of an algorithm. For a worst case run time analysis, we consider all possible problems of size n and look at the longest possible run time for all such problems. For an average case analysis, we consider all possible problems of size n and look at the average of the run times for all such problems. Usually, the average case analysis assumes that all problems of size n are equally likely to be encountered, although this is not always realistic — or even possible in the case where

there is an infinite number of different problems of a given size. In many cases, the average and the worst case run times are the same to within a constant multiple. This means that as far as asymptotic analysis is concerned, they are the same. That is, if the average case run time is $O(f(n))$ or $\Theta(f(n))$, then so is the worst case. However, later in the book, we will encounter a few cases where the average and worst case asymptotic analyses differ.

Most important, though, you should have a feeling for exactly what it means to say that the running time of an algorithm is $O(f(n))$ or $\Theta(f(n))$ for some common functions $f(n)$. The main point is that these notations do not tell you anything about the actual numerical value of the running time of the algorithm for any particular case.

They do not tell you anything at all about the running time for small values of n . What they do tell you is something about the rate of growth of the running time as the size of the problem increases.

Suppose you compare two algorithms that solve the same problem. The run time of one algorithm is $\Theta(n^2)$, while the run time of the second algorithm is $\Theta(n^3)$. What does this tell you? If you want to know which algorithm will be faster for some particular problem of size, say, 100, nothing is certain. As far as you can tell just from the asymptotic analysis, either algorithm could be faster for that particular case — or in any particular case. But what you can say for sure is that if you look at larger and larger problems, you will come to a point where the $\Theta(n^2)$ algorithm is faster than the $\Theta(n^3)$ algorithm. Furthermore, as you continue to increase the problem size, the relative advantage of the $\Theta(n^2)$ algorithm will continue to grow. There will be values of n for which the $\Theta(n^2)$ algorithm is a thousand times faster, a million times faster, a billion times faster, and so on. This is because for any positive constants a and b , the function $a \cdot n^3$ grows faster than the function $b \cdot n^2$ as n gets larger. (Mathematically, the limit of the ratio of $a \cdot n^3$ to $b \cdot n^2$ is infinite as n approaches infinity.)

This means that for “large” problems, a $\Theta(n^2)$ algorithm will definitely be faster than a $\Theta(n^3)$ algorithm. You just don’t know — based on the asymptotic analysis alone — exactly how large “large” has to be. In practice, in fact, it is likely that the $\Theta(n^2)$ algorithm will be faster even for fairly small values of n , and absent other information you would generally prefer a $\Theta(n^2)$ algorithm to a $\Theta(n^3)$ algorithm.

So, to understand and apply asymptotic analysis, it is essential to have some idea of the rates of growth of some common functions. For the power functions n , n^2 , n^3 , n^4 ..., the larger the exponent, the greater the rate of growth of the function.

Exponential functions such as 2^n and 10^n , where the n is in the exponent, have a growth rate that is faster than that of any power function. In fact, exponential functions grow so quickly that an algorithm whose run time grows exponentially is almost certainly impractical even for relatively modest values of n , because the running time is just too long. Another function that often turns up in asymptotic analysis is the logarithm function, $\log(n)$. There are actually many different logarithm functions,



but the one that is usually used in computer science is the so-called logarithm to the base two, which is defined by the fact that $\log(2^x) = x$ for any number x . (Usually, this function is written $\log_2(n)$, but I will leave out the subscript 2, since I will only use the base-two logarithm in this book.)

The logarithm function grows very slowly. The growth rate of $\log(n)$ is much smaller than the growth rate of n . The growth rate of $n \cdot \log(n)$ is a little larger than the growth rate of n , but much smaller than the growth rate of n^2 .

1.3.1. Masters Theorem

In the analysis of algorithms, the master theorem, which is a specific case of the Akra-Bazzi theorem, provides a cookbook solution in asymptotic terms for recurrence relations of types that occur in practice.

It was popularized by the canonical algorithms which introduces and proves Nevertheless, not all recurrence relations can be solved with the use of the master theorem.

Consider a problem that can be solved using recurrence algorithm such as below:

```

procedure T(n: size of problem)
defined    as:
    if n<kthen exit
    Do work of amount f(n)
    T(n/b)
    T(n/b)
    repeat for a total of a times...
    T(n/b)
end procedure
  
```

In above algorithm we are dividing the problem in to number of sub problems recursively, each sub problem being of size n/b . This can be visualized as building a call tree with each node of a tree an instance of one recursive call and its child nodes being instance of next calls. In above example, each node would have a number of child nodes.

Each node does amount of work that depends on size of sub problem n passed to that instance of recursive call and given by $f(n)$.

For example, if each recursive call is doing sorting then size of work does by each node in the tree would be at least $O(n \log(n))$. Total size of work done by entire tree is sum of work performed by all the nodes in the tree.

Algorithm such as above can be represented as recurrence relationship,

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

This recursive relationship can be successively substituted in to itself and expanded to obtain expression for total amount of work done.

Original Master theorem allows to easily calculate run time of such a recursive algorithm in Big O notation without doing expansion of above recursive relationship. A generalized form of Master Theorem by Akra and Bazzi introduced in 1998 is more usable, simpler and applicable on wide number of cases that occurs in practice.

The master theorem concerns recurrence relations of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \text{ where } a \geq 1, b > 1.$$



Node is either a redistribution point or a communication endpoint. The definition of a node depends on the network and protocol layer referred to.

In the application to the analysis of a recursive algorithm, the constants and function take on the following significance:

- n is the size of the problem.
- a is the number of subproblems in the recursion.
- n/b is the size of each subproblem. (Here it is assumed that all subproblems are essentially the same size.)
- $f(n)$ is the cost of the work done outside the recursive calls, which includes the cost of dividing the problem and the cost of merging the solutions to the subproblems.

1.4. DESIGNING OF ALGORITHMS

At first glance, the algorithms move-until-out and quick-sort have little in common. One process structures; the other processes lists. One creates a new structure for the generative step; the other **splits** up a list into three pieces and recur on two of them. In short, a comparison of the two examples of generative recursion suggests that the design of algorithms is an ad hoc activity and that it is impossible to come up with a general design recipe. A closer look, however, suggests a different picture.

First, even though we speak of algorithms as processes that solve problems, they are still functions that consume and produce data. In other words, we still choose data to represent a problem, and we must definitely understand the nature of our data if we wish to understand the process. Second, we describe the processes in terms of data, for example, “creating a new structure” or “partitioning a list of numbers.”

Third, we always distinguish between input data for which it is trivial to produce a solution and those for which it is not. Fourth, the generation of problems is the key to the design of algorithms. Although the idea of how to generate a new problem might be independent of a data representation, it must certainly be implemented for whatever form of representation we choose for our problem. Finally, once the generated problems have been solved, the solutions must be combined with other values.

1.4.1. Data Analysis and Design

The choice of a data representation for a problem often affects our thinking about the process. Sometimes the description of a process dictates a particular choice of representation. On other occasions, it is possible and worthwhile to explore alternatives. In any case, we must Analyse and define our data collections.

Contract, Purpose, Header

We also need a contract, a definition header, and a purpose statement. Since the generative step has no connection to the structure of the data definition, the purpose statement should not only specify what the function does but should also include a comment that explains in general terms how it works.

Function Examples

In our previous design recipes, the function examples merely specified which output the function should produce for some given input. For algorithms, examples should illustrate how the algorithm proceeds for some given input. This helps us to design, and readers to understand, the algorithm. For functions such as move-until-out the process is trivial and does not need more than a few words.

Template

Our discussion suggests a general template for algorithms:

```
(define (generative-recursive-fun problem)
  cond
    [(trivially-solvable? problem)
     (determine-solution problem)]
```

```

[else
  (combine-solutions
    ... problem...
    (generative-recursive-fun (generate-
      problem-      1 problem))
    (generative-recursive-fun (generate-
      problem-n problem))))))

```

Test

Once we have a complete function, we must also test it. As before, the goal of testing is to discover bugs and to eliminate them.



REMEMBER

Remember that testing cannot validate that the function works correctly for all possible inputs.

Termination

Unfortunately, the standard recipe is not good enough for the design of algorithms. Up to now, a function has always produced an output for any legitimate input. That is, the evaluation has always stopped. After all, by the nature of our recipe, each natural recursion consumes an immediate piece of the input, not the input itself. Because data is constructed in a hierarchical manner, this means that the input shrinks at every stage. Hence the function sooner or later consumes an atomic piece of data and stops.

With functions based on generative recursion, this is no longer true. The internal recursions do not consume an immediate component of the input but some new piece of data, which is generated from the input.

In addition, even the slightest mistake in translating the process description into a function definition may cause an infinite loop. The problem is most easily understood with an example. Consider the following definition of `smaller-items`, one of the two “problem generators” for `quick-sort`:

```

;; smaller-items: (listof number) number →
(listof number)
;; to create a list with all those numbers on
alon
;; that are smaller than or equal to threshold
(define (smaller-items alon threshold)
  (cond
    [(empty? alon) empty]
    [else (if (<= (first alon) threshold)
              (cons (first alon) (smaller-items
                                alon) threshold))
            (rest
              smaller-items (rest alon)
                            threshold))]))

```



Instead of < it employs $\bar{U}$ to compare numbers. As a result, this function produces (list 5) when applied to (list 5) and 5.

A termination argument, which explains why the process produces an output for every input and how the function implements this idea; or a warning, which explains when the process may not terminate.

For quick-sort, the argument might look like this:

At each step, quick-sort partitions the list into two sublists using smaller-items and larger-items. Each function produces a list that is smaller than the input (the second argument), even if the threshold (the first argument) is an item on the list. Hence each recursive application of quick-sort consumes a strictly shorter list than the given one. Eventually, quick-sort receives and returns empty.

Without such an argument an algorithm must be considered incomplete. A good termination argument may on occasion also reveal additional termination cases. For example, (smaller-items N (list N)) and (larger-items N(list N)) always produce empty for any N. Therefore we know that quick-sort's answer for (list N) is (list N). To add this knowledge to quick-sort, we simply add a cond-clause:

```
(define (quick-sort alon)
  (cond
    [(empty? alon) empty]
    [(empty? (rest alon)) alon]
    [else (append
            (quick-sort (smaller-items alon)
                        (first alon))
            (list (first alon))
            (quick-sort (larger-items alon)
                        (first alon))))]))
```

The condition (empty? (rest alon)) is one way to ask whether alon contains one item.

Define the function tabulate-div, which accepts a number n and tabulates the list of all of its divisors, starting with 1 and ending in n. A number d is a divisor of a number n if the remainder of dividing n by d is 0, that is, (= (remainder n d) 0) is true. The smallest divisor of any number is 1; the largest one is the number itself.



Exercise

Develop the function merge-sort, which sorts a list of numbers in ascending order, using the following two auxiliary functions:



The first one, *make-singles*, constructs a list of one-item lists from the given list of numbers. For example,

(equal? (make-singles (list 2 5 9 3))

(list (list 2) (list 5) (list 9) (list 3)))

The second one, *merge-all-Neighbours*, merges pairs of Neighbouring lists. More specifically, it consumes a list of lists (of numbers) and merges Neighbours. For example,

(equal? (merge-all-Neighbours (list (list 2) (list 5) (list 9) (list 3)))

(list (list 2 5) (list 3 9)))

(equal? (merge-all-Neighbours (list (list 2 5) (list 3 9)))

(list (list 2 3 5 9)))

In general, this function yields a list that is approximately half as long as the input. Why is the output not always half as long as the input?

Structural Versus Generative Recursion

The template for algorithms is so general that it even covers functions based on structural recursion. Consider the version with one termination clause and one generation step:

```
(define (generative-recursive-fun problem)
  (cond
    [(trivially-solvable? problem)
     (determine-solution problem)]
    [else
     (combine-solutions
      problem
      (generative-recursive-fun (generate-
        problem problem))))])
```

If we replace *trivially-solvable?* with *empty?* and *generate-problem* with *rest*, the outline is a template for a list-processing function:

```
(define (generative-recursive-fun problem)
  (cond
    [(empty? problem) (determine-solution
      problem)]
    [else
     (combine-solutions
      problem
      (generative-recursive-fun (rest
        problem))))])
```



1.4.2 Making Choices

A user cannot distinguish sort and quick-sort. Both consume a list of numbers; both produce a list that consists of the same numbers arranged in ascending order. To an observer, the functions are completely equivalent. This raises the question of which of the two a Programmer should provide. More generally, if we can develop a function using structural recursion and an equivalent one using generative recursion, what should we do? To understand this choice better, let us discuss another classical example of generative recursion from mathematics: the problem of finding the greatest common divisor of two positive natural numbers.

All such numbers have at least one divisor in common:

1. On occasion, this is also the only common divisor. For example, 2 and 3 have only 1 as common divisor because 2 and 3, respectively, are the only other divisors.

Then again, 6 and 25 are both numbers with several divisors:

- 6 is evenly divisible by 1, 2, 3, and 6;
- 25 is evenly divisible by 1, 5, and 25.

Still, the greatest common divisor of 25 and 6 is 1. In contrast, 18 and 24 have many common divisors:

- 18 is evenly divisible by 1, 2, 3, 6, 9, and 18;
- 24 is evenly divisible by 1, 2, 3, 4, 6, 8, 12, and 24.

The greatest common divisor is 6.

Following the design recipe, we start with a contract, a purpose statement, and a header:

```
;; gcd: N[ $\Rightarrow$  1] N[ $\Rightarrow$  1]  $\rightarrow$  N
;; to find the greatest common divisor of n
and m
(define (gcd n m)
  ...)
```

The contract specifies the precise inputs: natural numbers that are greater or equal to 1 (not 0).

Now we need to make a decision whether we want to pursue a design based on structural or on generative recursion. Since the answer is by no means obvious, we develop both.



REMEMBER

The structural version, we must consider which input the function should process: n , m , or both.

A moment's consideration suggests that what we really need is a function that starts with the smaller of the two and outputs the first number smaller or equal to this input that evenly divides both n and m .

```
;; gcd-structural: N[ $\Rightarrow$  1] N[ $\Rightarrow$  1]  $\rightarrow$  N
;; to find the greatest common divisor of n
and m
;; structural recursion using data definition
of N[ $\Rightarrow$  1] (define (gcd-structural n m) (local
  ((define (first-divisor- $\Leftarrow$  i)
    (cond
      [(= i 1) 1]
      [else (cond
        [(and (= (remainder n i) 0)
              (= (remainder m i) 0))
         i]
        [else (first-divisor- $\Leftarrow$  (-
          i 1))]))]))
    (first-divisor- $\Leftarrow$  (min m
      n)))))
```

The conditions “evenly divisible” have been encoded as $(= (\text{remainder } n \ i) \ 0)$ and $(= (\text{remainder } m \ i) \ 0)$. The two ensure that i divides n and m without a remainder. Testing `gcd-structural` with the examples shows that it finds the expected answers. Although the design of `gcd-structural` is rather straightforward, it is also naive. It simply tests for every number whether it divides both n and m evenly and returns the first such number. For small natural numbers, this process works just fine.

Consider the following example, however:

```
(gcd-structural 101135853 45014640)
```

The result is 177 and to get there `gcd-structural` had to compare 101135676, that is, $101135853 - 177$, numbers. This is a large effort and even reasonably fast computers spend several minutes on this task.

Exercises

Enter the definition of `gcd-structural` into the Definitions window and evaluate `(time (gcd-structural 101135853 45014640))` in the Interactions window.

After testing `gcd-structural` conduct the performance tests in the Full Scheme language (without debugging), which evaluates expressions faster than the lower language levels but with less protection. Add `(require-library “core.ss”)` to the top of the Definitions window. Have some reading handy!

Since mathematicians recognized the inefficiency of the “structural algorithm” a long time ago, they studied the problem of finding divisors in more depth. The essential insight is that for two natural numbers larger and smaller, their greatest



common divisor is equal to the greatest common divisor of smaller and the remainder of larger divided into smaller.

Here is how we can put this insight into equational form:

```
(gcd larger smaller)
= (gcd smaller (remainder larger smaller))
```

Since (remainder larger smaller) is smaller than both larger and smaller, the right-hand side use of gcd consumes smaller first.

Here is how this insight applies to our small example:

- The given numbers are 18 and 24.
- According to the mathematicians' insight, they have the same greatest common divisor as 18 and 6.
- And these two have the same greatest common divisor as 6 and 0.

And here we seem stuck because 0 is nothing expected. But, 0 can be evenly divided by every number, so we have found our answer: 6.

Working through the example not only explains the idea but also suggests how to discover the case with a trivial solution. When the smaller of the two numbers is 0, the result is the larger number.

Putting everything together, we get the following definition:

```
;; gcd-generative: N[≧ 1] N[≧1] → N
;; to find the greatest common divisor of n
and m
;; generative recursion: (gcd n m) = (gcd n
(remainder m n)) if (≤ m n)
(define (gcd-generative n m)
  (local ((define (clever-gcd larger smaller)
    (cond
      [(= smaller 0) larger]
      [else (clever-gcd smaller
        (remainder larger smaller))]))))
    (clever-gcd (max m n) (min m n))))
```

The local definition introduces the workhorse of the function: clever-gcd, a function based on generative recursion. Its first line discovers the trivially solvable case by comparing smaller to 0 and produces the matching solution. The generative step uses smaller as the new first argument and (remainder



Algorithm is a self-contained step-by-step set of operations to be performed. Algorithms perform calculation, data processing, and/or automated reasoning tasks.

larger smaller) as the new second argument to `clever-gcd`, exploiting the above equation. If we now use `gcd-generative` with our complex example from above:

```
(gcd-generative 101135853 45014640)
```

we see that the response is nearly instantaneous. A hand-evaluation shows that `clever-gcd` recurs only nine times before it produces the solution: 177. In short, generative recursion has helped find us a much faster solution to our problem.

1.4.3. Process

For the maximum sub-array problem, if you did not know any better, you would probably implement a solution that Analyses every possible subarray, and returns the one with the maximum sum:

```
class Array
  def sum
    result = 0
    self.each do |i|
      result+=i
    end
    return result
end

#test every sub array - brute force!
def max_sub_array_order_ncubed
  left_index = 0
  right_index = 0
  max_value = self [left_index.. right_index].sum
  for i in (0..self.length)
    for j in (i..self.length
      this_value = self[i..j].sum
      if (this_value > max_value)
        max_value = this_value
        left_index=i
        right_index=j
      end
    end
  end
  return self[left_index..right_index]
end
```

If you were a bit more clever, you might notice that `self[i..j].sum` is equal to `self[i..(j-1)].sum + self[j]` in the innermost loop (the `sum` method itself), and use an accumulator there as opposed to calculating it each time. That takes you down from n^3 to n^2 time.



But there are (at least) two other ways to solve this problem:

- A divide and conquer approach that uses recursion and calculates the left and right maximum contiguous subarrays (MCS), along with the MCS that contains the right-most element in the left side and the left-most element in the right side. It compares the three and returns the one with the maximum sum. This gets us to $O(n \log n)$ time.
- An approach you will call “expanding sliding window.” If memory serves me correct, this aptly describes it or was the way a professor of mine described it. In any case, the “expanding sliding window” can do it in one pass ($O(n)$ time), at the cost of a few more variables.

1.5. SORTING OF ALGORITHMS

It is not always possible to say that one algorithm is better than another, as relative performance can vary depending on the type of data being sorted. In some situations, most of the data are in the correct order, with only a few items needing to be sorted.

In other situations the **data** are completely mixed up in a random order and in others the data will tend to be in reverse order.

Different algorithms will perform differently according to the data being sorted. Four common algorithms are the exchange or bubble sort, the selection sort, the insertion sort and the quick sort.

The selection sort is a good one to use with students. It is intuitive and very simple to Programme. It offers quite good performance, its particular strength being the small number of exchanges needed.

For a given number of data items, the selection sort always goes through a set number of comparisons and exchanges, so its performance is predictable.

Procedure SelectionSort (d: DataArrayType; n: integer) {n is the number of elements}



Data is a set of values of qualitative or quantitative variables.

```

for k = 1 to n-1 do begin
  small = k
  for j = k+1 to n do
    if d[j] < d[small] then small = j
  {Swap elements k and small}
  Swap(d, k, small)
end

```

1.5.1. Exchange (Bubble) Sort

Element	1	2	3	4	5	6	7	8
Data	27	63	1	72	64	58	14	9
1st pass	27	1	63	64	58	14	9	72
2nd pass	1	27	63	58	14	9	64	72
3rd pass	1	27	58	14	9	63	64	72...

The first two data items (27 and 63) are compared and the smaller one placed on the left hand side. The second and third items (63 and 1) are then compared and the smaller one placed on the left and so on.

After all the data has been passed through once, the largest data item (72) will have “bubbled” through to the end of the list. At the end of the second pass, the second largest data item (64) will be in the second last position. For n data items, the process continues for $n-1$ passes, or until no exchanges are made in a single pass.

1.5.2. Insertion Sort

Element	1	2	3	4	5	6	7	8
Data	27	63	1	72	64	58	14	9
1st pass	27	63	1	72	64	58	9	14
2nd pass	27	63	1	72	64	9	14	58
3rd pass	27	63	1	72	9	14	58	64...

The insertion sort starts with the last two elements and creates a correctly sorted sub-list, which in the example contains 9 and 14. It then looks at the next element (58) and inserts it into the sub-list in its correct position.

It takes the next element (64) and does the same, continuing until the sub-list contains all the data.

1.5.3. Selection Sort

Element	1	2	3	4	5	6	7	8
Data	27	63	1	72	64	58	14	9
1st pass	1	63	27	72	64	58	14	9
2nd pass	1	9	27	72	64	58	14	63
3rd pass	1	9	14	72	64	58	27	63...

The selection sort marks the first element (27). It then goes through the remaining data to find the smallest number (1). It swaps this with the first element and the smallest element is now in its correct position. It then marks the second element (63) and looks through the remaining data for the next smallest number (9). These two numbers are then swapped. This process continues until $n-1$ passes have been made.

1.5.4. Quick Sort

Element	1	2	3	4	5	6	7	8
Data	27	63	1	72	64	58	14	9
1st pass	1	9	63	72	64	58	14	27
2nd pass	1	9	14	27	64	58	72	63
3rd pass	1	9	14	27	58	63	72	64
4th pass	1	9	14	27	58	63	64	72 sorted!

The quick sort takes the last element (9) and places it such that all the numbers in the left sub-list are smaller and all the numbers in the right sub-list are bigger. It then quick sorts the left sub-list ({1}) and then quick sorts the right sub-list. This is a recursive algorithm, since it is defined in terms of itself. This reduces the complexity of Programming it, however it is the least intuitive of the four algorithms.

SUMMARY

- An algorithm is a procedure or formula for solving a problem, based on conducting a sequence of specified actions.
- An encryption algorithm transforms data according to specified actions to protect it.
- Each algorithm is a list of well-defined instructions for completing a task. Starting from an initial state, the instructions describe a computation that proceeds through a well-defined series of successive states, eventually terminating in a final ending state.
- An algorithm is a precise list of precise steps, the order of computation will always be critical to the functioning of the algorithm.
- In computer systems, an algorithm is basically an instance of logic written in software by software developers to be effective for the intended “target” computer(s), in order for the software on the target machines to *do something*.

KNOWLEDGE CHECK

1. For the heap sort, access to nodes involves simple operations.
 - a. arithmetic
 - b. binary
 - c. algebraic
 - d. logarithmic
2. A (an) is a left-complete binary tree that conforms to the heap order
 - a. binary tree
 - b. binary search tree
 - c. heap
 - d. array
3. Which of the following data structure is linear data structure?
 - a. Trees
 - b. Graphs
 - c. Arrays
 - d. None of above



4. **When determining the efficiency of algorithm, the space factor is measured by**
 - a. Counting the maximum memory needed by the algorithm
 - b. Counting the minimum memory needed by the algorithm
 - c. Counting the average memory needed by the algorithm
 - d. Counting the maximum disk space needed by the algorithm
5. **A linked-list is a dynamic structure**
 - a. True
 - b. False

REVIEW QUESTIONS

1. Discuss about formalization and termination.
2. Explain the characteristics of an algorithm.
3. Discuss about computer algorithms.
4. Write short note on master's theorem.
5. What do you understand by data analysis and design?
6. Write short note on:
 - a. Exchange (Bubble) Sort
 - b. Quick Sort

CHECK YOUR RESULT

1. (a) 2. (c) 3. (c) 4. (1) 5. (a)

REFERENCES

1. Techtarget, <http://whatis.techtarget.com/definition/algorithm>
2. Tutorialspoint, https://www.tutorialspoint.com/data_structures_algorithms/algorithms_basics.htm
3. Study.com, <http://study.com/academy/lesson/what-is-a-computer-algorithm-design-examples-optimization.html>

MATHEMATICAL ASPECTS AND ANALYSIS OF ALGORITHMS

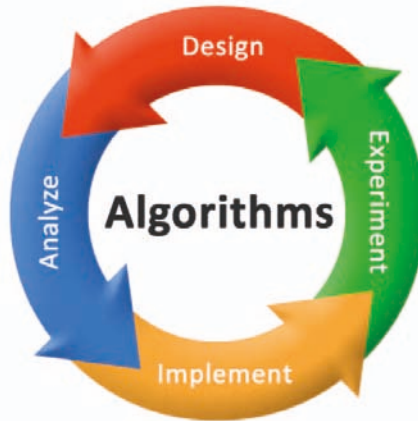
LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Discuss the mathematical analysis of non-recursive algorithm
2. Explain the mathematical analysis of recursive algorithm
3. Describe the empirical analysis of algorithms

INTRODUCTION

You have learnt the concept of analysis framework. Now this framework is applied to certain algorithms and we can compute their efficiency. For simplicity we have categorized the algorithms in two broad categories as: non recursive and recursive algorithms. You will mainly discuss how to analyze recursive and non-recursive algorithms mathematically. This discussion is well supported by appropriate algorithmic examples.



2.1 MATHEMATICAL ANALYSIS OF NON RECURSIVE ALGORITHM

First of all we will see the general plan for analyzing the **efficiency** of non-recursive algorithms. This plan tells us the steps to be followed while analyzing such algorithms.

2.2.1 General Plan for Analyzing Efficiency of Non Recursive Algorithms

- Decide the input size based on parameter n .
- Identify algorithm's basic operation(s).
- Check how many times the basic operation is executed. Then find whether the execution of basic operation depends upon the input size n . Determine worst, average, and best cases for input of size n . If the basic operation depends upon worst case, average case and best case then that has to be analyzed separately.
- Set up a sum for the number of times the basic operation is executed.
- Simplify the sum using standard formula and rules

As mentioned in the step five we have to simplify the sum value using some standard formula and rules.

Summation Formula and rules

$$1. \quad \sum_{i=1}^n 1 = 1 + 1 + 1 + 1 + \dots + 1 = n \in \Theta(n)$$

$$2. \sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \in \Theta(n^2)$$

$$3. \sum_{i=1}^n i^k = 1 + 2^k + 3^k + \dots + n^k = \frac{n^{k+1}}{k+1} \in \Theta(n^{k+1})$$

$$4. \sum_{i=1}^n i^k = 1 + a + \dots + a^n = \frac{a^{k+1} - 1}{a - 1} \in \Theta(a^n)$$

$$5. \sum_{i=1}^n (a_i \pm b_i) = \sum_{i=1}^n a_i \pm \sum_{i=1}^n b_i$$

$$6. \sum_{i=1}^n c a_i = c \sum_{i=1}^n a_i$$

$$7. \sum_{i=1}^n 1 = -k + 1$$

Where n and a are some upper lower limits.



Efficiency is the (often measurable) ability to avoid wasting materials, energy, efforts, money, and time in doing something or in producing a desired result.

Examples

Now we will discuss a few non recursive algorithms. We will find the efficiency of these algorithms with the help of general plan.

Finding the element with maximum value in a given array

```
Algorithm Max_Element(A[0...n-1])
//Problem Description: This algorithm is for finding the
//maximum value element from the array
//Input: array A[0..n-1]
//Output: Returns the largest element from array

Max_value ← A[0]
for i ← 1 to n-1 do
{
    if (A[i] > Max_value) then
        Max_value ← A[i]
}
return Max_value
```

2.1.2 Mathematical Analysis

Step 1: The input size is n i.e. total number of elements in array.

Step 2: The basic operation is comparison in loop for finding larger value.

Step 3: The comparison is executed on each repetition of the loop. As the comparison is made for each value of n there is no need to find best case, worst case and average case analysis.

Step 4: Let, $C(n)$ be the number of times the comparison is executed. The algorithm makes comparison each time the loop executes. That means with each new value of i the comparison is made. Hence for $i=1$ to $n - 1$ times the comparison is made.

Therefore we can formulate $C(n)$ as-

$C(n)$ = one comparison made for each value of i

Step 5: Let us simplify the sum

$$c(n) = \sum_{i=1}^{n-1} 1$$

$$c(n) = n - 1 \in \Theta(n) \quad \left[\text{Using the rule } \sum_{i=1}^{n-1} 1 = n \in \Theta(n) \right]$$

Thus the efficiency of above algorithm is $\Theta(n)$

Finding whether all the elements in an array are distinct. This problem is called element uniqueness problem

```

Algorithm Unique_Element(A[0..n-1])
//Problem Description: This algorithm finds whether array
//elements are distinct or not
//Input: Array A[0..n-1]
//Output: Returns False if elements are not distinct. If
//all the elements of an array are distinct then it returns
//True
for i ← 0 to n - 2 do
{
    for j ← i+1 to n-1 do
    {
        if (A[i]=A[j]) then
            return False
    }
}
return True
  
```

2.1.3 Analysis of Non-recursive Algorithms

- We just count the number of basic operations.
- Loops will become series sums
- So we'll need some series formulas

Example: Maximum Element

Algorithm Max Element($A[0...n-1]$)

```

maxval ← A[0]

for i ← 1 to n-1 do
    if A[i] > maxval then maxval ← A[i]

return maxval

```

What is the problem size? n

Most frequent operation? Comparison in the for loop

Depends on worst case or best case? No, has to go through the entire array

$C(n)$ = number of comparisons

$$C(n) = \sum_{i=1}^{n-1} 1 = n-1 \in \Theta(n)$$

Analysis Procedure

1. Decide on problem size specification
2. Identify basic operation
3. Check worst, best, average case
4. Count: set up the sum
5. Evaluate or determine order of sum

Series Rules and Formulas

Multiplication of a Series: $\sum_{i=l}^u c a_i = c \sum_{i=l}^u a_i$

Sum of two sequences: $\sum_{i=l}^u (a_i + b_i) = \sum_{i=l}^u a_i + \sum_{i=l}^u b_i$

Sum of constant sequences: $\sum_{i=l}^u 1 = u - l + 1$

Sum of linear sequences: $\sum_{i=0}^n i = n(n+1)/2$ = length of sequence times the average of the first and last elements

Example: Uniqueness

Progress through the sequence from 0 to $n-2$ checking the remainder with equality
 Algorithm *UniqueElements*($A[0...n-1]$)

```

    for  $i \leftarrow 0$  to  $n-2$  do
        for  $j \leftarrow i+1$  to  $n-1$  do
            if  $A[i] = A[j]$  return false
    return true
    
```

1. Problem size? n
2. Basic operation? if-test
3. Worst and best case are different. Best case is when the first two elements are equal then $\Theta(n)$
 Worst case is if array elements are unique then all sequences of the for loops are executed

4. The sum:

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

5. Solve

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} [n-1-i] = \sum_{k=n-1}^1 k \text{ where } k = n - i - 1$$

$$C_{\text{worst}}(n) = \sum_{k=1}^{n-1} k = (n-1)(n-1+1)/2 = n(n-1)/2 \in \Theta(n^2)$$

Note for a unique array there is minimal of $n(n-1)/2$ comparisons. Is this necessary, is there a better algorithm?

Yes we could pre-sort.

Another Example: Binary Length

The number of binary digits in a number

Algorithm *Binary*(n)

```

    count  $\leftarrow 1$ 
    while  $n > 1$  do
    
```

count++

$n \leftarrow \text{floor}(n/2)$

return *count*

1. Problem size? integer, n
2. Basic operation? comparison in the while loop
3. Worst and best case are the same.
4. The sum:

How many times is the while loop executed?

approximately $\lg(n)$, exactly $\lg(n) + 1$ because it must fail once

$$C(n) = \sum_{i=1}^{\lg(n)+1} 1$$

5. Solve

$$C(n) = \lg(n) + 1 - 1 + 1 \in \Theta(\lg(n))$$

2.2 MATHEMATICAL ANALYSIS OF RECURSIVE ALGORITHM

First of all we will see the general plan for analyzing the efficiency of recursive algorithms. This plan tells us the steps to be followed while analyzing such algorithms.

2.2.1 General Plan for Analyzing Efficiency of Recursive

- Decide the input size based on parameter n .
- Identify algorithm's basic operation(s).
- Check how many times the basic operation is executed. Then find whether the execution of basic operation depends upon the input size n . Determine worst, average, and best cases for input of size n . If the basic operation depends upon worst case, average case and best case then that has to be analyzed separately.
- Set up the recurrence relation with some initial condition and expressing the basic operation.
- Solve the recurrence or at least determine the order of growth. While solving the recurrence we will use the forward and backward substitution method. And then correctness of formula can be proved with the help of mathematical induction method.

Let us analyze some recursive **algorithms** mathematically.

*Examples 1:**Computing factorial of some number n*

The factorial of some number can be obtained by performing repeated multiplication.

For instance: If $n=5$ then

Step 1 : $n! = 5!$

Step 2 : $4! * 5$

Step 3 : $3! * 4 * 5$

Step 4 : $2! * 3 * 4 * 5$

Step 5 : $1! * 2 * 3 * 4 * 5$

Step 6 : $0! * 1 * 2 * 3 * 4 * 5$

Step 7 : $1 * 1 * 2 * 3 * 4 * 5$

as $0! = 1$



Algorithm is a self-contained step-by-step set of operations to be performed.

The above mentioned steps can be written in pseudo-code form as:

Algorithm Factorial (n)

//Problem Description : This algorithm computes n! using

//recursive function

//Input : A non negative integer n

//Output : returns the factorial value.

if (n=0)

return 1

else

return Factorial (n-1)*n

2.2.2 Mathematical Analysis

Step 1: The factorial algorithm works for input size n.

Step 2: The basic operation in computing factorial is multiplication.

Step 3: The recursive function call can be formulated as

$$F(n) = F(n - 1) * n$$

where $n > 0$

Then the basic operation multiplications is given as $M(n)$. And $M(n)$ is multiplication count to compute factorial (n) .

$$M(n) = M(n-1) + 1$$

Step 4: In step 3 the recurrence relation is obtained.

$$M(n) = M(n-1) + 1$$

Now we will solve recurrence using

■ Forward substitution

$$M(1) = M(0) + 1$$

$$M(2) = M(1) + 1 = 1 + 1 = 2$$

$$M(3) = M(2) + 1 = 2 + 1 = 3$$

■ Backward substitution

$$\begin{aligned}
 M(n) &= \underbrace{M(n-1)}_{\downarrow} + 1 \\
 &= \left[\underbrace{M(n-2)}_{\downarrow} + 1 \right] + 1 = M(n-2) + 2 \\
 &= [M(n-3) + 1] + 1 + 1 = M(n-3) + 3
 \end{aligned}$$

From the substitution methods we can establish a general formula as:

$$M(n) = M(n-i) + i$$

Now let us prove correctness of this formula using mathematical induction as follows.

Prove $M(n) = n$ by using mathematical induction

Basis: Let, $n = 0$ then

$$M(n) = 0$$

$$\text{i.e. } M(0) = 0 = n$$

Induction: if we assume $M(n-1) = n-1$ then

$$M(n) = M(n-1) + 1$$

$$= n - 1 + 1$$

$$= n$$

i.e: ----- $M(n) = n$

Thus the time complexity of factorial function is $\Theta(n)$

2.2.3 Analysis of Recursive Algorithms

What is a recursive algorithm?

Example: Factorial

$n! = 1 \cdot 2 \cdot 3 \dots n$ and $0! = 1$ (called initial case)

So the recursive definition $n! = n \cdot (n-1)!$

Algorithm $F(n)$

if $n = 0$ then return 1 // base case

else $F(n-1) \cdot n$ // recursive call

Basic operation? Multiplication during the recursive call

Formula for multiplication

$$M(n) = M(n-1) + 1$$

is a recursive formula too. This is typical.

We need the initial case which corresponds to the base case

$$M(0) = 0$$

There are no multiplications

Solve by the method of *backward substitutions*

$$M(n) = M(n-1) + 1$$

$$= [M(n-2) + 1] + 1 = M(n-2) + 2 \text{ substituted } M(n-2) \text{ for } M(n-1)$$

$$= [M(n-3) + 1] + 2 = M(n-3) + 3 \text{ substituted } M(n-3) \text{ for } M(n-2)$$

... a pattern evolves

$$= M(0) + n$$

$$= n$$

Not surprising!

Therefore $M(n) \in \Theta(n)$

Procedure for Recursive Algorithm

1. Specify problem size
2. Identify basic operation
3. Worst, best, average case
4. Write recursive relation for the number of basic operation. Don't forget the initial conditions (IC)
5. Solve recursive relation and order of growth

Example: Tower Hanoi

Explain the problem using figure

Demo and show recursion

1. Problem size is n , the number of discs
2. The basic operation is moving a disc from rod to another
3. There is no worst or best case
4. Recursive relation for moving n discs

$$M(n) = M(n-1) + 1 + M(n-1) = 2M(n-1) + 1$$

$$\text{IC: } M(1) = 1$$

5. Solve using backward substitution

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \\ &= 2[2M(n-2) + 1] + 1 = 2^2M(n-2) + 2 + 1 \\ &= 2^2[2M(n-3) + 1] + 2 + 1 = 2^3M(n-3) + 2^2 + 2 + 1 \end{aligned}$$

...

$$M(n) = 2^i M(n-i) + \sum_{j=0}^{i-1} 2^j = 2^i M(n-i) + 2^i - 1$$

...

$$\begin{aligned} M(n) &= 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 = 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 \\ &= 2^n - 1 \end{aligned}$$

$$M(n) \in \Theta(2^n)$$

Terrible: Can we do it better?

Where did the exponential term come from? Because two recursive calls are made. Suppose three recursive calls are made, what is the order of growth.

Example: Binary Representation**Algorithm** $BinRec(n)$ if $n = 1$ then return 1else return $BinRec(\text{floor}(n/2)) + 1$

1. Problem size is n
2. Basic operation is the addition in the recursive call
3. There is no difference between worst and best case
4. Recursive relation including initial conditions

**REMEMBER**

Especially if the problem size is measured by the level of the recursive tree and the operation count is total number of nodes.

$$A(n) = A(\text{floor}(n/2)) + 1$$

$$\text{IC } A(1) = 0$$

5. Solve recursive relation

The division and floor function in the argument of the **recursive** call makes the analysis difficult.

We could make the variable substitution, $n = 2^k$, could get rid of the definition, but the substitution skips a lot of values for n .

Smoothness Rule

$T(n)$ eventually non-decreasing and $f(n)$ be smooth {eventually non-decreasing and $f(2n) \in \Theta(f(n))$ }

if $T(n) \in \Theta(f(n))$ for n powers of b then $T(n) \in \Theta(f(n))$ for all n .

Works for O and Ω .

substitute $n = 2^k$ (also $k = \lg(n)$)

$$A(2^k) = A(2^{k-1}) + 1 \text{ and IC } A(2^0) = 0$$

$$A(2^k) = [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2$$

$$= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3$$

...

$$= A(2^{k-i}) + i$$

...

$$= A(2^{k-k}) + k$$

$$A(2^k) = k$$

Substitute back $k = \lg(n)$

$$A(n) = \lg(n) \in \Theta(\lg n)$$



Recursion occurs
when a thing is
defined in terms of
itself or of its type.

2.2.4 Explicit Formula for n^{th} Fibonacci number

The algorithm for Fibonacci series always requires n number to compute n^{th} value. Hence input size of this algorithm is n . And the basic operation in this algorithm is to perform addition of previous two numbers. But obtaining formula for n^{th} Fibonacci number with recurrence relation using forward and backward substitution method is complex.

Example: Fibonacci Numbers Sequence

A classic example for more elaborate recursion relations.

Fibonacci Sequence: 0, 1, 1 2, 3, 5, 8, 13, 21, 34,

Fibonacci (1202) proposed for the growth of rabbits

Can be defined by the simple recurrence

$$F(n) = F(n-1) + F(n-2) \text{ for } n > 1$$

IC: $F(0) = 0$ and $F(1) = 1$ (why do we need two ICs)

Homogenous second-order linear recurrence with constant coefficients

$$ax(n) + bx(n-1) + cx(n-2) = 0$$

**REMEMBER**

Hence we will use homogeneous second order linear recurrence with constant coefficients to obtain n th Fibonacci number. The linear second order recurrence with constant coefficient is a special class of recurrence in which solution is neither obtained by forward or backward substitution.

Homogenous because the recurrence equals zero.

Why second-order linear? Substitute the proposed solution $x(n) = r^n$

$$a r^n + b r^{n-1} + c r^{n-2} = 0$$

divide by r^n

$a + b r + c r^2 = 0$, *characteristic equation* is a second order polynomial.

The real roots are solutions

$$x(n) = \alpha r_1^n + \beta r_2^n$$

α and β are determined from the

Apply to Fibonacci Recursion

$F(n) - F(n-1) - F(n-2) = 0$, homogenous second order with constant coefficients

$r^2 - r - 1 = 0$, characteristic equation

$$r_{1,2} = (1 \pm \sqrt{5})/2 = \phi \text{ or } \phi^* \{ \phi = (1+\sqrt{5})/2 \text{ and } \phi^* = (1-\sqrt{5})/2 \}$$

The general form of the solution

$$F(n) = \alpha \phi^n + \beta \phi^{*n} \text{ where } \alpha \text{ and } \beta \text{ are unknowns}$$

Using the ICs

$$\alpha + \beta = 0$$

$$\phi \alpha + \phi^* \beta = 1$$

Solve by substituting the first equation into the second, get $\alpha = 1/\sqrt{5}$ and $\beta = -1/\sqrt{5}$

So

$$F(n) = 1/\sqrt{5} (\phi^n - \phi^{*n}) = \phi^n / \sqrt{5} \text{ rounded to the nearest integer}$$



Example: Recursive Algorithm for Fibonacci Numbers**Algorithm** $F(n)$ **if** $n \leq 1$ **then return** n **else return** $F(n-1) + F(n-2)$

1. Problem size is n , the sequence number for the Fibonacci number
2. Basic operation is the sum in recursive call
3. No difference between worst and best case
4. Recurrence relation

$$A(n) = A(n-1) + A(n-2) + 1$$

$$\text{IC: } A(0) = A(1) = 0$$

or

$$A(n) - A(n-1) - A(n-2) = 1, \text{ Inhomogeneous recurrences because of the } 1$$

In general solution to the inhomogeneous problem is equal to the sum of solution to homogenous problem plus solution only to the inhomogeneous part. The undetermined coefficients of the solution for the homogenous problem are used to satisfy the IC.

In this case $A(n) = B(n) + I(n)$ where

$A(n)$ is solution to complete inhomogeneous problem

$B(n)$ is solution to homogeneous problem

$I(n)$ solution to only the inhomogeneous part of the problem

We guess at $I(n)$ and then determine the new IC for the homogenous problem for $B(n)$

For this problem the correct guess is $I(n) = 1$

substitute $A(n) = B(n) - 1$ into the recursion and get

$$B(n) - B(n-1) - B(n-2) = 0 \text{ with IC } B(0) = B(1) = 1$$

The same as the relation for $F(n)$ with different IC

We do not really need the exact solution; We can conclude

$$A(n) = B(n) - 1 = F(n+1) - 1 \in \Theta(\varphi^n), \text{ exponential}$$

There is the Master Theorem that gives the asymptotic limit for many common problems.

Iterative Algorithm**Algorithm** $Fib(n)$ $F[1] \leftarrow 0; F[1] \leftarrow 1$ **for** $i \leftarrow 2$ **to** n **do** $F[i] \leftarrow F[i-1] + F[i-2]$ **return** $F[n]$ Order of growth is $\Theta(n)$.

Why is it so much better than the recursive algorithm? Draw the recursive tree.

**DID YOU
KNOW?**

Since the 1950s computers have seen dramatic increases in both the available computational power and in the available amount of memory, so current acceptable levels would have been unacceptable even 10 years ago.

2.3 EMPIRICAL ANALYSIS OF ALGORITHMS

Empirical algorithmic is the area within computer science that uses empirical methods to study the behaviour of algorithms. It can be used in the analysis of algorithms. Methods from empirical algorithmic complement theoretical methods for the analysis of algorithms. Through the principled application of empirical methods, particularly from statistics, it is often possible to obtain insights into the behaviour of algorithms that are inaccessible to theoretical analysis, in particular, high-performance heuristic algorithms for hard combinatorial problems. Empirical methods can also be used to achieve substantial improvements in algorithmic efficiency.

There are two main branches of empirical algorithmic: the first (known as empirical analysis) deals with the analysis and characterization of the behaviour of algorithms, and the second (known as algorithm design or algorithm engineering) is focused on empirical methods for improving the performance of algorithms. The former uses mostly techniques and tools from statistics, while the latter is based on approaches from statistics, machine learning and optimization.

You have discussed the mathematical analysis of algorithms for both recursive and non-recursive algorithms. The analysis of algorithms can be done in two ways –

- Mathematical analysis
- Empirical analysis

We have already discussed the mathematical analysis. We can analyze an algorithm by another method called Empirical analysis. Empirical analysis of algorithm means observing behavior of an algorithm for certain set of input. In empirical analysis actual program is written for the corresponding algorithm and with the help of some suitable input set, the algorithm is analyzed. Let us discuss the steps to be followed while analyzing algorithm empirically.

2.3.1 General Plan for Empirical Analysis of Algorithm

- Understand the purpose of experiment of given algorithm.
 - Decide the efficiency metric M . Also decide the measurement unit. For example operation's count vs. time.
 - Decide on characteristics of the input.
 - Create a program for implementing the algorithm. This program is ready for experiment.
 - Generate a sample of input.
 - Run the algorithm for some set of input sample. Record the results obtained.
 - Analyze the resultant data.
- Let us discuss each step in detail –

1. Understand the purpose of experiment of given algorithm.

The algorithm is analyzed empirically for following reasons:

- The accuracy of an algorithm needs to be checked
- For the same problem there can be several algorithms available and one can compare the efficiencies of different algorithms for same problem. Or one can compare different implementations for the same problem.
- One can decide the efficiency of particular algorithm on particular machine.
- The hypothesis for algorithm's efficiency class can be developed

2. Decide the efficiency metric M . Also decide the measurement unit.

The efficiency of an algorithm can be measured by following different methods –

- Insert a counter in the algorithm in order to count the number of times the basic operation is executed. This is a straightforward method of counting an efficiency of algorithm.

**REMEMBER**

Algorithmic visualization is a technique in which images are used to convey the information about the algorithm.

For example: If we write a function for calculating sum of n number in an array then we can find the efficiency of that function by inserting a frequency count. The frequency count is a count that denotes how many times the particular statement is executed.

```

Line 1: int sum_element(int a[10],int n)
Line 2: {
Line 3:   int i, sum=0;

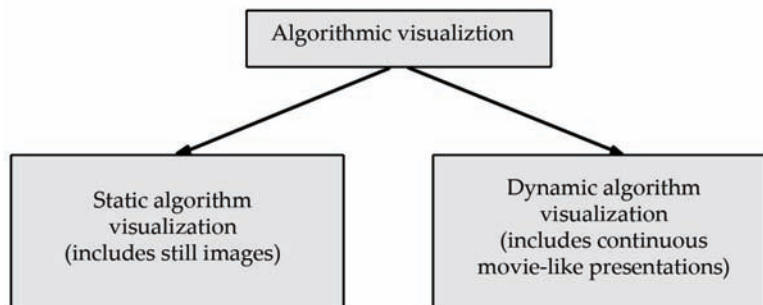
Line 4:   for(i=0;i<n;i++)
Line 5:   {
Line 6:     sum=sum+a[i];
Line 7:   }
Line 8:   return sum;
Line 9: }
```

2.3.2 Algorithm Visualization

We have studied that algorithm can be analyzed mathematically as well as empirically. There is third way also to analyses the algorithm and i.e. algorithmic visualization.

In algorithmic visualization the operations should be illustrated with some visual components or by some animations. The behavior of algorithm on different inputs should be highlighted. Not only this, it is always effective to represent the comparison made with different algorithms solving same problem.

There are two approaches of algorithmic visualization.



The dynamic representation of algorithmic visualization is called algorithm animation. In 1981, a 30-minute color sound film entitled “sorting out sorting” is created at University of Toronto. From this film the inspiration is taken for representing



the algorithm visually. Creating such systems is difficult task but once it is developed the further analysis of algorithm becomes simplified. Let us understand algorithmic visualization with some examples -

Example 1:

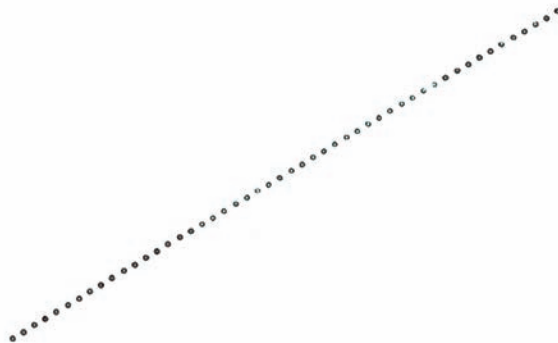
The Bubble sort is illustrated here. The top three elements have occupied their proper positions and highlighted elements are to be swapped now.

Example 2:

The following snapshot shows insertion sort method for random input set of 50. The time shown in Msecs also shows the time taken by the algorithm to execute.



Similarly the elements can be sorted in ascending order by insertion sort method. It is illustrated by following snapshot.



Features of algorithmic visualization

While showing the algorithm by animations, the following features should be adopted by the developed system:

1. The visualization should be consistent.
2. It should be interactive so that any ordinary user should understand it easily.
3. It should be clear and concise.
4. There should be user friendliness with the developed system.
5. While developing such visualization first the developer should understand the knowledge level of the user and then accordingly the system should be designed.
6. Emphasis should be on visual component rather than producing textual information.
7. Such systems should keep the user interested.
8. The symbolic and iconic representations should be incorporated.
9. The algorithmic analysis as well as comparison with different algorithms solving the same problem should be included in such animated systems.
10. The execution history should be included in such systems.

Applications of Algorithm Visualization

- *Research:* Many uncovered features of algorithm can be effectively shown by visual systems and researcher can get the benefit of such systems for further development and studies.
- *2. Education:* Students can learn algorithms very easily using animated systems. Various visual components included in the animated systems help the student to understand and analyze the algorithm in simplest manner. For example different sorting algorithms can be analyzed and understood with the help of bar charts or line graphs.

SUMMARY

- Now this framework is applied to certain algorithms and we can compute their efficiency.
- First of all we will see the general plan for analyzing the efficiency of recursive algorithms. This plan tells us the steps to be followed while analyzing such algorithms.
- The algorithm for Fibonacci series always requires n number to compute n th value. Hence input size of this algorithm is n .
- Hence we will use homogeneous second order linear recurrence with constant coefficients to obtain n^{th} Fibonacci number.
- Empirical algorithmic is the area within computer science that uses empirical methods to study the behaviour of algorithms.
- There are two main branches of empirical algorithmic: the first (known as empirical analysis) deals with the analysis and characterization of the behaviour of algorithms, and the second (known as algorithm design or algorithm engineering) is focused on empirical methods for improving the performance of algorithms.
- Empirical algorithmic is the area within computer science that uses empirical methods to study the behaviour of algorithms.
- There are two main branches of empirical algorithmic: the first (known as empirical analysis) deals with the analysis and characterization of the behaviour of algorithms, and the second (known as algorithm design or algorithm engineering) is focused on empirical methods for improving the performance of algorithms.

KNOWLEDGE CHECK

1. Two main measures for the efficiency of an algorithm are
 - a. Processor and memory
 - b. Complexity and capacity
 - c. Time and space
 - d. Data and space
2. The time factor when determining the efficiency of algorithm is measured by
 - a. Counting microseconds
 - b. Counting the number of key operations

- c. Counting the number of statements
 - d. Counting the kilobytes of algorithm
3. **The space factor when determining the efficiency of algorithm is measured by**
- a. Counting the maximum memory needed by the algorithm
 - b. Counting the minimum memory needed by the algorithm
 - c. Counting the average memory needed by the algorithm
 - d. Counting the maximum disk space needed by the algorithm
4. **Which of the following case does not exist in complexity theory**
- a. Best case
 - b. Worst case
 - c. Average case
 - d. Null case
5. **The Worst case occur in linear search algorithm when**
- a. Item is somewhere in the middle of the array
 - b. Item is not in the array at all
 - c. Item is the last element in the array
 - d. Item is the last element in the array or is not there at all

REVIEW QUESTIONS

- Explain the general plan for analyzing efficiency of non-recursive algorithms.
- Describe the analysis of non-recursive algorithms.
- Give the overview on analysis of recursive algorithms
- Explain the general plan for empirical analysis of algorithm.
- What do you understand by algorithm visualization?

CHECK YOUR RESULT

1. (c) 2. (b) 3. (a) 4. (d) 5. (d)

REFERENCES

1. <http://www.csl.mtu.edu/cs4321/www/Lectures/Lecture%204%20-%20Analysis%20of%20Recursive%20Algorithms.htm>
2. <http://www.csl.mtu.edu/cs4321/www/Lectures/Lecture%203%20-%20Analysis%20of%20Nonrecursive%20Algorithms.htm>
3. [http://www.allsyllabus.com/aj/note/Computer_Science/Analysis_and_Design_of_Algorithms/Unit2/Mathematical%20analysis%20\(Time%20Efficiency\)%20of%20recursive%20Algorithms.php#.WIBURaeGPcs](http://www.allsyllabus.com/aj/note/Computer_Science/Analysis_and_Design_of_Algorithms/Unit2/Mathematical%20analysis%20(Time%20Efficiency)%20of%20recursive%20Algorithms.php#.WIBURaeGPcs)
4. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.37.213&rep=rep1&type=pdf>

ADVANCE DATA STRUCTURE

CHAPTER

3

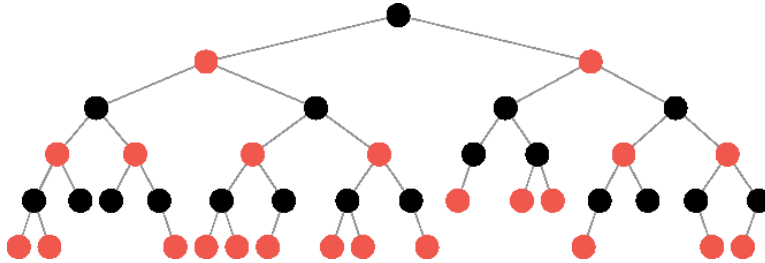
LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Define the basic concepts of data structure and algorithms
2. Explain arrays and linked lists
3. Discuss about binary search tree
4. Describe the stack and queue

INTRODUCTION

Data Structures are the programmatic way of storing data so that data can be used efficiently. Almost every enterprise application uses various types of data structures in one or the other way. This chapter will give you a great understanding on Data Structures needed to understand the complexity of enterprise level applications and need of algorithms, and data structures.



Data Structure is a way of collecting and organizing data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and storage. For example, we have data player's name "Smith" and age 26. Here "Smith" is of String data type and 26 is of integer data type. In simple language, Data Structures are structures programmed to store ordered data, so that various operations can be performed on it easily.

This chapter is meant as a basic introduction into elementary algorithmic principles and data structures used in computer science. In the latter field, the focus is on processing information in a systematic and often automatized way. One goal in the design of solution methods (algorithms) is about making efficient use of hardware resources such as computing time and memory. It is true that hardware development is very fast; the processors' speed increases rapidly.

3.1 BASIC CONCEPTS OF DATA STRUCTURE AND ALGORITHMS

Before introducing data structures we should understand that computers do store, retrieve, and process a large amount of data. If the data is stored in well-organized way on storage media and in computer's memory then it can be accessed quickly for processing that further reduces the latency and the user is provided fast response.



Data structure introduction refers to a scheme for organizing data, or in other words a data structure is an arrangement of data in computer's memory in such a way that it could make the data quickly available to the processor for required calculations. A data structure should be seen as a logical concept that must address two fundamental concerns. First, how the data will be stored, and second, what operations will be performed on it? As data structure is a scheme for data organization so the functional definition of a data structure should be independent of its implementation. The functional definition of a data structure is known as ADT (Abstract Data Type) which is independent of **implementation**. The implementation part is left on developers who decide which technology better suits to their project needs.

A stack ADT is a structure which supports operations such as push and pop. A stack can be implemented in a number of ways, for example using an array or using a linked list.



Along with data structures introduction, in real life, problem solving is done with help of data structures and algorithms. An algorithm is a step by step process to solve a problem. In programming, algorithms are implemented in form of methods or functions or routines. To get a problem solved we not only want algorithm but also an efficient algorithm. One criteria of efficiency is time taken by the algorithm, another could be the memory it takes at run time.

Sometimes, we can have more than one algorithm for the same problem to process a data structure, and we have to choose the best one among available algorithms. This is done by algorithm analysis. The best algorithm is the one which has a fine balance between time taken and memory consumption. But, as we know the best exists rarely, and we generally give more priority to the time taken by the algorithm rather than the memory it consumes. Also, as memory is getting cheaper and computers have more memory today than previous time, therefore, run time analysis becomes more significant than memory.



Implementation is the realization of an application, or execution of a plan, idea, model, design, specification, standard, algorithm, or policy.

3.1.1 Basic Concepts

This chapter explains the basic terms related to data structure.

Data Definition

Data Definition defines a particular data with the following characteristics.

- Atomic – Definition should define a single concept.
- Traceable – Definition should be able to be mapped to some data element.
- Accurate – Definition should be unambiguous.
- Clear and Concise – Definition should be understandable.

Data Object

Data Object represents an object having a data.

Data Type

Data type is a way to classify various types of data such as integer, string, etc. which determines the values that can be used with the corresponding type of data, the type of operations that can be performed on the corresponding type of data. There are two data types –

- Built-in Data Type
- Derived Data Type

Built-in Data Type

Those data types for which a language has built-in support are known as Built-in Data types. For example, most of the languages provide the following built-in data types.

- Integers
- Boolean (true, false)
- Floating (Decimal numbers)
- Character and Strings

Derived Data Type

Those data types which are implementation independent as they can be implemented in one or the other way are known as derived data types. These data types are normally built by the combination of primary or built-in data types and associated operations on them. For example –



- List
- Array
- Stack
- Queue

Basic Operations

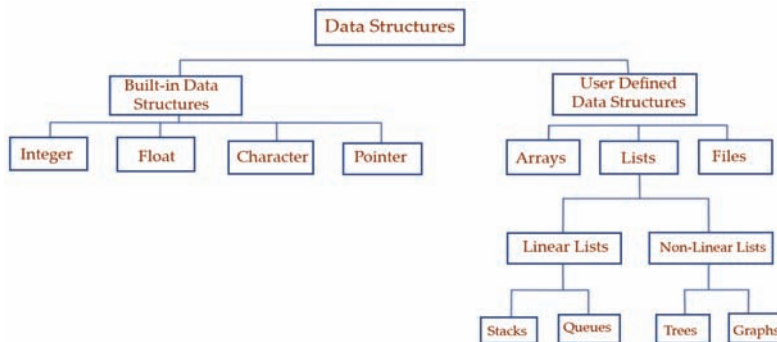
The data in the data structures are processed by certain operations. The particular data structure chosen largely depends on the frequency of the operation that needs to be performed on the data structure.

- Traversing
- Searching
- Insertion
- Deletion
- Sorting
- Merging

3.1.2 Classification of Data Structures

Data structures can be broadly classified in two categories - linear structures and hierarchical structures. Arrays, linked lists, stacks, and queues are linear structures, while trees, graphs, heaps etc. are hierarchical structures.

Every data structure has its own strengths, and weaknesses. Also, every data structure specially suits to specific problem types depending upon the operations performed and the data organization. For example, an array is suitable for read operations. Following is a quick introduction to important data structures.



Arrays

Arrays are statically implemented data structures by some programming languages like C and C++; hence the size of this data structure must be known at compile time and cannot be altered at run time. But modern programming languages, for example, Java implements arrays as objects and give the programmer a way to alter the size of them at run time. Arrays are the most common data structure used to store data.

Arrays are unarguably easier data structures to use and access. But inserting an item to an array and deleting it from the array are situation dependent. If you want to insert an item at a particular position which is already occupied by some element then you have to shift all items one position right from the position new element has to be inserted then insert the new item. The time taken by insert operation is depend on how big the array is, and at which position the item is being inserted. The same is true about deletion of an item.

If the array is unsorted then search operation is also proved costly and takes $O(N)$ time in worst case, where N is size of the array. But if the array is sorted then search performance is improved magically and takes $O(\log N)$ time in worst case.



Binary number is a number expressed in the binary numeral system or base-2 numeral system which represents numeric values using two different symbols: typically 0 and 1.

Linked List

Linked list data structure provides better memory management than arrays. Because linked list is allocated memory at run time, so, there is no waste of memory. Performance wise linked list is slower than array because there is no direct access to linked list elements.

Linked list is proved to be a useful data structure when the number of elements to be stored is not known ahead of time. There are many flavors of linked list you will see: linear, circular, doubly, and doubly circular.

Stack

Stack is a last-in-first-out strategy data structure; this means that the element stored in last will be removed first. Stack has specific but very useful applications; some of them are as follows:

- Solving Recursion - recursive calls are placed onto a stack, and removed from there once they are processed.
- Evaluating post-fix expressions
- Solving Towers of Hanoi
- Backtracking
- Depth-first search
- Converting a decimal number into a **binary number**

DID YOU KNOW?

In the early 60's G.M. Adelson-Velsky and E.M. Landis invented the first self-balancing binary search tree data structure, calling it AVL Tree.

Queue

Queue is a first-in-first-out data structure. The element that is added to the queue data structure first, will be removed from the queue first. Dequeue, priority queue, and circular queue are the variants of queue data structure. Queue has the following application uses:

- Access to shared resources (e.g., printer)
- Multiprogramming
- Message queue

Trees

Tree is a hierarchical data structure. The very top element of a tree is called the root of the tree. Except the root element every element in a tree has a parent element, and zero or more children elements. All elements in the left sub-tree come before the root in sorting order, and all those in the right sub-tree come after the root.

Tree is the most useful data structure when you have hierarchical information to store. For example, directory structure of a file system; there are many variants of tree you will come across. Some of them are Red-black tree, threaded binary tree, AVL tree, etc.

Heap

Heap is a binary tree that stores a collection of keys by satisfying heap property. Max heap and min heap are two flavors of heap data structure. The heap property for max heap is: each node should be greater than or equal to each

**REMEMBER**

Both graphs and digraphs can be represented using dynamic links or using arrays. As always, the representation should facilitate the intended operations and make them convenient to implement.

of its children. While, for min heap it is: each node should be smaller than or equal to each of its children. Heap data structure is usually used to implement priority queues.

Dictionary

Dictionary is a data structure that maintains a set of items indexed on basis of keys. Dictionary stores data in form of key-element pairs.

Hash Table

Hash Table is again a data structure that stores data in form of key-element pairs. A key is a non-null value which is mapped to an element. And, the element is accessed on the basis of the key associated with it. Hash table is a useful data structure for implementing dictionary.

Graph

Graph is a networked data structure that connects a collection of nodes called vertices, by connections, called edges. An edge can be seen as a path or communication link between two nodes. These edges can be either directed or undirected. If a path is directed then you can move in one direction only, while in an undirected path the movement is possible in both directions.

3.2 ARRAYS

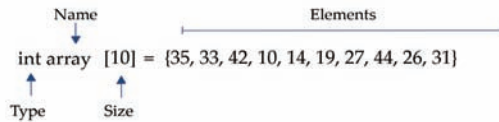
Array is a container which can hold a fix number of items and these items should be of the same type. Most of the data structures make use of arrays to implement their algorithms. Following are the important terms to understand the concept of Array.

- **Element** – Each item stored in an array is called an element
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element

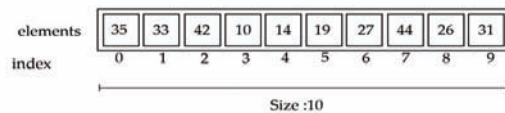


3.2.1 Array Representation

Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



As per the above illustration, following are the important points to be considered.

- Index starts with 0.
- Array length is 8 which means it can store 8 elements.
- Each element can be accessed via its index. For example, we can fetch an element at index 6 as 9.



REMEMBER

In 'C', variables must be declared before their use and must be declared with a relevant data type, which specifies the range of values that may be stored in the variable of that data type.

3.2.2 Basic Operations

Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.

In C, when an array is initialized with size, then it assigns defaults values to its elements in following order.

Data Type	Default Value
bool	false
char	0
int	0
float	0.0
double	0.0f
void	
wchar_t	0

Insertion Operation

Insert operation is to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array.

Here, we see a practical implementation of insertion operation, where we add data at the end of the array –

Algorithm

Let Array be a linear unordered array of MAX elements.

Example

Let **LA** be a Linear Array (unordered) with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm where ITEM is inserted into the K^{th} position of LA –

1. Start
2. Set $J = N$
3. Set $N = N + 1$
4. Repeat steps 5 and 6 while $J \geq K$
5. Set $LA[J+1] = LA[J]$
6. Set $J = J - 1$
7. Set $LA[K] = \text{ITEM}$
8. Stop

Example

Following is the implementation of the above algorithm –

```
#include <stdio.h>
```



```

main() {
    int LA[] = {1,3,5,7,8};
    int item = 10, k = 3, n = 5;
    int i = 0, j = n;
    printf("The original array elements are :\n");
    for(i = 0; i<n; i++) {
        printf("LA[%d] = %d \n", i, LA[i]);
    }
    n = n + 1;
    while( j >= k) {
        LA[j+1] = LA[j];
        j = j - 1;
    }
    LA[k] = item;
    printf("The array elements after insertion :\n");
    for(i = 0; i<n; i++) {
        printf("LA[%d] = %d \n", i, LA[i]);
    }
}

```

When we compile and execute the above program, it produces the following result –

Output

The original array elements are:

LA[0] = 1

LA[1] = 3

LA[2] = 5

$LA[3] = 7$

$LA[4] = 8$

The array elements after insertion:

$LA[0] = 1$

$LA[1] = 3$

$LA[2] = 5$

$LA[3] = 10$

$LA[4] = 7$

$LA[5] = 8$

Array Insertions

In the above section, we have learnt how the insertion operation works. It is not always necessary that an element is inserted at the end of an array. Following can be a situation with array insertion –

- Insertion at the beginning of an array
- Insertion at the given index of an array
- Insertion after the given index of an array
- Insertion before the given index of an array

Insertion at the Beginning of an Array

When the insertion happens at the beginning, it causes all the existing data items to shift one step downward. Here, we design and implement an **algorithm** to insert an element at the beginning of an array.

Algorithm

We assume A is an array with N elements. The maximum numbers of elements it can store is defined by MAX. We shall first check if an array has any empty space to store any element and then we proceed with the insertion process.

```

begin

IF N = MAX, return
ELSE
    N = N + 1

    For All Elements in A
        Move to next adjacent location

    A[FIRST] = New_Element

end

```

Implementation in C

```

#include <stdio.h>
#define MAX 5

void main() {
    int array[MAX] = {2, 3, 4, 5};
    int N = 4;      // number of elements in array
    int i = 0;      // loop variable
    int value = 1;  // new data element to be stored in array

    // print array before insertion
    printf("Printing array before insertion -\n");

    for(i = 0; i < N; i++) {
        printf("array[%d] = %d \n", i, array[i]);
    }

    // now shift rest of the elements downwards
    for(i = N; i >= 0; i--) {
        array[i+1] = array[i];
    }

    // add new element at first position
    array[0] = value;

    // increase N to reflect number of elements
    N++;
}

```



Algorithm is a procedure or formula for solving a problem, based on conducting a sequence of specified actions.

```
// print to confirm
printf("Printing array after insertion -\n");

for(i = 0; i < N; i++) {
    printf("array[%d] = %d\n", i, array[i]);
}
}
```

This program should yield the following output –

Printing array before insertion –

array[0] = 2

array[1] = 3

array[2] = 4

array[3] = 5

Printing array after insertion –

array[0] = 0

array[1] = 2

array[2] = 3

array[3] = 4

array[4] = 5

Insertion at the Given Index of an Array

In this scenario, we are given the exact location (index) of an array where a new data element (value) needs to be inserted. First we shall check if the array is full, if it is not, then we shall move all data elements from that location one step downward. This will make room for a new data element.

Algorithm

We assume A is an array with N elements. The maximum numbers of elements it can store is defined by MAX.

```

begin

IF N = MAX, return
ELSE
    N = N + 1

    SEEK Location index

    For All Elements from A[index] to A[N]
        Move to next adjacent location

    A[index] = New_Element

end

```

Implementation in C

```

#include <stdio.h>
#define MAX 5

void main() {
    int array[MAX] = {1, 2, 4, 5};

    int N = 4;        // number of elements in array
    int i = 0;        // loop variable
    int index = 2;    // index location to insert new value
    int value = 3;    // new data element to be inserted

    // print array before insertion
    printf("Printing array before insertion -\n");
    for(i = 0; i < N; i++) {
        printf("array[%d] = %d \n", i, array[i]);
    }

    // now shift rest of the elements downwards
    for(i = N; i >= index; i--) {
        array[i+1] = array[i];
    }

    // add new element at first position
    array[index] = value;

    // increase N to reflect number of elements
    N++;
}

```

```
// print to confirm
printf("Printing array after insertion -\n");

for(i = 0; i < N; i++) {
    printf("array[%d] = %d\n", i, array[i]);
}
}
```

If we compile and run the above program, it will produce the following result –

Printing array before insertion –

array[0] = 1

array[1] = 2

array[2] = 4

array[3] = 5

Printing array after insertion –

array[0] = 1

array[1] = 2

array[2] = 3

array[3] = 4

array[4] = 5

Insertion after the Given Index of an Array

In this scenario we are given a location (index) of an array after which a new data element (value) has to be inserted. Only the seek process varies, the rest of the activities are the same as in the above example.

Algorithm

We assume A is an array with N elements. The maximum numbers of elements it can store is defined by MAX.


```

begin

IF N = MAX, return
ELSE
    N = N + 1

    SEEK Location index

    For All Elements from A[index + 1] to A[N]
        Move to next adjacent location

    A[index + 1] = New_Element

end

```

Implementation in C

```

#include <stdio.h>
#define MAX 5

void main() {
    int array[MAX] = {1, 2, 4, 5};

    int N = 4;        // number of elements in array
    int i = 0;        // loop variable
    int index = 1;    // index location after which value will be inserted
    int value = 3;    // new data element to be inserted

    // print array before insertion
    printf("Printing array before insertion -\n");
}

```

```
for(i = 0; i < N; i++) {
    printf("array[%d] = %d \n", i, array[i]);
}

// now shift rest of the elements downwards
for(i = N; i >= index + 1; i--) {
    array[i + 1] = array[i];
}

// add new element at first position
array[index + 1] = value;

// increase N to reflect number of elements
N++;

// print to confirm
printf("Printing array after insertion -\n");

for(i = 0; i < N; i++) {
    printf("array[%d] = %d\n", i, array[i]);
}
}
```

If we compile and run the above program, it will produce the following result –

```
Printing array before insertion -
array[0] = 1
array[1] = 2
array[2] = 4
array[3] = 5
Printing array after insertion -
array[0] = 1
array[1] = 2
array[2] = 3
array[3] = 4
array[4] = 5
```



Insertion before the Given Index of an Array

In this scenario we are given a location (index) of an array before which a new data element (value) has to be inserted. This time we seek till index-1, i.e., one location ahead of the given index.

Algorithm

We assume A is an array with N elements. The maximum numbers of elements it can store is defined by MAX.

begin

IF N = MAX, return

ELSE

 N = N + 1

 SEEK Location index

 For All Elements from A[index - 1] to A[N]

 Move to next adjacent location

 A[index - 1] = New_Element

end

Implementation in C

```
#include <stdio.h>
#define MAX 5
```

```
void main() {
    int array[MAX] = {1, 2, 4, 5};

    int N = 4;          // number of elements in array
    int i = 0;          // loop variable
    int index = 3;       // index location before which value will be inserted
    int value = 3;       // new data element to be inserted

    // print array before insertion
    printf("Printing array before insertion -\n");
    for(i = 0; i < N; i++) {
        printf("array[%d] = %d \n", i, array[i]);
    }

    // now shift rest of the elements downwards
    for(i = N; i >= index + 1; i--) {
        array[i + 1] = array[i];
    }

    // add new element at first position
    array[index + 1] = value;

    // increase N to reflect number of elements
    N++;

    // print to confirm
    printf("Printing array after insertion -\n");

    for(i = 0; i < N; i++) {
        printf("array[%d] = %d\n", i, array[i]);
    }
}
```

If we compile and run the above program, it will produce the following result –
Printing array before insertion –

```
array[0] = 1
array[1] = 2
```



```
array[2] = 4
array[3] = 5
```

Printing array after insertion -

```
array[0] = 1
array[1] = 2
array[2] = 4
array[3] = 5
array[4] = 3
```

Deletion Operation

Deletion refers to removing an existing element from the array and re-organizing all elements of an array.

Algorithm

Consider **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm to delete an element available at the K^{th} position of LA.

1. Start
2. Set $J = K$
3. Repeat steps 4 and 5 while $J < N$
4. Set $LA[J-1] = LA[J]$
5. Set $J = J+1$
6. Set $N = N-1$
7. Stop

Example

Following is the implementation of the above algorithm –

```
#include <stdio.h>

main() {
    int LA[] = {1,3,5,7,8};
    int k = 3, n = 5;
```

```
int i, j;
printf("The original array elements are :\n");
for(i = 0; i<n; i++) {
    printf("LA[%d] = %d \n", i, LA[i]);
}
j = k;
while( j < n) {
    LA[j-1] = LA[j];
    j = j + 1;
}
n = n -1;
printf("The array elements after deletion :\n");
for(i = 0; i<n; i++) {
    printf("LA[%d] = %d \n", i, LA[i]);
}
}
```

When we compile and execute the above program, it produces the following result –

Output

The original array elements are:

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

The array elements after deletion:

LA[0] = 1

LA[1] = 3

LA[2] = 7

LA[3] = 8

Search Operation

You can perform a search for an array element based on its value or its index.

Algorithm

Consider **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm to find an element with a value of **ITEM** using sequential search.

1. Start
2. Set $J = 0$
3. Repeat steps 4 and 5 while $J < N$
4. IF LA[J] is equal ITEM THEN GOTO STEP 6
5. Set $J = J + 1$
6. PRINT J, ITEM
7. Stop

Example

Following is the implementation of the above algorithm –

```
#include <stdio.h>

main() {
    int LA[] = {1,3,5,7,8};
    int item = 5, n = 5;
    int i = 0, j = 0;
```

```
printf("The original array elements are :\n");
for(i = 0; i<n; i++) {
    printf("LA[%d] = %d \n", i, LA[i]);
}
while( j < n){
    if( LA[j] == item ) {
        break;
    }
    j = j + 1;
}
printf("Found element %d at position %d\n", item, j+1);
}
```

When we compile and execute the above program, it produces the following result –

Output

The original array elements are:

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

Found element 5 at position 3

Update Operation

Update operation refers to updating an existing element from the array at a given index.

Algorithm

Consider **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm to update an element available at the K^{th} position of LA.

1. Start
2. Set $LA[K-1] = \text{ITEM}$
3. Stop

Example

Following is the implementation of the above algorithm –

```
#include <stdio.h>

main() {
    int LA[] = {1,3,5,7,8};
    int k = 3, n = 5, item = 10;
    int i, j;
    printf("The original array elements are :\n");
    for(i = 0; i<n; i++) {
        printf("LA[%d] = %d \n", i, LA[i]);
    }
    LA[k-1] = item;
    printf("The array elements after updation :\n");
    for(i = 0; i<n; i++) {
        printf("LA[%d] = %d \n", i, LA[i]);
    }
}
```

When we compile and execute the above program, it produces the following result –

Output

The original array elements are:

$LA[0] = 1$

$LA[1] = 3$

$LA[2] = 5$

$LA[3] = 7$

$LA[4] = 8$

The array elements after updation:

$LA[0] = 1$

$LA[1] = 3$

$LA[2] = 10$

$LA[3] = 7$

$LA[4] = 8$

3.3 LINKED LISTS

Linked lists can be thought of from a high level perspective as being a series of nodes. Each node has at least a single pointer to the next node, and in the last node's case a null pointer representing that there are no more nodes in the linked list.

In DSA our implementations of linked lists always maintain head and tail pointers so that insertion at either the head or tail of the list is a constant time operation. Random insertion is excluded from this and will be a linear operation. As such, linked lists in DSA have the following characteristics:

1. Insertion is $O(1)$
2. Deletion is $O(n)$
3. Searching is $O(n)$

Out of the three operations the one that stands out is that of insertion. In DSA we chose to always maintain pointers (or more aptly references) to the node(s) at the head and tail of the linked list and so performing a traditional insertion to either the front or back of the linked list is an $O(1)$ operation. An exception to this rule is performing an insertion before a node that is neither the head nor tail in a singly linked list. When the node we are inserting before is somewhere in the middle of the linked list (known as random insertion) the complexity is $O(n)$. In order to add before the designated



node we need to traverse the linked list to find that node's current predecessor. This traversal yields an $O(n)$ run time.

This data structure is trivial, but linked lists have a few key points which at times make them very attractive:

- the list is dynamically resized, thus it incurs no copy penalty like an array or vector would eventually incur; and
- Insertion is $O(1)$

3.3.1 Singly Linked List

Singly linked lists are one of the most primitive data structures. Each node that makes up a singly linked list consists of a value, and a reference to the next node (if any) in the list.

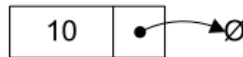


Figure 3.1: Singly linked list node.

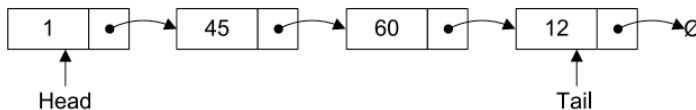


Figure 3.2: A singly linked list populated with integers.

Insertion

In general when people talk about insertion with respect to linked lists of any form they implicitly refer to the adding of a node to the tail of the list. When you use an API like that of DSA and you see a general purpose method that adds a node to the list, you can assume that you are adding the node to the tail of the list not the head.

Adding a node to a singly linked list has only two cases:

1. $\text{head} = \emptyset$ in which case the node we are adding is now both the head and tail of the list; or
2. we simply need to append our node onto the end of the list updating the tail reference appropriately

1. algorithm Add(value)
2. Pre: value is the value to add to the list
3. Post: value has been placed at the tail of the list
4. $n \leftarrow \text{node}(\text{value})$
5. if head = $\emptyset$
6. head $\leftarrow n$
7. tail $\leftarrow n$
8. else
9. tail.Next $\leftarrow n$
10. tail $\leftarrow n$
11. end if
12. end Add

As an example of the above algorithm consider adding the following sequence of integers to the list: 1, 45, 60, and 12, the resulting list is that of Figure 3.2.

Searching

Searching a linked list is straightforward: we simply traverse the list checking the value we are looking for with the value of each node in the linked list. The algorithm listed in this section is very similar to that used for traversal in below section.

```

algorithm Contains(head, value)
  Pre: head is the head node in the list
        value is the value to search for
  Post: the item is either in the linked list, true; otherwise false
  n  $\leftarrow$  head
  while n  $\neq \emptyset$  and n.Value  $\neq$  value
    n  $\leftarrow$  n.Next
  end while
  if n =  $\emptyset$ 
    return false
  end if
  return true
end Contains

```

Deletion

Deleting a node from a linked list is straightforward but there are a few cases we need to account for:



1. the list is empty; or
2. the node to remove is the only node in the linked list; or
3. we are removing the head node; or
4. we are removing the tail node; or
5. the node to remove is somewhere in between the head and tail; or
6. the item to remove does not exist in the linked list

The algorithm whose cases we have described will remove a node from anywhere within a list irrespective of whether the node is the head etc. If you know that items will only ever be removed from the head or tail of the list then you can create much more concise algorithms. In the case of always removing from the front of the linked list deletion becomes an $O(1)$ operation.

algorithm Remove(*head*, *value*)

Pre: *head* is the head node in the list

value is the value to remove from the list

Post: *value* is removed from the list, true; otherwise false

if *head* = $\emptyset$

 // case 1

return false

end if

n $\leftarrow$ *head*

if *n*.Value = *value*

if *head* = *tail*

 // case 2

head $\leftarrow$ $\emptyset$

tail $\leftarrow$ $\emptyset$

else

 // case 3

head $\leftarrow$ *head*.Next

end if

return true

end if

while *n*.Next $\neq \emptyset$ **and** *n*.Next.Value $\neq$ *value*

n $\leftarrow$ *n*.Next

end while

if *n*.Next $\neq \emptyset$

if *n*.Next = *tail*

 // case 4

tail $\leftarrow$ *n*

end if

 // this is only case 5 if the conditional on line 25 was *false*

n.Next $\leftarrow$ *n*.Next.Next

return true

end if

```
// case 6
return false
end Remove
```

Traversing the list

Traversing a singly linked list is the same as that of traversing a doubly linked list. You start at the head of the list and continue until you come across a node that is $\emptyset$. The two cases are as follows:

1. node = $\emptyset$, we have exhausted all nodes in the linked list; or
2. we must update the node reference to be node.Next

The algorithm described is a very simple one that makes use of a simple while loop to check the first case.

```
algorithm Traverse(head)
  Pre: head is the head node in the list
  Post: the items in the list have been traversed
  n ← head
  while n ≠ 0
    yield n.Value
    n ← n.Next
  end while
end Traverse
```

Traversing the list in reverse order

Traversing a singly linked list in a forward manner (i.e. left to right) is simple as demonstrated in above section. However, what if we wanted to traverse the nodes in the linked list in reverse order for some reason? The algorithm to perform such a traversal is very simple, and just like demonstrated in above we will need to acquire a reference to the predecessor of a node, even though the fundamental characteristics of the nodes that make up a singly linked list make this an expensive operation. For each node, finding its predecessor is an $O(n)$ operation, so over the course of traversing the whole list backwards the cost becomes $O(n^2)$.

Figure 3.3 depicts the following algorithm being applied to a linked list with the integers 5, 10, 1, and 40.



algorithm ReverseTraversal(*head*, *tail*)

Pre: *head* and *tail* belong to the same list

Post: the items in the list have been traversed in reverse order

if *tail* $\neq \emptyset$

curr $\leftarrow$ *tail*

while *curr* $\neq$ *head*

prev $\leftarrow$ *head*

while *prev*.Next $\neq$ *curr*

prev $\leftarrow$ *prev*.Next

end while

yield *curr*.Value

curr $\leftarrow$ *prev*

end while

yield *curr*.Value

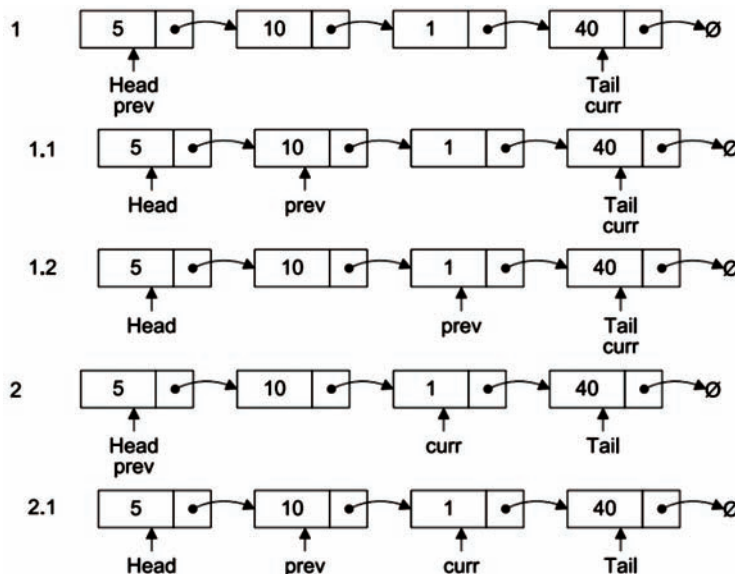
end if

end ReverseTraversal

This algorithm is only of real interest when we are using singly linked lists, as you will soon see that doubly linked lists make reverse list traversal simple and efficient, as shown in below.

3.3.2 Doubly Linked List

Doubly linked lists are very similar to singly linked lists. The only difference is that each node has a reference to both the next and previous nodes in the list.



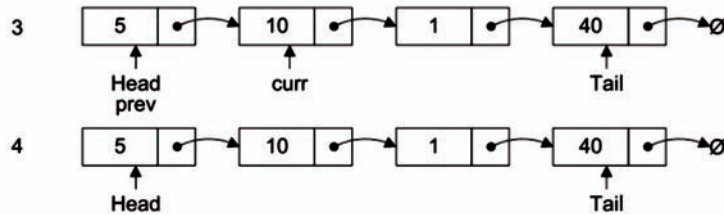


Figure 3.3: Reverse traversal of a singly linked list.



Figure 3.4: Doubly linked list node.

The following algorithms for the doubly linked list are exactly the same as those listed previously for the singly linked list:

- Searching
- Traversal

Insertion

The only major difference between the before algorithm is that we need to remember to bind the previous pointer of n to the previous tail node if n was not the first node to be inserted into the list.

```

algorithm Add(value)
  Pre: value is the value to add to the list
  Post: value has been placed at the tail of the list
   $n \leftarrow \text{node}(\textit{value})$ 
  if  $\textit{head} = \emptyset$ 
     $\textit{head} \leftarrow n$ 
     $\textit{tail} \leftarrow n$ 
  else
     $n.\textit{Previous} \leftarrow \textit{tail}$ 
     $\textit{tail}.\textit{Next} \leftarrow n$ 
     $\textit{tail} \leftarrow n$ 
  end if
end Add
  
```

Figure 3.5 shows the doubly linked list after adding the sequence of integers.

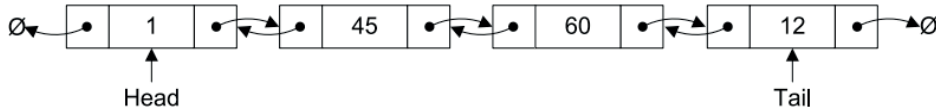


Figure 3.5: Doubly linked list populated with integers.

Deletion

As you may have guessed the cases that we use for deletion in a doubly linked list are exactly the same as those defined in above. Like insertion we have the added task of binding an additional reference to the correct value.

algorithm Remove(*head*, *value*)

Pre: *head* is the head node in the list

value is the value to remove from the list

Post: *value* is removed from the list, true; otherwise false

if *head* = ∅

return false

end if

if *value* = *head*.Value

if *head* = *tail*

if *head* = *tail*

head ← ∅

tail ← ∅

else

head ← *head*.Next

head.Previous ← ∅

end if

return true

end if

n ← *head*.Next

while *n* ≠ ∅ **and** *value* ≠ *n*.Value

n ← *n*.Next

end while

if *n* = *tail*

tail ← *tail*.Previous

tail.Next ← ∅

return true

else if *n* ≠ ∅

n.Previous.Next ← *n*.Next

n.Next.Previous ← *n*.Previous

return true

```

end if
return false
end Remove

```



KEYWORD

Pointer is a programming language object, whose value refers to another value stored elsewhere in the computer memory using its memory address.

Reverse Traversal

Singly linked lists have a forward only design, which is why the reverse traversal algorithm defined in above section required some creative invention. Doubly linked lists make reverse traversal as simple as forward traversal except that we start at the tail node and update the **pointer** in the opposite direction. Figure 3.6 shows the reverse traversal algorithm in action.

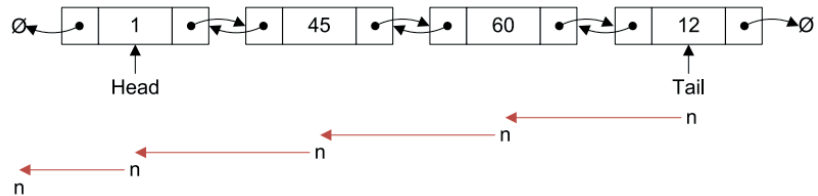


Figure 3.6: Doubly linked list reverse traversal.

algorithm ReverseTraversal(*tail*)

Pre: *tail* is the tail node of the list to traverse

Post: the list has been traversed in reverse order

$n \leftarrow tail$

while $n \neq \emptyset$

yield $n.Value$

$n \leftarrow n.Previous$

end while

end ReverseTraversal

3.4 BINARY SEARCH TREE

Binary search trees (BSTs) are very simple to understand. We start with a root node with value x , where the left sub tree of x contains nodes with values $< x$ and the right sub tree contains nodes whose values are $\geq x$. Each node follows the same rules with respect to nodes in their left and right sub trees. BSTs are of interest because they have operations which are favorably fast: insertion, look up, and deletion can all be



done in $O(\log n)$ time. It is important to note that the $O(\log n)$ times for these operations can only be attained if the BST is reasonably balanced; for a tree data structure with self-balancing properties.

In the following examples you can assume, unless used as a parameter alias that *root* is a reference to the root node of the tree.

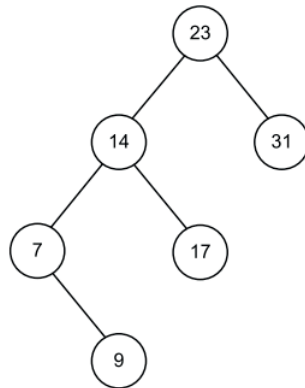


Figure 3.7: Simple unbalanced binary search tree.



REMEMBER

If a given key is less than the key at the root of a BST, then the floor of key must be in the left sub tree.

3.4.1 Insertion

As mentioned above insertion is an $O(\log n)$ operation provided that the tree is moderately balanced.

```

algorithm Insert(value)
  Pre: value has passed custom type checks for type T
  Post: value has been placed in the correct location in the tree
  if root =  $\emptyset$ 
    root  $\leftarrow$  node(value)
  else
    InsertNode(root, value)
  end if
end Insert
  
```

and

```

algorithm InsertNode(current, value)
  Pre: current is the node to start from
  Post: value has been placed in the correct location in the tree
  if value < current.Value
    if current.Left =  $\emptyset$ 
      current.Left  $\leftarrow$  node(value)
    else
      InsertNode(current.Left, value)
    end if
  else
    if current.Right =  $\emptyset$ 
      current.Right  $\leftarrow$  node(value)
    else
      InsertNode(current.Right, value)
    end if
  end if
end InsertNode

```

The insertion algorithm is split for a good reason. The first algorithm (non-recursive) checks a very core base case - whether or not the tree is empty. If the tree is empty then we simply create our root node and finish. In all other cases we invoke the recursive InsertNode algorithm which simply guides us to the first appropriate place in the tree to put value. Note that at each stage we perform a binary chop: we either choose to recurse into the left sub tree or the right by comparing the new value with that of the current node. For any totally ordered type, no value can simultaneously satisfy the conditions to place it in both sub trees.

3.4.2 Searching

Searching a BST is even simpler than insertion. The pseudo code is self-explanatory but we will look briefly at the premise of the algorithm nonetheless.

We have talked above about insertion, we go either left or right with the right sub tree containing values that are $\geq x$ where x is the value of the node we are inserting. When searching the rules are made a little more atomic and at any one time we have four cases to consider:

1. the root = $\emptyset$ in which case value is not in the BST; or
2. root.Value = value in which case value is in the BST; or
3. value < root.Value, we must inspect the left sub tree of root for value; or
4. value > root.Value, we must inspect the right sub tree of root for value

algorithm Contains(*root*, *value*)

Pre: *root* is the root node of the tree, *value* is what we would like to locate

Post: *value* is either located or not

if *root* = $\emptyset$

 return false

end if

if *root*.Value = *value*

 return true

else if *value* < *root*.Value

 return Contains(*root*.Left, *value*)

else

 return Contains(*root*.Right, *value*)

end if

end Contains

3.4.3 Deletion

Removing a node from a BST is fairly straightforward, with four cases to consider:

1. the value to remove is a leaf node; or
2. the value to remove has a right sub tree, but no left sub tree; or
3. the value to remove has a left sub tree, but no right sub tree; or
4. the value to remove has both a left and right sub tree in which case we promote the largest value in the left sub tree

There is also an implicit fifth case whereby the node to be removed is the only node in the tree. This case is already covered by the first, but should be noted as a possibility nonetheless.

Of course in a BST a value may occur more than once. In such a case the first occurrence of that value in the BST will be removed.

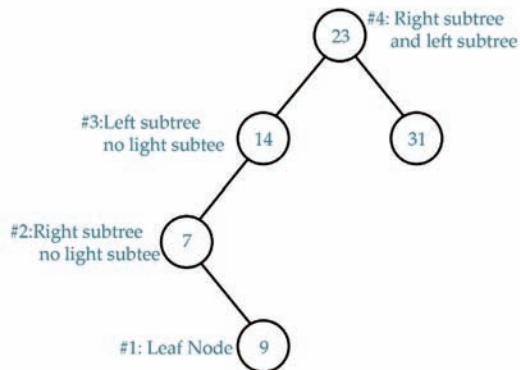


Figure 3.8: binary search tree deletion cases.

The Remove algorithm given below relies on two further helper algorithms named FindParent, and FindNode which are described in below respectively.

algorithm Remove(*value*)

Pre: *value* is the value of the node to remove, *root* is the root node of the BST

Count is the number of items in the BST

Post: node with *value* is removed if found in which case yields true, otherwise false

nodeToRemove $\leftarrow$ FindNode(*value*)

if *nodeToRemove* = $\emptyset$

return false // value not in BST

end if

parent $\leftarrow$ FindParent(*value*)

if Count = 1

root $\leftarrow \emptyset$ // we are removing the only node in the BST

else if *nodeToRemove*.Left = $\emptyset$ **and** *nodeToRemove*.Right = null

 // case #1

if *nodeToRemove*.Value < *parent*.Value

parent.Left $\leftarrow \emptyset$

else

parent.Right $\leftarrow \emptyset$

end if

else if *nodeToRemove*.Left = $\emptyset$ **and** *nodeToRemove*.Right $\neq \emptyset$

 // case # 2

if *nodeToRemove*.Value < *parent*.Value

parent.Left $\leftarrow$ *nodeToRemove*.Right

else

parent.Right $\leftarrow$ *nodeToRemove*.Right

end if

else if *nodeToRemove*.Left $\neq \emptyset$ **and** *nodeToRemove*.Right = $\emptyset$

 // case #3

if *nodeToRemove*.Value < *parent*.Value

parent.Left $\leftarrow$ *nodeToRemove*.Left

else

parent.Right $\leftarrow$ *nodeToRemove*.Left

end if

else

 // case #4

largestValue $\leftarrow$ *nodeToRemove*.Left

while *largestValue*.Right $\neq \emptyset$

 // find the largest value in the left subtree of *nodeToRemove*

largestValue $\leftarrow$ *largestValue*.Right


```

    end while
    // set the parents' Right pointer of largestValue to  $\emptyset$ 
    FindParent(largestValue.Value).Right  $\leftarrow \emptyset$ 
    nodeToRemove.Value  $\leftarrow$  largestValue.Value
end if
Count  $\leftarrow$  Count -1
return true
end Remove

```

3.4.4 Finding the parent of a given node

The purpose of this algorithm is simple - to return a reference (or pointer) to the parent node of the one with the given value. We have found that such an algorithm is very useful, especially when performing extensive tree transformations.

```

algorithm FindParent(value, root)
    Pre: value is the value of the node we want to find the parent of
           root is the root node of the BST and is  $\neq \emptyset$ 
    Post: a reference to the parent node of value if found; otherwise  $\emptyset$ 
    if value = root.Value
        return  $\emptyset$ 
    end if
    if value < root.Value
        if root.Left =  $\emptyset$ 
            return  $\emptyset$ 
        else if root.Left.Value = value
            return root
        else
            return FindParent(value, root.Left)
        end if
    else
        if root.Right =  $\emptyset$ 
            return  $\emptyset$ 
        else if root.Right.Value = value
            return root
        else
            return FindParent(value, root.Right)
        end if
    end if
end FindParent

```

A special case in the above algorithm is when the specified value does not exist in the BST, in which case we return $\emptyset$. Callers to this algorithm must take account of this possibility unless they are already certain that a node with the specified value exists.

3.4.5 Attaining a reference to a node

This algorithm is very similar to above algorithm, but instead of returning a reference to the parent of the node with the specified value, it returns a reference to the node itself. Again, $\emptyset$ is returned if the value is not found.

```

algorithm FindNode(root, value)
  Pre: value is the value of the node we want to find the parent of
        root is the root node of the BST
  Post: a reference to the node of value if found; otherwise  $\emptyset$ 
  if root =  $\emptyset$ 
    return  $\emptyset$ 
  end if
  if root.Value = value
    return root
  else if value < root.Value
    return FindNode(root.Left, value)
  else
    return FindNode(root.Right, value)
  end if
end FindNode
  
```

Astute readers will have noticed that the FindNode algorithm is exactly the same as the Contains algorithm with the modification that we are returning a reference to a node not true or false. Given FindNode, the easiest way of implementing Contains is to call FindNode and compare the return value with $\emptyset$.

3.4.6 Finding the smallest and largest values in the binary search tree

To find the smallest value in a BST you simply traverse the nodes in the left sub tree of the BST always going left upon each encounter with a node, terminating when you find a node with no left sub tree. The opposite is the case when finding the largest value in the BST. Both algorithms are incredibly simple, and are listed simply for completeness.

The base case in both FindMin, and FindMax algorithms is when the Left (FindMin), or Right (FindMax) node references are $\emptyset$ in which case we have reached the last node.


```

algorithm FindMin(root)
  Pre: root is the root node of the BST
         root  $\neq \emptyset$ 
  Post: the smallest value in the BST is located
  if root.Left =  $\emptyset$ 
    return root.Value
  end if
  FindMin(root.Left)
end FindMin

```

and

```

algorithm FindMax(root)
  Pre: root is the root node of the BST
         root  $\neq \emptyset$ 
  Post: the largest value in the BST is located
  if root.Right =  $\emptyset$ 
    return root.Value
  end if
  FindMax(root.Right)
end FindMax

```

3.4.7 Tree Traversals

There are various strategies which can be employed to traverse the items in a tree; the choice of strategy depends on which node visitation order you require. In this section we will touch on the traversals that DSA provides on all data structures that derive from Binary Search Tree.

Preorder

When using the preorder algorithm, you visit the root first, then traverse the left sub tree and finally traverse the right sub tree. An example of preorder traversal is shown in Figure 3.9.

```

algorithm Preorder(root)
  Pre: root is the root node of the BST
  Post: the nodes in the BST have been visited in preorder
  if root  $\neq \emptyset$ 
    yield root.Value
    Preorder(root.Left)
    Preorder(root.Right)
  end if
end Preorder

```

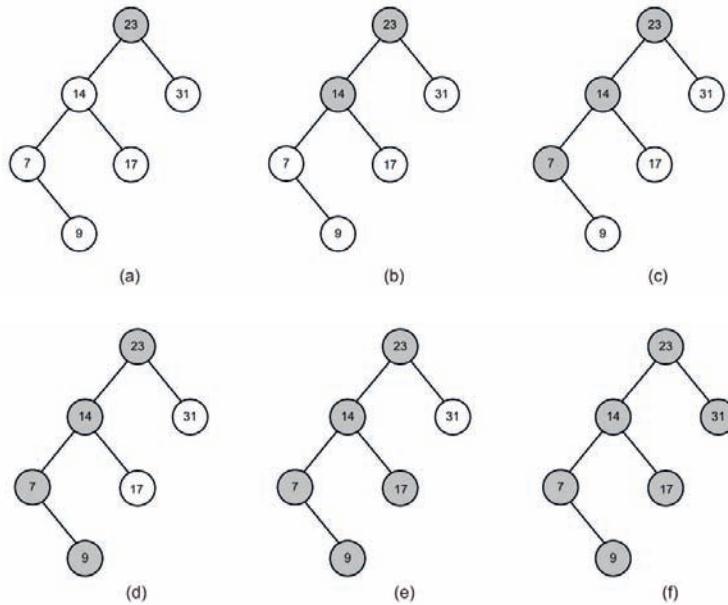


Figure 3.9: Preorder visit binary search tree example.

Postorder

This algorithm is very similar to that described in above algorithm, however the value of the node is yielded after traversing both sub trees. An example of postorder traversal is shown in Figure 3.10.

algorithm Postorder(*root*)

Pre: *root* is the root node of the BST

Post: the nodes in the BST have been visited in postorder

if *root* $\neq \emptyset$

 Postorder(*root*.Left)

 Postorder(*root*.Right)

yield *root*.Value

end if

end Postorder

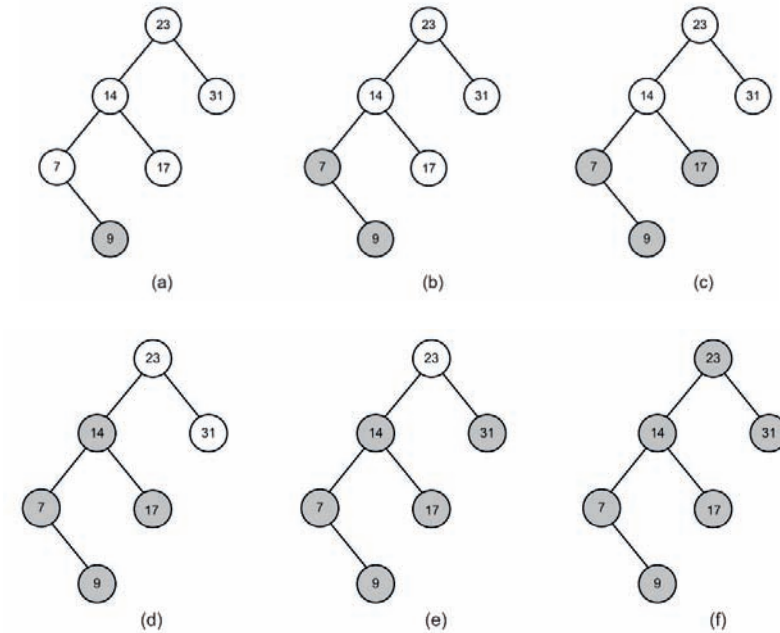
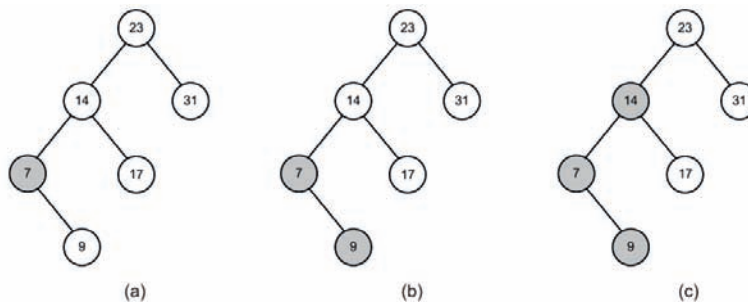


Figure 3.10: Postorder visit binary search tree example.

Inorder

Another variation of the algorithms defined in above algorithms is that of inorder traversal where the value of the current node is yielded in between traversing the left sub tree and the right sub tree. An example of inorder traversal is shown in Figure 3.11.



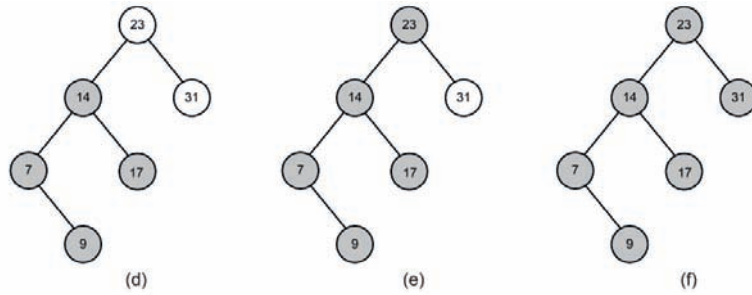


Figure 3.11: Inorder visit binary search tree example.

algorithm Inorder(*root*)

Pre: *root* is the root node of the BST

Post: the nodes in the BST have been visited in inorder

if *root* $\neq \emptyset$

 Inorder(*root*.Left)

yield *root*.Value

 Inorder(*root*.Right)

end if

end Inorder

One of the beauties of inorder traversal is that values are yielded in their comparison order. In other words, when traversing a populated BST with the inorder strategy, the yielded sequence would have property $x_i \leq x_{i+1}$.

Breadth First

Traversing a tree in breadth first order yields the values of all nodes of a particular depth in the tree before any deeper ones. In other words, given a depth d we would visit the values of all nodes at d in a left to right fashion, then we would proceed to $d + 1$ and so on until we have no more nodes to visit. An example of breadth first traversal is shown in Figure 3.12.

Traditionally breadth first traversal is implemented using a list (vector, resizable array, etc.) to store the values of the nodes visited in breadth first order and then a queue to store those nodes that have yet to be visited.

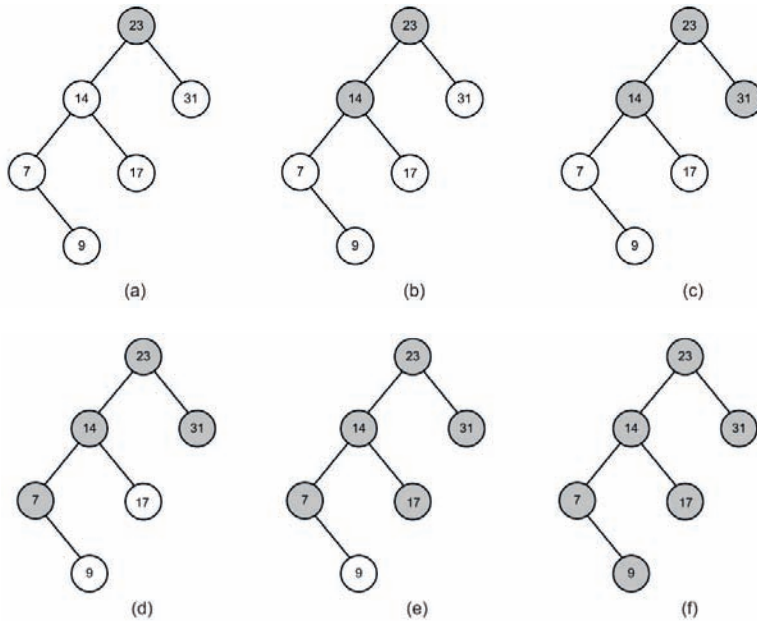


Figure 3.12: Breadth first visit binary search tree example.

algorithm BreadthFirst(*root*)

Pre: *root* is the root node of the BST

Post: the nodes in the BST have been visited in breadth first order

q $\leftarrow$ queue

while *root* $\neq \emptyset$

yield *root*.Value

if *root*.Left $\neq \emptyset$

q.Enqueue(*root*.Left)

end if

if *root*.Right $\neq \emptyset$

q.Enqueue(*root*.Right)

end if

if !*q*.IsEmpty()

root $\leftarrow$ *q*.Dequeue()

else

root $\leftarrow \emptyset$

end if

end while

end BreadthFirst

3.5 STACK AND QUEUE

In this section, we introduce two closely-related data types for manipulating arbitrarily large collections of objects: the stack and the queue. Stacks and queues are special

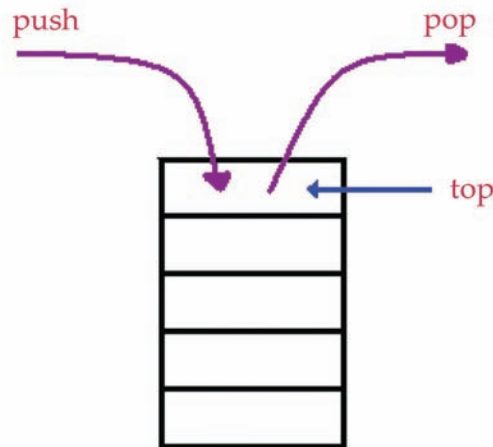
**REMEMBER**

A stack may be represented by an array, but the new item should be placed in the next available place in the array rather than at the end of the array.

cases of the idea of a collection. Each is characterized by four operations: create the collection, insert an item, remove an item, and test whether the collection is empty.

3.5.1 Stack

A stack is a container of objects that are inserted and removed according to the last-in first-out (LIFO) principle. In the pushdown stacks only two operations are allowed: push the item into the stack, and pop the item out of the stack. A stack is a limited access data structure - elements can be added and removed from the stack only at the top. Push adds an item to the top of the stack, pop removes the item from the top. A helpful analogy is to think of a stack of books; you can remove only the top book, also you can add a new book on the top.



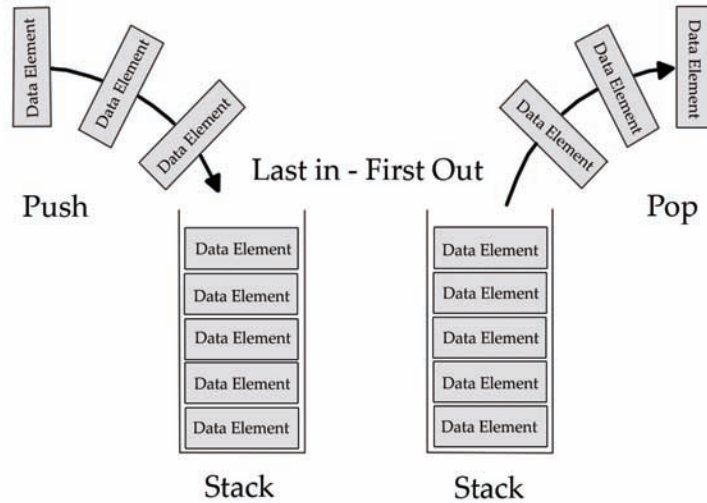
A stack is a recursive data structure. Here is a structural definition of a Stack:

- a stack is either empty or
- it consists of a top and the rest which is a stack;

Stack Representation

The following diagram depicts a stack and its operations –





A stack can be implemented by means of Array, Structure, Pointer, and Linked List. Stack can either be a fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack using arrays, which makes it a fixed size stack implementation.

Basic Operations

Stack operations may involve initializing the stack, using it and then de-initializing it. Apart from these basic stuffs, a stack is used for the following two primary operations –

- `push()` – Pushing (storing) an element on the stack
- `pop()` – Removing (accessing) an element from the stack

When data is pushed onto stack.

To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –

- `peek()` – get the top data element of the stack, without removing it
- `isFull()` – check if stack is full
- `isEmpty()` – check if stack is empty

At all times, we maintain a pointer to the last pushed data on the stack. As this pointer always represents the top of the stack, hence named `top`. The `top` pointer provides top value of the stack without actually removing it.

Applications

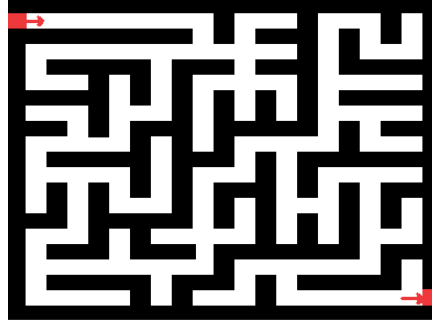
- The simplest application of a stack is to reverse a word. You push a given

word to stack - letter by letter - and then pop letters from the stack.

- Another application is an “undo” mechanism in text editors; this operation is accomplished by keeping all text changes in a stack.



Backtracking is a general algorithm for finding all solutions to some computational problems, notably constraint satisfaction problems that incrementally builds candidates to the solutions, and abandons each partial candidate *c* as soon as it determines that *c* cannot possibly be completed to a valid solution.



Backtracking.

This is a process when you need to access the most recent data element in a series of elements. Think of a labyrinth or maze - how do you find a way from an entrance to an exit?

Once you reach a dead end, you must backtrack. But backtrack to where? Therefore, at each choice point you store on a stack all possible choices. Then **backtracking** simply means popping a next choice from the stack.

Language processing:

- Space for parameters and local variables is created internally using a stack.
- Compiler's syntax check for matching braces is implemented by using stack.
- support for recursion

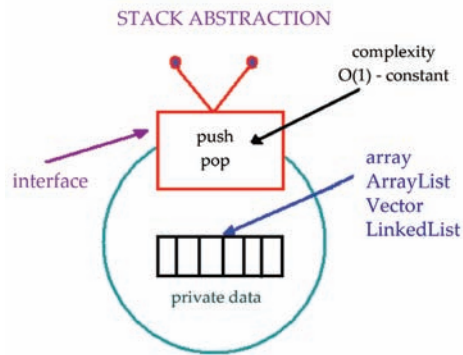
Implementation

In the standard library of classes, the data type stack is an *adapter* class, meaning that a stack is built on top of other data structures. The underlying structure for a stack could be an array, a vector, an Array List, a linked list, or any other collection. Regardless of the type of the underlying data structure, a Stack must implement the same functionality. This

is achieved by providing a unique interface:

```
public interface StackInterface<AnyType>
{
    public void push(AnyType e);
    public AnyType pop();
    public AnyType peek();
    public boolean isEmpty();
}
```

The following picture demonstrates the idea of implementation *by composition*.



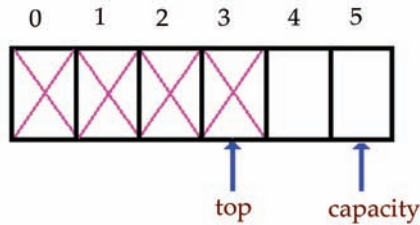
Another implementation requirement (in addition to the above interface) is that all stack operations must run in constant time $O(1)$. Constant time means that there is some constant k such that an operation takes k nanoseconds of computational time regardless of the stack size.

Array-based implementation

In an array-based implementation we maintain the following fields: an array A of a default size (≥ 1), the variable top that refers to the top element in the stack and the $capacity$ that refers to the array size. The variable top changes from -1 to $capacity - 1$. We say that a stack is empty when $top = -1$, and the stack is full when $top = capacity - 1$.

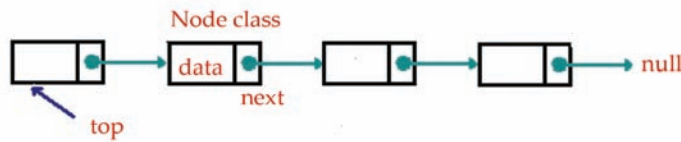
In a fixed-size stack abstraction, the capacity stays unchanged, therefore when top reaches $capacity$, the stack object throws an exception.

In a dynamic stack abstraction when top reaches $capacity$, we double up the stack size.



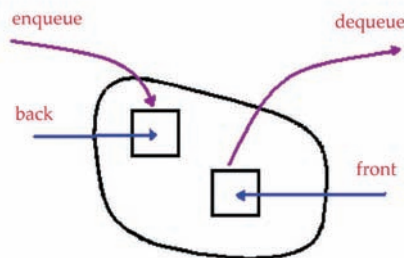
Linked List-based implementation

Linked List-based implementation provides the best (from the efficiency point of view) dynamic stack implementation.



3.5.2 Queue

A queue is a container of objects (a linear collection) that are inserted and removed according to the first-in first-out (FIFO) principle. An excellent example of a queue is a line of students in the food court of the UC. New additions to a line made to the back of the queue, while removal (or serving) happens in the front. In the queue only two operations are allowed enqueue and dequeue. Enqueue means to insert an item into the back of the queue, dequeue means removing the front item. The picture demonstrates the FIFO access.



The difference between stacks and queues is in removing. In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.

Queue Representation

As we now understand that in queue, we access both ends for different reasons. The following diagram given below tries to explain queue representation as data structure –



As in stacks, a queue can also be implemented using Arrays, Linked-lists, Pointers and Structures. For the sake of simplicity, we shall implement queues using one-dimensional array.

Basic Operations

Queue operations may involve initializing or defining the queue, utilizing it, and then completely erasing it from the memory. Here we shall try to understand the basic operations associated with queues –

- enqueue() – add (store) an item to the queue
- dequeue() – remove (access) an item from the queue

Few more functions are required to make the above-mentioned queue operation efficient. These are –

- peek() – Gets the element at the front of the queue without removing it
- isfull() – Checks if the queue is full
- isempty() – Checks if the queue is empty

In queue, we always dequeue (or access) data, pointed by front pointer and while enqueueing (or storing) data in the queue we take help of rear pointer.

Implementation

In the standard library of classes, the data type queue is an *adapter* class, meaning that a queue is built on top of other data structures. The underlying structure for a queue could be an array, a Vector, an ArrayList, a LinkedList, or any other collection. Regardless of the type of the underlying data structure, a queue must implement the same functionality. This is achieved by providing a unique interface.

interface QueueInterface<AnyType>

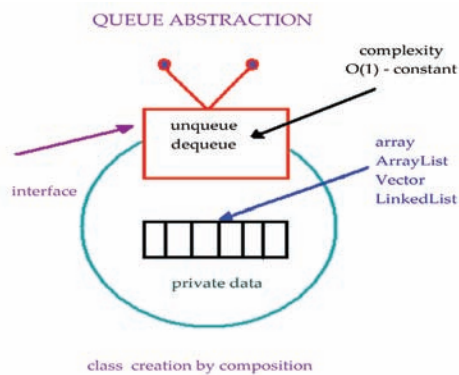
```
{
    public boolean isEmpty();
```

```

public AnyType getFront();
public AnyType dequeue();
public void enqueue(AnyType e);
public void clear();
}

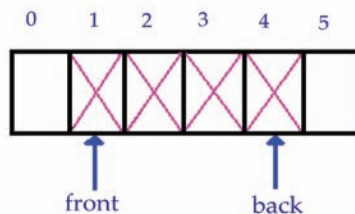
```

Each of the above basic operations must run at constant time $O(1)$. The following picture demonstrates the idea of implementation by composition.

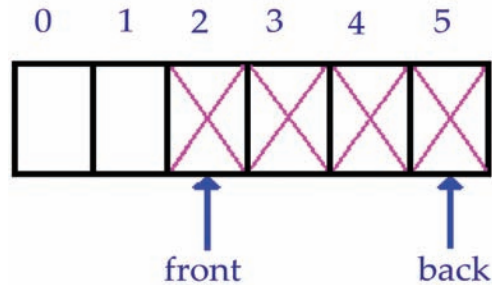


Circular Queue

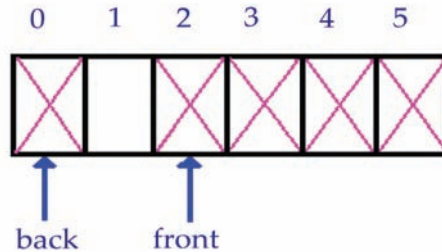
Given an array A of a default size (≥ 1) with two references *back* and *front*, originally set to -1 and 0 respectively. Each time we insert (enqueue) a new item, we increase the back index; when we remove (dequeue) an item - we increase the front index. Here is a picture that illustrates the model after a few steps:



As you see from the picture, the queue logically moves in the array from left to right. After several moves *back* reaches the end, leaving no space for adding new elements.



However, there is a free space before the front index. We shall use that space for enqueueing new items, i.e. the next entry will be stored at index 0, then 1, until *front*. Such a model is called a wrap around queue or a circular queue.



Finally, when *back* reaches *front*, the queue is full. There are two choices to handle a full queue: a) throw an exception; b) double the array size.

The circular queue implementation is done by using the modulo operator (denoted %), which is computed by taking the remainder of division (for example, $8\%5$ is 3). By using the modulo operator, we can view the queue as a circular array, where the “wrapped around” can be simulated as “ $\text{back} \% \text{array_size}$ ”. In addition to the back and front indexes, we maintain another index: *cur* - for counting the number of elements in a queue. Having this index simplifies a logic of implementation.

Applications

The simplest two search techniques are known as Depth-First Search (DFS) and Breadth-First Search (BFS). These two searches are described by looking at how the search tree (representing all the possible paths from the start) will be traversed.

Depth-First Search with a Stack

In depth-first search we go down a path until we get to a dead end; then we *backtrack* or back up (by popping a stack) to get an alternative path.

- Create a stack
- Create a new choice point
- Push the choice point onto the stack
- while (not found and stack is not empty)
 - Pop the stack
 - Find all possible choices after the last one tried
 - Push these choices onto the stack
- Return

Breadth-First Search with a Queue

In breadth-first search we explore all the nearest possibilities by finding all possible successors and enqueue them to a queue.

- Create a queue
- Create a new choice point
- Enqueue the choice point onto the queue
- while (not found and queue is not empty)
 - Dequeue the queue
 - Find all possible choices after the last one tried
 - Enqueue these choices onto the queue
- Return



ROLE MODEL

DONALD ERVIN KNUTH: AMERICAN MATHEMATICIAN AND COMPUTER SCIENTIST

Donald Ervin Knuth, (born Jan. 10, 1938, Milwaukee, Wis., U.S.), American mathematician and computer scientist. Knuth earned a doctorate in mathematics in 1963 from the California Institute of Technology. A pioneer in computer science, he took time out during the 1970s from writing his highly acclaimed multivolume *The Art of Computer Programming* in order to develop TeX, a document-preparation system. Because of its precise control of special characters and mathematical formulas, TeX and its variants soon became standard for submitting typeset-ready scientific and mathematical research papers for publication. Knuth has received many awards and honours, including the Kyoto Prize (1996), the A.M. Turing Award (1974), and the National Medal of Science (1979).

Donald Knuth's parents were Ervin Henry Knuth and Louise Marie Bohning. Donald's father Ervin was a school teacher who taught in a Lutheran school. He played a very important role in determining Donald's interests, and it was through his father that Donald gained his love for education, music, and mathematics. Ervin played the church organ at the Sunday church services and Donald soon became a passionate lover of the organ.

Donald attended Lutheran schools and from the special emphasis that was placed on English grammar in these schools came Knuth's love of investigating sentence structure. His fascination with this in his first couple of years of secondary school would lead naturally towards writing computer code when he eventually encountered computers, but this did not happen until after his school education was complete. During these first years at secondary school there were other signs of where Knuth's interests would eventually lead. One episode, repeated in most biographies of Knuth but still worth repeating in this one, concerns "Ziegler's Giant Bar."

He entered a competition set up by the confectionary manufacturer Ziegler. The aim was to see how many words could be made with the letters of "Ziegler's Giant Bar" and for the schoolboy Knuth this was exactly the sort of challenge that he loved. He spent two weeks during which he pretended to be ill and, using a dictionary, he came up with 4500 words. The judges for the competition had only found 2500 and Knuth was an easy winner. He commented afterwards that had he thought to use the apostrophe he could have found many more! His school benefited by receiving a

television set as a prize. At high school Knuth's interests were more directed towards music than they were to mathematics. His musical interests involved both playing and composing music and he decided at that stage that he would study music after graduating from high school. Knuth played the saxophone, and later the tuba, in his school band. Although he spent much time with his musical interests, Knuth most certainly did not neglect his other school subjects. He graduated from High School in 1956 with the highest grade point average that anyone had ever achieved at his school.

At school he had begun to show an interest in mathematics and he amused himself by trying to visualize surfaces in several dimensions by plotting the graphs obtained by keeping all but one of the variables fixed. This is an excellent way to understand mathematical functions and today with the aid of computers this, and more sophisticated techniques, can quickly give students a deep understanding. However Knuth had to plot his graphs by doing hand calculations for every value which he plotted, showing the same sort of dedication to putting hours of work into problems that he had shown with the "Ziegler's Giant Bar" competition. One might have thought that his teachers would have believed that he could succeed at college in almost any subject he chose given his outstanding school performance, but this was not really so. The problem was that Knuth did not believe in himself at this stage in his life and so his teachers doubted whether he had the personality, in particular the confidence, to succeed.

It shows how undecided Knuth was about the direction his studies might take that when offered a scholarship to Case Institute of Technology in Cleveland, Ohio, to study physics he accepted despite his previous intentions to study music. He entered the physics course at the Case Institute in September 1956. There were really two reasons why, from his second year on, Knuth started to move towards mathematics and away from physics. One day when Knuth was meant to be performing with the College band he missed the bus taking the band to the performance so, finding himself with free time, he tried to solve a challenge problem that one of his mathematics professors had set. Solving it earned Knuth an automatic "A" in that class and also the right sort of boost he needed to think that perhaps mathematics rather than physics was for him. Secondly he found that physics practical did not suit him, so in the end the move towards mathematics became a natural one to make.

In fact Knuth already had his first encounter with computers in his first year at Case before he made the move towards mathematics. He had to use the IBM 650 and consulted the manual to find out how to write programs:-

... the manual we got from IBM would show examples of programs and I knew I could do ... better than that. So I thought I might have some talent.

Knuth used his growing expertise at writing computer programs to produce one in 1958 to analyse the performance of the College basketball team. It led to some publicity and IBM used a photograph of Knuth in their advertising. One might have expected that events would begin to help him overcome his inferiority complex but he still felt that he was not up to standard. This had the effect of making him put



in large amounts of extra work at his academic studies. The result was that when he graduated with his B.S. in June 1960 he was awarded a distinction and, in a quite unique move, the College awarded him a Master's Degree at the same time, such was the brilliance of his performance. Knuth was awarded two Fellowships, a Woodrow Wilson Fellowship and a National Foundation Fellowship in the year of his graduation.

It is a real achievement to publish a mathematics paper while still a doctoral student, but Knuth managed to publish two papers in the year he completed his undergraduate degree. These were *An imaginary number system* and *On methods of constructing sets of mutually orthogonal Latin squares using a computer I* the latter paper being written jointly with R C Bose and I M Chakravarti. In the first Knuth describes an imaginary number system using the imaginary number $2i$ as its base, giving methods for the addition, subtraction and multiplication of the numbers. In the second paper Knuth and his co-authors give two sets of five mutually orthogonal Latin squares of order 12.

In the autumn of 1960 Knuth entered the California Institute of Technology and, in June 1963, he was awarded a Ph.D. in mathematics for his thesis *Finite semifields and projective planes*. In fact in addition to the work for his doctorate in mathematics, Knuth had from 1960 begun to put his very considerable computing expertise to uses other than writing papers becoming a software development consultant to the Burroughs Corporation in Pasadena, California. Knowledge of his computing expertise was so well established by 1962 that, although he was still a doctoral student at the time, Addison-Wesley approached him and asked him to write a text on compilers. He began that project in the summer of 1962.

His publications from this time show that he was applying computing to combinatorial mathematical problems which were not connected to the work he was undertaking for his thesis. For example he computed Euler's constant to 1271 decimal places and published the result in 1962. In the same year he published work on the evaluation of polynomials by computer. Despite Knuth's remarkable mathematical productivity he did find time for other things. During his years as a graduate student Knuth married Nancy Jill Carter on 24 June 1961. Their two children John Martin Knuth and Jennifer Sierra Knuth were born in 1965 and 1966 respectively.

We noted above that the title of Knuth's Ph.D. thesis was *Finite semifields and projective planes*. A semifield is an algebraic structure satisfying all the usual axioms for a division ring except associativity of multiplication. The thesis contains a wealth of information on finite semifields and their connections to certain types of projective planes. After completion of his doctorate in 1963 Knuth became an Assistant Professor of Mathematics at the California Institute of Technology, being promoted to Associate Professor in 1966. From 1964 to 1967 he worked as an Editor of Programming Languages for the Association for Computing Machinery. He continued to apply computing to algebraic and combinatorial mathematics problems. For example in 1964 he published tables of data for finite fields which enabled rapid computer calculations to be carried out. His great love of music, which he had almost devoted his life to, continued and

in 1965 he joined the American Guild of Organists. He would continue to play music, compose music and has even designed of his own pipe organ.

By 1966 his book on compilers had grown to 3000 handwritten pages and Addison-Wesley realized that here was a much more major work than they had originally envisaged. Discussions led to a decision that Knuth should produce a seven volume work covering much more than compilers. The work became *The Art of Computer Programming* and publication began in 1968 when Volume 1: *Fundamental Algorithms* appeared. Volume 2: *Seminumerical algorithms* came out in the following year, and Volume 3: *Sorting and searching* in 1973. In the Preface Knuth writes that these are:-

... books that have been designed to train the reader in the various skills which go into a programmer's craft... [They are] not meant to serve as an introduction to computer programming; the reader is supposed to have some previous experience. [I aim to provide] (a) reference books which summarize the knowledge which has been acquired in several important fields, and (b) textbooks for self-study or for college courses in the computer and information sciences.

Knuth's aim was to:-

... organize and summarize what is known about the fast subject of computer methods and to give it firm mathematical and historical foundations.

... show that the connection between computers and mathematics is far deeper and more intimate than these traditional relationships would imply.

M Muller, reviewing these wonderful books, writes that:-

Knuth has already made a timely and great contribution. He has managed to provide organization of ideas where little existed before; he has provided many ideas which in essence are new and helpful in obtaining a basis of abstraction, integration, or unification of efforts of earlier workers in the various fields covered.

In 1968 Knuth was appointed as Professor of Computer Science at Stanford University. At the same time as he left the California Institute of Technology he also resigned his consultancy position with the Burroughs Corporation. Knuth remained at Stanford University for the remainder of his career. He was appointed Fletcher Jones Professor of Computer Science in 1977 and in 1990 he was named Professor of The Art of Computer Programming. In 1993 he became Professor Emeritus at Stanford University and continued to live on the University Campus.

Knuth has made many contributions to mathematics and computing. One particular contribution we should mention is the Knuth-Bendix algorithm, one of the fundamental algorithms for computing with algebraic structures, particularly with groups and semigroups. This important contribution, published jointly with his student Peter B Bendix in 1970, attempts to solve solve the word problem in algebraic systems by deriving consequences of given relations to give, in some sense, a complete set. Another contribution, which has totally changed the whole way that mathematics is printed and communicated is Knuth's invention of TeX, a language for typesetting mathematical and scientific articles. Starting in 1976 Knuth took ten years off his other



projects to work on the development of TeX and METAFONT, a computer software system for alphabet design.

TeX has changed the technology of mathematics and science publishing since it enables mathematicians and scientists to produce the highest quality of printing of mathematical articles yet this can be achieved simply using a home computer. However, it has not only changed the way that mathematical and scientific articles are published but also in the way that they are communicated. In the 17th century a mathematician would have written a letter to another mathematician and they would discuss their everyday lives in English, French or German, say, but whenever they came to explain a piece of mathematics they would use Latin. Nowadays mathematicians communicate by e-mail and whenever they want to explain a piece of mathematics they require mathematical symbols which almost always they communicate using TeX. Nobody, to our knowledge, has tried to measure the impact of TeX on the level of mathematical production, and indeed this would be a very difficult thing to measure, but nevertheless I [EFR] am certain that the added ease of production and communication of mathematics using TeX has had a major impact on the subject over the last ten years, say.

We should mention a few of the many further contributions by Knuth: semantics of programming languages; attribution grammar; the development of $LR(k)$ parsing; the Knuth-Morris-Pratt algorithm which searches for a string of characters; and structured documentation and literate programming. The work on $LR(k)$ parsing appeared in a 1965 paper *On the translation of languages from left to right*. In this paper Knuth writes:-

There has been much recent interest in languages whose grammar is sufficiently simple that an efficient left-to-right parsing algorithm can be mechanically produced from the grammar. In this paper, we define $LR(k)$ grammars, which are perhaps the most general ones of this type, and they provide the basis for understanding all of the special tricks which have been used in the construction of parsing algorithms for languages with simple structure, e.g., algebraic languages.

The Knuth-Morris-Pratt pattern matching algorithm was published in the 1977 paper *Fast pattern matching in strings*. Knuth continues to publish important contributions to computer science, combinatorics and algebra, the topic of his doctoral thesis. For example in the latter area he published *Efficient representation of perm groups* in 1991. He writes in the introduction:-

This note presents an elementary version of C C Sims's algorithm for computing strong generators of a given perm group, together with a proof of correctness and some notes about appropriate low-level data structures.

For his quite remarkable contributions Knuth has received many honours - far too many to be mentioned in an article of this length. Let us just list a small selection. He was the first recipient of the Grace Murray Hopper Award from the Association for Computing Machinery in 1971; he was elected a Fellow of the American Academy of Arts and Science in 1973; in 1974 he won the Alan M Turing Award from the Association

for Computing Machinery; he was elected to the National Academy of Sciences in 1975; in the same year he won the Lester R Ford Award from the Mathematical Association of America; he was awarded the National Science Medal in 1979 (presented to him by President Carter); he was elected to the National Academy of Engineering in 1981; he was elected an honorary member of the IEEE in 1982 and awarded their Computer Pioneer Award in the same year; he was awarded the Steele Prize for Expository Writing from the American Mathematical Society in 1986; he was awarded the Franklin Medal in 1988; he was elected to the Académie des Sciences in 1992; he was awarded the Adelskold Medal from the Swedish Academy of Sciences in 1994; he was awarded the John von Neumann Medal from the IEEE in 1995; and the Kyoto Prize from the Inamori Foundation in 1996.

Let us mention a few of the honors that Knuth has received since 2000. He has received honorary degrees from a large number of universities world-wide: Waterloo University, Canada (2000), Tübingen University (2001), the University of Oslo (2002), Antwerp University (2003), Harvard University (2003), the University of Macedonia (2003), Montreal University (2004), ETH Zürich (2005), Concordia University (2006), Wisconsin University (2006), the University of Bordeaux (2007). In 2003 he was elected to the Royal Society of London, and in 2008 to the Russian Academy of Sciences. He was awarded the Gold Medal from the State Engineering University of Armenia in 2006, and in the same year the gold medal from Yerevan State University. In 2001 the minor planet “(21656) Knuth” was named after him.

In 2015 Knuth was elected to Honorary Membership of the London Mathematical Society in its 150th Anniversary year.

SUMMARY

- Data Structures are the programmatic way of storing data so that data can be used efficiently.
- Data Structures is about rendering data elements in terms of some relationship, for better organization and storage.
- If the data is stored in well-organized way on storage media and in computer's memory then it can be accessed quickly for processing that further reduces the latency and the user is provided fast response.
- The functional definition of a data structure is known as ADT (Abstract Data Type) which is independent of implementation.
- Data type is a way to classify various types of data such as integer, string, etc. which determines the values that can be used with the corresponding type of data, the type of operations that can be performed on the corresponding type of data.
- Linked list data structure provides better memory management than arrays. Because linked list is allocated memory at run time, so, there is no waste of memory.

KNOWLEDGE CHECK

1. **Linked list search complexity is**
 - a. $O(1)$
 - b. $O(n)$
 - c. $O(\log n)$
 - d. $O(\log \log n)$
2. **A linked-list is a dynamic structure**
 - a. true
 - b. false
3. **Queue data structure works on**
 - a. LIFO
 - b. FIFO
 - c. FILO
 - d. none of the above

4. **Graph traversal is different from a tree traversal, because**
 - a. trees are not connected
 - b. graphs may have loops
 - c. trees have root
 - d. None is true as tree is a subset of graph
5. **In conversion from prefix to postfix using stack data-structure, if operators and operands are pushed and popped exactly once, then the run-time complexity is –**
 - a. $O(1)$
 - b. $O(n)$
 - c. $O(\log n)$
 - d. $O(n^2)$

CHECK YOUR RESULT

1. (b) 2. (a) 3. (b) 4. (c) 5. (b)

REVIEW QUESTIONS

1. Define the classification of data structures.
2. Give five orderings of the keys A X C S E R H that, when inserted into an initially empty BST, produce the best-case tree.
3. Differentiate between singly and doubly linked lists.
4. What do you mean by binary search tree?
5. Elaborate stack and queue.

REFERENCES

1. Tutorialspoint, https://www.tutorialspoint.com/data_structures_algorithms/
2. Tutorialspoint, http://www.tutorialspoint.com/data_structures_algorithms/data_structures_algorithms_tutorial.pdf
3. <http://cs-fundamentals.com/data-structures/introduction-to-data-structures.php>
4. Studytonight, <http://www.studytonight.com/data-structures/introduction-to-data-structures>

5. <http://lib.mdp.ac.id/ebook/Karya%20Umum/Dsa.pdf>
6. <http://introcs.cs.princeton.edu/java/43stack/>
7. <https://www.cs.cmu.edu/~adamchik/15-121/lectures/Stacks%20and%20Queues/Stacks%20and%20Queues.html>

SEARCHING AND SORTING TECHNIQUES

CHAPTER

4

LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Explain the searching algorithms
2. Define the sorting algorithm

INTRODUCTION

How do you find someone's phone number in the phone book? How do you find your keys when you've misplaced them? If a deck of cards has less than 52 cards, how do you determine which card is missing? Searching for items and sorting through items are tasks that we do every day. Searching and sorting are also common tasks in computer programs. We search for all occurrences of a word in a file in order to replace it with another word. We sort the items on a list into alphabetical or numerical order. Because searching and sorting are common computer tasks, we have well-known algorithms, or recipes, for doing searching and sorting.

4.1 SEARCHING ALGORITHMS

An algorithm is a step-by-step procedure or method for solving a problem by a computer in a given number of steps. The steps of an algorithm may include repetition depending upon the problem for which the algorithm is being developed. The algorithm is written in human readable and understandable form.

4.1.1 Linear Search



KEYWORD

Sequential search is a method for finding a target value within a list. It sequentially checks each element of the list for the target value until a match is found or until all the elements have been searched.

Linear search is a very simple search algorithm. A **sequential search** is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.

```
int[] a = {5, 6, 4, 9, 1, 3};
int num = 9; boolean b = false;
for(int i = 0; i < a.length; i++) {
    if(a[i] == num) {
        b = true;
        break;
    }
}
if(b) {
    // Found
}
```

Linear search

Algorithm

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i + 1$

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit

Pseudocode

```

procedure linear_search (list, value)
  for each item in the list
    if match item == value
      return the item's location
    end if
  end for
end procedure

```



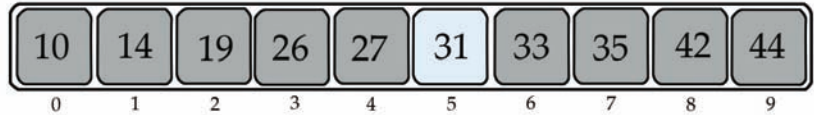
In 1946, John Mauchly made the first mention of binary search as part of the Moore School Lectures, the first ever set of lectures regarding any computer-related topic.

4.1.2 Binary Search

Binary search is a fast search algorithm with run-time complexity of $O(\log n)$. This search algorithm works on the principle of divide and conquer. For this algorithm to work properly, the data collection should be in the sorted form. Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the right of the middle item. Otherwise, the item is searched for in the sub-array to the left of the middle item. This process continues on the sub-array as well until the size of the sub-array reduces to zero.

How Binary Search Works?

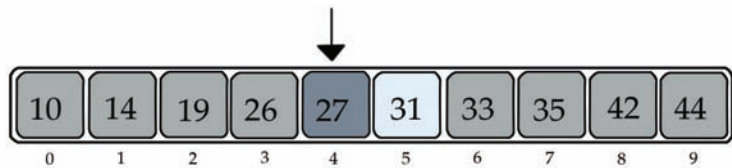
For a binary search to work, it is mandatory for the target array to be sorted. We shall learn the process of binary search with a pictorial example. The following is our sorted array and let us assume that we need to search the location of value 31 using binary search.



First, we shall determine half of the array by using this formula –

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Here it is, $0 + (9 - 0) / 2 = 4$ (integer value of 4.5). So, 4 is the mid of the array.

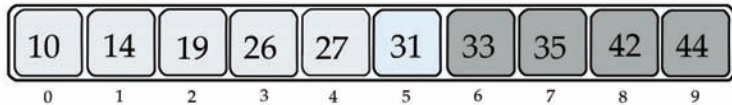


Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match.



REMEMBER

The value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

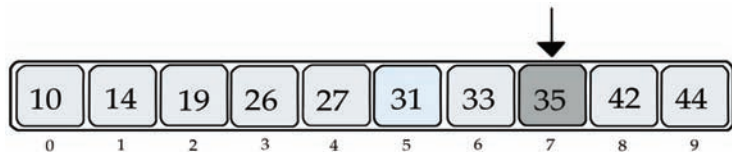


We change our low to mid + 1 and find the new mid value again.

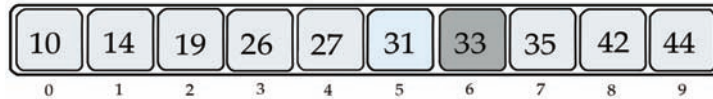
$$\text{low} = \text{mid} + 1$$

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

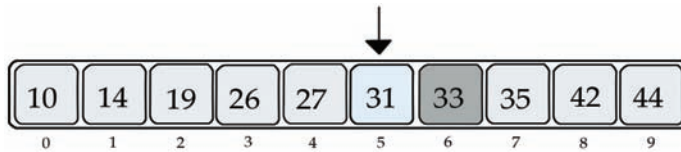
Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.



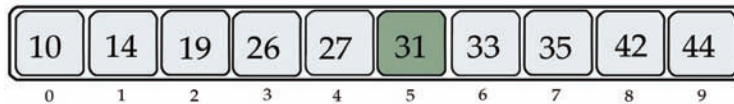
The value stored at location 7 is not a match, rather it is less than what we are looking for. So, the value must be in the lower part from this location.



Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.



We conclude that the target value 31 is stored at location 5.

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Pseudocode

The pseudocode of binary search algorithms should look like this –

Procedure binary_search

A ← sorted array

n ← size of array

x ← value to be searched

Set lowerBound = 1

Set upperBound = n

while x not found

if upperBound < lowerBound

EXIT: x does not exist.

```

set midPoint = lowerBound + ( upperBound - lowerBound ) / 2
if A[midPoint] < x
set lowerBound = midPoint + 1

```

```

if A[midPoint] > x
set upperBound = midPoint - 1
if A[midPoint] = x
EXIT: x found at location midPoint
end while

```

```

end procedure

```



Binary search is a digital scheme for locating a specific object in a large set. Each object in the set is given a key.

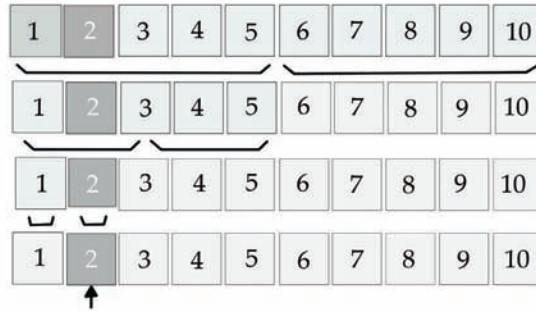
4.1.3 Interpolation Search

Interpolation search is an improved variant of **binary search**. This search algorithm works on the probing position of the required value. For this algorithm to work properly, the data collection should be in a sorted form and equally distributed.

Binary search has a huge advantage of time complexity over linear search. Linear search has worst-case complexity of $O(n)$ whereas binary search has $O(\log n)$. There are cases where the location of target data may be known in advance. For example, in case of a telephone directory, if we want to search the telephone number of Morphiuss. Here, linear search and even binary search will seem slow as we can directly jump to memory space where the names start from 'M' are stored.

Positioning in Binary Search

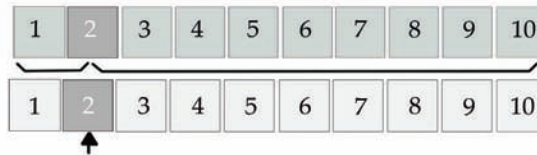
In binary search, if the desired data is not found then the rest of the list is divided in two parts, lower and higher. The search is carried out in either of them.



Even when the data is sorted, binary search does not take advantage to probe the position of the desired data.

Position Probing in Interpolation Search

Interpolation search finds a particular item by computing the probe position. Initially, the probe position is the position of the middle most item of the collection.



If a match occurs, then the index of the item is returned. To split the list into two parts, we use the following method –

$$\text{mid} = \text{Lo} + ((\text{Hi} - \text{Lo}) / (\text{A}[\text{Hi}] - \text{A}[\text{Lo}])) * (\text{X} - \text{A}[\text{Lo}])$$

where –

A = list

Lo = Lowest index of the list

Hi = Highest index of the list

A[n] = Value stored at index n in the list

If the middle item is greater than the item, then the probe position is again calculated in the sub-array to the right of the middle item. Otherwise, the item is searched in the sub-array to the left of the middle item. This process continues on the sub-array as well until the size of sub-array reduces to zero. Runtime complexity of interpolation search algorithm is $O(\log(\log n))$ as compared to $O(\log n)$ of BST in favorable situations.

Algorithm

As it is an improvisation of the existing BST algorithm, we are mentioning the steps to search the 'target' data value index, using position probing –

Step 1 – Start searching data from middle of the list.

Step 2 – If it is a match, return the index of the item, and exit.

Step 3 – If it is not a match, probe position.

Step 4 – Divide the list using probing formula and find the new middle.

Step 5 – If data is greater than middle, search in higher sub-list.

Step 6 – If data is smaller than middle, search in lower sub-list.

Step 7 – Repeat until match.

Pseudocode

A → Array list

N → Size of A

X → Target Value

Procedure Interpolation_Search()

Set Lo → 0

Set Mid → -1

Set Hi → N-1

While X does not match

if Lo equals to Hi OR A[Lo] equals to A[Hi]

EXIT: Failure, Target not found

end if

Set Mid = Lo + ((Hi - Lo) / (A[Hi] - A[Lo])) * (X - A[Lo])


```

if A[Mid] = X
EXIT: Success, Target found at Mid
else
if A[Mid] < X
Set Lo to Mid+1
else if A[Mid] > X
Set Hi to Mid-1
end if
end if
End While
End Procedure

```

4.1.4 Hash Table

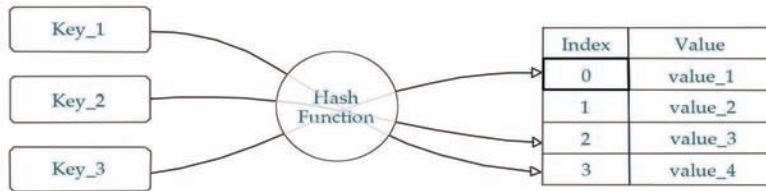
Hash Table is a data structure which stores data in an associative manner. In a hash table, data is stored in an array format, where each data value has its own unique index value. Access of data becomes very fast if we know the index of the desired data. Thus, it becomes a **data structure** in which insertion and search operations are very fast irrespective of the size of the data. Hash Table uses an array as a storage medium and uses hash technique to generate an index where an element is to be inserted or is to be located from.

Hashing

Hashing is a technique to convert a range of key values into a range of indexes of an array. We're going to use modulo operator to get a range of key values. Consider an example of hash table of size 20, and the following items are to be stored. Item are in the (key, value) format.



Data structure is a particular way of organizing data in a computer so that it can be used efficiently.



- (1,20)
- (2,70)
- (42,80)
- (4,25)
- (12,44)
- (14,32)
- (17,11)
- (13,78)
- (37,98)

Sr. No.	Key	Hash	Array Index
1	1	$1 \% 20 = 1$	1
2	2	$2 \% 20 = 2$	2
3	42	$42 \% 20 = 2$	2
4	4	$4 \% 20 = 4$	4
5	12	$12 \% 20 = 12$	12
6	14	$14 \% 20 = 14$	14
7	17	$17 \% 20 = 17$	17
8	13	$13 \% 20 = 13$	13
9	37	$37 \% 20 = 17$	17

Linear Probing

It may happen that the hashing technique is used to create an already used index of the array. We can search the next empty location in the array by looking into the next cell until we find an empty cell. This technique is called linear probing.

Sr. No.	Key	Hash	Array Index	After Linear Probing, Array Index
1	1	$1 \% 20 = 1$	1	1
2	2	$2 \% 20 = 2$	2	2
3	42	$42 \% 20 = 2$	2	3
4	4	$4 \% 20 = 4$	4	4
5	12	$12 \% 20 = 12$	12	12
6	14	$14 \% 20 = 14$	14	14

7	17	$17 \% 20 = 17$	17	17
8	13	$13 \% 20 = 13$	13	13
9	37	$37 \% 20 = 17$	17	18

Basic Operations

Following are the basic primary operations of a hash table.

- Search – Searches an element in a hash table.
- Insert – inserts an element in a hash table.
- Delete – Deletes an element from a hash table.

Data Item

Define a data item having some data and key, based on which the search is to be conducted in a hash table.

```
struct DataItem {
    int data;
    int key;
};
```

Hash Method

Define a hashing method to compute the hash code of the key of the data item.

```
int hashCode(int key){
    return key % SIZE;
}
```

Search Operation

Whenever an element is to be searched, compute the hash code of the key passed and locate the element using that hash code as index in the array. Use linear probing to get the element ahead if the element is not found at the computed hash code.

```
struct DataItem *search(int key){
    //get the hash
    int hashIndex = hashCode(key);
    //move in array until an empty
    while(hashArray[hashIndex] != NULL){
        if(hashArray[hashIndex]->key == key)
            return hashArray[hashIndex];
        //go to next cell
        ++hashIndex;
    }
    //wrap around the table
    hashIndex %= SIZE;
```

```

}
return NULL;
}

```

Insert Operation

Whenever an element is to be inserted, compute the hash code of the key passed and locate the index using that hash code as an index in the array. Use linear probing for empty location, if an element is found at the computed hash code.

```

void insert(int key,int data){
    struct DataItem *item = (struct DataItem*) malloc(sizeof(struct DataItem));
    item->data = data;
    item->key = key;
    //get the hash
    int hashIndex = hashCode(key);
    //move in array until an empty or deleted cell
    while(hashArray[hashIndex] != NULL && hashArray[hashIndex]->key != -1){
        //go to next cell
        ++hashIndex;
    }
    //wrap around the table
    hashIndex %= SIZE;
    hashArray[hashIndex] = item;
}

```

Delete Operation

Whenever an element is to be deleted, compute the hash code of the key passed and locate the index using that hash code as an index in the array. Use linear probing

to get the element ahead if an element is not found at the computed hash code. When found, store a dummy item there to keep the performance of the hash table intact.

```
struct DataItem* delete(struct DataItem* item){
    int key = item->key;
    //get the hash
    int hashIndex = hashCode(key);
    //move in array until an empty
    while(hashArray[hashIndex] !=NULL){
        if(hashArray[hashIndex]->key == key){
            struct DataItem* temp = hashArray[hashIndex];
            //assign a dummy item at deleted position
            hashArray[hashIndex] = dummyItem;
            return temp;
        }
        //go to next cell
        ++hashIndex;
        //wrap around the table
        hashIndex %= SIZE;
    }
    return NULL;
}
```



KEYWORD

Lexicographical order is a generalization of the way the alphabetical order of words is based on the alphabetical order of their component letters.

4.2 SORTING ALGORITHM

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular

order. Most common orders are in numerical or **lexicographical order**. The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats. Following are some of the examples of sorting in real-life scenarios:

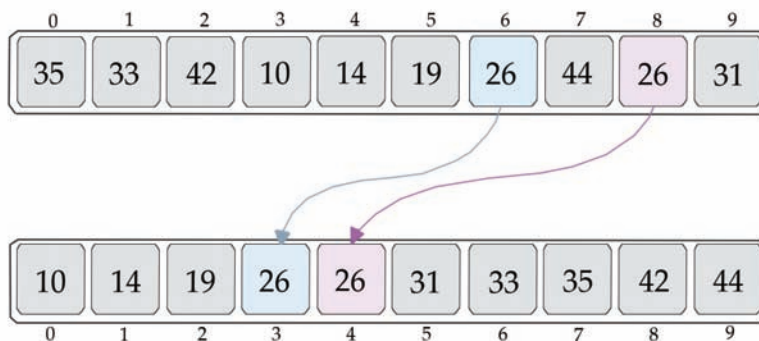
- Telephone Directory – The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
- Dictionary – The dictionary stores words in an alphabetical order so that searching of any word becomes easy.

In-place Sorting and Not-in-place Sorting

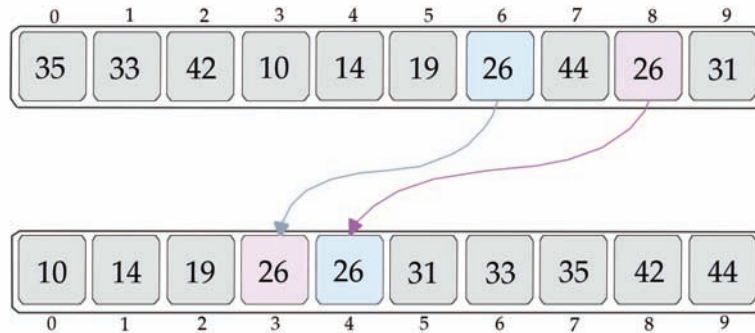
Sorting algorithms may require some extra space for comparison and temporary storage of few data elements. These algorithms do not require any extra space and sorting is said to happen in-place, or for example, within the array itself. This is called in-place sorting. Bubble sort is an example of in-place sorting. However, in some sorting algorithms, the program requires space which is more than or equal to the elements being sorted. Sorting which uses equal or more space is called not-in-place sorting. Merge-sort is an example of not-in-place sorting.

Stable and Not Stable Sorting

If a sorting algorithm, after sorting the contents, does not change the sequence of similar content in which they appear, it is called stable sorting.



If a sorting algorithm, after sorting the contents, changes the sequence of similar content in which they appear, it is called unstable sorting.



Stability of an algorithm matters when we wish to maintain the sequence of original elements.

Adaptive and Non-Adaptive Sorting Algorithm

A sorting algorithm is said to be adaptive, if it takes advantage of already 'sorted' elements in the list that is to be sorted. That is, while sorting if the source list has some element already sorted, adaptive algorithms will take this into account and will try not to re-order them.

A non-adaptive algorithm is one which does not take into account the elements which are already sorted. They try to force every single element to be re-ordered to confirm their sortedness.

Important Terms

Some terms are generally coined while discussing sorting techniques –

Increasing Order

A sequence of values is said to be in increasing order, if the successive element is greater than the previous one. For example, 1, 3, 4, 6, 8, 9 are in increasing order, as every next element is greater than the previous element.

Decreasing Order

A sequence of values is said to be in decreasing order, if the successive element is less than the current one. For example, 9, 8, 6, 4, 3, 1 are in decreasing order, as every next element is less than the previous element.

Non-Increasing Order

A sequence of values is said to be in non-increasing order, if the successive element is less than or equal to its previous element in the sequence. This order occurs when the sequence contains duplicate values. For example, 9, 8, 6, 3, 3, 1 are in non-increasing order, as every next element is less than or equal to (in case of 3) but not greater than any previous element.

Non-Decreasing Order

A sequence of values is said to be in non-decreasing order, if the successive element is greater than or equal to its previous element in the sequence. This order occurs when the sequence contains duplicate values. For example, 1, 3, 3, 6, 8, 9 are in non-decreasing order, as every next element is greater than or equal to (in case of 3) but not less than the previous one.

4.2.1 Bubble Sort Algorithm

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where n is the number of items.

How Bubble Sort Works?

We take an unsorted array for our example. Bubble sort takes $O(n^2)$ time so we're keeping it short and precise.



Bubble sort starts with very first two elements, comparing them to check which one is greater.



Value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this –



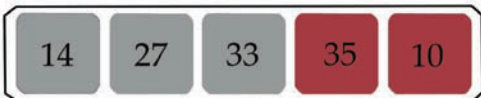
Next we compare 33 and 35. We find that both are in already sorted positions.



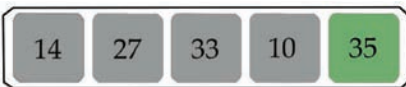
Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.



We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –



To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –



Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sort learns that an array is completely sorted.



Now we should look into some practical aspects of bubble sort.

Algorithm

We assume list is an array of n elements. We further assume that swap function swaps the values of the given array elements.

```
begin BubbleSort(list)
```

```
  for all elements of list
```

```
    if list[i] > list[i+1]
```

```
      swap(list[i], list[i+1])
```

```
    end if
```

```
  end for
```

```
  return list
```

```
end BubbleSort
```

Pseudocode

We observe in algorithm that Bubble Sort compares each pair of array element unless the whole array is completely sorted in an ascending order. This may cause a few complexity issues like what if the array needs no more swapping as all the elements are already ascending. To ease-out the issue, we use one flag variable swapped which will help us see if any swap has happened or not. If no swap has occurred, i.e. the array requires no more processing to be sorted, it will come out of the loop.

Pseudocode of BubbleSort algorithm can be written as follows –
 procedure bubbleSort(list : array of items)

```
  loop = list.count;
```

```
for i = 0 to loop-1 do:
  swapped = false
  for j = 0 to loop-1 do:
    /* compare the adjacent elements */
    if list[j] > list[j+1] then
      /* swap them */
      swap( list[j], list[j+1] )
    swapped = true
  end if
end for

/*if no number was swapped that means
array is sorted now, break the loop.*/

if(not swapped) then
  break
end if

end for

end procedure return list
```

Implementation

One more issue we did not address in our original algorithm and its improvised pseudocode, is that, after every iteration the highest values settles down at the end



of the array. Hence, the next iteration need not include already sorted elements. For this purpose, in our implementation, we restrict the inner loop to avoid already sorted values.

4.2.2 Insertion Sort

This is an in-place comparison-based sorting algorithm. Here, a sub-list is maintained which is always sorted. For example, the lower part of an array is maintained to be sorted. An element which is to be 'insert'ed in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence the name, insertion sort.

The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array). This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$, where n is the number of items.

How Insertion Sort Works?

We take an unsorted array for our example.



Insertion sort compares the first two elements.



It finds that both 14 and 33 are already in ascending order. For now, 14 is in sorted sublist.



Insertion sort moves ahead and compares 33 with 27.



And finds that 33 is not in the correct position.



It swaps 33 with 27. It also checks with all the elements of sorted sub-list. That the sorted sub-list has only one element 14, and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.



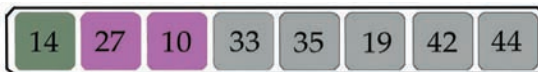
By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10.



So we swap them.



However, swapping makes 27 and 10 unsorted.



Hence, we swap them too.



Again we find 14 and 10 in an unsorted order.



We swap them again. By the end of third iteration, we have a sorted sub-list of 4 items



This process goes on until all the unsorted values are covered in a sorted sub-list.

Algorithm

We can derive simple steps by which we can achieve insertion sort

Step 1 – If it is the first element, it is already sorted. return 1;

Step 2 – Pick next element



Step 3 – Compare with all elements in the sorted sub-list

Step 4 – Shift all the elements in the sorted sub-list that is greater than the value to be sorted

Step 5 – Insert the value

Step 6 – Repeat until list is sorted

Pseudocode

```

procedure insertionSort( A : array of items )
  int holePosition
  int valueToInsert
  for i = 1 to length(A) inclusive do:
    /* select value to be inserted */
    valueToInsert = A[i]
    holePosition = i

    /*locate hole position for the element to be inserted */
    while holePosition > 0 and A[holePosition-1] > valueToInsert
      do:
        A[holePosition] = A[holePosition-1]
        holePosition = holePosition -1
    end while
    /* insert the number at hole position */
    A[holePosition] = valueToInsert

  end for
end procedure

```

4.2.3 Selection Sort

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list. The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the **sorted array**. This process continues moving unsorted array boundary by one element to the right. This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where n is the number of items.

How Selection Sort Works?

Consider the following depicted array as an example.



For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.



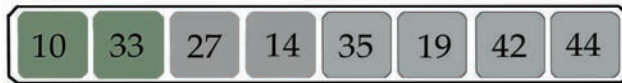
For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.



We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.

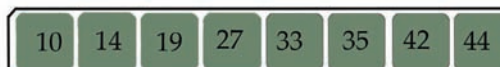


After two iterations, two least values are positioned at the beginning in a sorted manner.



The same process is applied to the rest of the items in the array.

Following is a pictorial depiction of the entire sorting process –



Sorted array is an array data structure in which each element is sorted in numerical, alphabetical, or some other order, and placed at equally spaced addresses in computer memory.

Now, some programming aspects of selection sort.

Algorithm

Step 1 – Set MIN to location 0

Step 2 – Search the minimum element in the list

Step 3 – Swap with value at location MIN

Step 4 – Increment MIN to point to next element

Step 5 – Repeat until list is sorted



REMEMBER

The important property is that the extra storage required should not increase as the size of the input increases.

Pseudocode

procedure selection sort

list : array of items

n : size of list

for i = 1 to n - 1

/* set current element as minimum*/

min = i

/* check the element to be minimum */

for j = i+1 to n

if list[j] < list[min] then

min = j;

end if

end for

/* swap the minimum element with the current element*/

if indexMin != i then

swap list[min] and list[i]



```

end if
end for
end procedure

```

4.2.4 Merge Sort Algorithm

Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being $O(n \log n)$, it is one of the most respected algorithms. Merge sort first divides the array into equal halves and then combines them in a sorted manner.

How Merge Sort Works?

To understand merge sort, we take an unsorted array as the following –



We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.



We further divide these arrays and we achieve atomic value which can no more be divided.



Now, we combine them in exactly the same manner as they were broken down. We first compare the element for each list and then combine them into another list in a sorted manner. That 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.



After the final merging, the list should look like this –



Algorithm

Merge sort keeps on dividing the list into equal halves until it can no more be divided. By definition, if it is only one element in the list, it is sorted. Then, merge sort combines the smaller sorted lists keeping the new list sorted too.

Step 1 – if it is only one element in the list it is already sorted, return.

Step 2 – divide the list recursively into two halves until it can no more be divided.

Step 3 – merge the smaller lists into new list in sorted order.

Pseudocode

We shall now see the pseudocodes for merge sort functions. As our algorithms point out two main functions – divide & merge.

```
procedure mergesort( var a as array )
```

```

if ( n == 1 ) return a
var l1 as array = a[0] ... a[n/2]
var l2 as array = a[n/2+1] ... a[n]
l1 = mergesort( l1 )
l2 = mergesort( l2 )
return merge( l1, l2 )
end procedure

procedure merge( var a as array, var b as array )
var c as array
while ( a and b have elements )
if ( a[0] > b[0] )
add b[0] to the end of c
remove b[0] from b
else
add a[0] to the end of c
remove a[0] from a
end if
end while

while ( a has elements )
add a[0] to the end of c
remove a[0] from a
end while

while ( b has elements )

```

add $b[0]$ to the end of c

remove $b[0]$ from b

end while

return c

end procedure

4.2.5 Shell Sort

Shell sort is a highly efficient sorting algorithm and is based on insertion sort algorithm. This algorithm avoids large shifts as in case of insertion sort, if the smaller value is to the far right and has to be moved to the far left. This algorithm uses insertion sort on a widely spread elements, first to sort them and then sorts the less widely spaced elements. This spacing is termed as interval. This interval is calculated based on Knuth's formula as –

$$h = h * 3 + 1$$

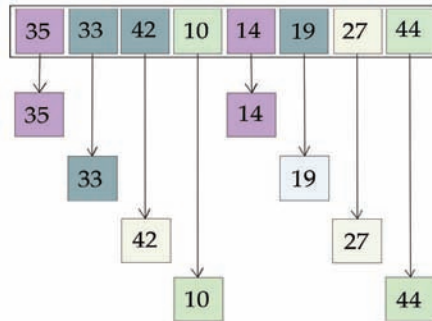
where –

h is interval with initial value 1

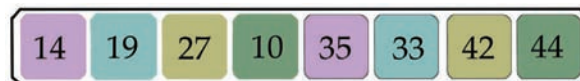
This algorithm is quite efficient for medium-sized data sets as its average and worst case complexity are of $O(n)$, where n is the number of items.

How Shell Sort Works?

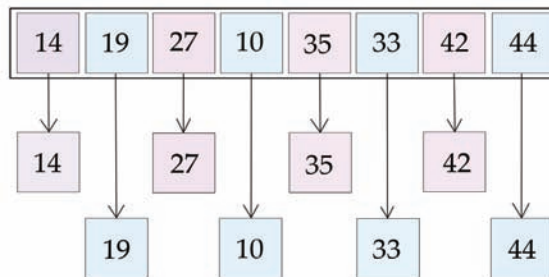
Let us consider the following example to have an idea of how shell sort works. For our example and ease of understanding, we take the interval of 4. Make a virtual sub-list of all values located at the interval of 4 positions. Here these values are {35, 14}, {33, 19}, {42, 27} and {10, 14}



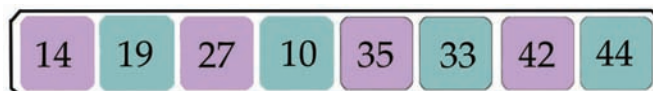
We compare values in each sub-list and swap them (if necessary) in the original array. After this step, the new array should look like this –



Then, we take interval of 2 and this gap generates two sub-lists - {14, 27, 35, 42}, {19, 10, 33, 44}

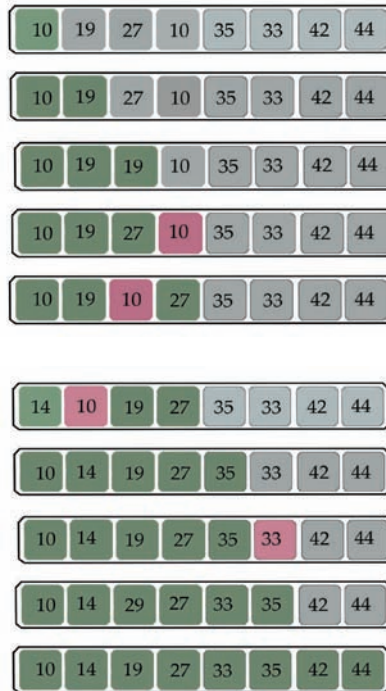


We compare and swap the values, if required, in the original array. After this step, the array should look like this –



Finally, we sort the rest of the array using interval of value 1. Shell sort uses insertion sort to sort the array.

Following is the step-by-step depiction –



That it required only four swaps to sort the rest of the array.

Algorithm

Following is the algorithm for shell sort.

Step 1 – Initialize the value of h

Step 2 – Divide the list into smaller sub-list of equal interval h

Step 3 – Sort these sub-lists using insertion sort

Step 3 – Repeat until complete list is sorted

Pseudocode

Following is the pseudocode for shell sort.

```
procedure shellSort()
```



A : array of items

/* calculate interval*/

while interval < A.length /3 do:

interval = interval * 3 + 1

end while

while interval > 0 do:

for outer = interval; outer < A.length; outer ++ do:

/* select value to be inserted */

valueToInsert = A[outer]

inner = outer;

/*shift element towards right*/

while inner > interval -1 && A[inner - interval] >= valueToInsert do:

A[inner] = A[inner - interval]

inner = inner - interval

end while

/* insert the number at hole position */

A[inner] = valueToInsert

end for

/* calculate interval*/

interval = (interval -1) /3;

end while

end procedure



Sorting algorithm is an algorithm that puts elements of a list in a certain order. The most-used orders are numerical order and lexicographical order.

4.2.6 Quick Sort

Quick sort is a highly efficient **sorting algorithm** and is based on partitioning of array of data into smaller arrays. A large array is partitioned into two arrays one of which holds values smaller than the specified value, say pivot, based on which the partition is made and another array holds values greater than the pivot value. Quick sort partitions an array and then calls itself recursively twice to sort the two resulting sub-arrays. This algorithm is quite efficient for large-sized data sets as its average and worst case complexity are of $O(n \log n)$, where n is the number of items.

Partition in Quick Sort

Following animated representation explains how to find the pivot value in an array.



The pivot value divides the list into two parts. And recursively, we find the pivot for each sub-lists until all lists contains only one element.

Quick Sort Pivot Algorithm

Based on our understanding of partitioning in quick sort, we will now try to write an algorithm for it, which is as follows.

Step 1 – Choose the highest index value has pivot

Step 2 – Take two variables to point left and right of the list excluding pivot

Step 3 – left points to the low index

Step 4 – right points to the high

Step 5 – while value at left is less than pivot move right

Step 6 – while value at right is greater than pivot move left

Step 7 – if both step 5 and step 6 does not match swap left and right

Step 8 – if $\text{left} \geq \text{right}$, the point where they met is new pivot

Quick Sort Pivot Pseudocode

The pseudocode for the above algorithm can be derived as –

```
function partitionFunc(left, right, pivot)
    leftPointer = left - 1
    rightPointer = right
    while True do
        while A[++leftPointer] < pivot do
            //do-nothing
        end while
        while rightPointer > 0 && A[--rightPointer] > pivot do
            //do-nothing
        end while
        if leftPointer >= rightPointer
            break
        else
            swap leftPointer, rightPointer
        end if
    end while
    swap leftPointer, right
    return leftPointer
end function
```

Quick Sort Algorithm

Using pivot algorithm recursively, we end up with smaller possible partitions. Each partition is then processed for quick sort. We define recursive algorithm for quicksort as follows –

Step 1 – Make the right-most index value pivot

Step 2 – partition the array using pivot value

Step 3 – quicksort left partition recursively

Step 4 – quicksort right partition recursively

Quick Sort Pseudocode

To get more into it, the pseudocode for quick sort algorithm –

procedure quickSort(left, right)

if right-left $\leq$ 0

return

else

pivot = A[right]

partition = partitionFunc(left, right, pivot)

quickSort(left,partition-1)

quickSort(partition+1,right)

end if

end procedure



ROLE MODEL

BETTY HOLBERTON: THE FIRST COMPUTER PROGRAMMERS,

Betty Holberton was born in 1917 in Philadelphia as Frances Elizabeth Snyder. She attended the University of Pennsylvania during the 1940s and studied journalism which allowed her to travel a lot. During the Second World War, when men were fighting in the war, the US army was hiring women to calculate ballistic trajectories. Betty Snyder was chosen by the Moore School of Engineering and soon by the ENIAC along with Ruth Teitelbaum, Marlyn Meltzer and others as a 'computer'. The Moore School at UPenn was funded by the US Army during the Second World War. Here a group of about 80 women worked manually calculating ballistic trajectories - complex differential calculations. These women were called 'computers'. In 1945, the Army decided to fund an experimental project – the first all-electronic digital computer and six of the women 'computers' were selected to be its first programmers. Among these six was Betty Holberton.

The ENIAC was the first all-electronic digital computer, a huge machine of forty black 8-foot panels. The programmers had none of the programming tools of today and it was a challenge to make the ENIAC work. The six programmers had to physically conduct the ballistic program using 3000 switches and dozens of switches and digital trays to route the data and program pulses through the machine. They used analog technology to calculate ballistic trajectory equations. Initially the ENIAC was classified. In 1946, the ENIAC computer was unveiled before the public and the press. The six women were the only generation of programmers to program the ENIAC.

After the war, Holberton worked at Remington Rand, a business machines manufacturer and then at the National Bureau of Standards, a measurement standards laboratory. Here she worked on the first two revisions of the FORTRAN language standard – FORTRAN77 and FORTRAN90. In 1959, she was the Chief of the Programming Research Branch, Applied Mathematics Laboratory at the David Taylor Model Basin, one of the largest ship model basins test facilities in the world. Holberton also worked for the Eckert-Mauchly Computer Corporation which had departed from UPenn's Moore School of Electrical Engineering. Here, she worked on the BINAC, the first commercial digital computer in the world and on the UNIVAC, a line of electronic digital stored-program computers. Holberton also wrote the first statistical analysis package used for the 1950 US census.

In the last few years of her life, Holberton received recognition as a computer pioneer. In 1997, along with the six other original ENIAC programmers, she was inducted into the Women in Technology International Hall of Fame. She was, however, the only one out of the six ENIAC programmers to receive the Augusta Ada Lovelace Award, the highest award given by the Association of Women in computing. In 1997 again, she received the IEEE Computer Pioneer Award from the IEEE Computer Society. Holberton died in Rockville, Maryland in 2001.

SUMMARY

- An algorithm is a step-by-step procedure or method for solving a problem by a computer in a given number of steps. The steps of an algorithm may include repetition depending upon the problem for which the algorithm is being developed.
- Linear search is a very simple search algorithm. A sequential search is made over all items one by one.
- Binary search is a fast search algorithm with run-time complexity of $O(\log n)$. This search algorithm works on the principle of divide and conquer.
- Interpolation search is an improved variant of binary search. This search algorithm works on the probing position of the required value.
- Hash Table is a data structure which stores data in an associative manner. In a hash table, data is stored in an array format, where each data value has its own unique index value. Access of data becomes very fast if we know the index of the desired data.
- Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order. The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner.
- A sorting algorithm is said to be adaptive, if it takes advantage of already <sorted> elements in the list that is to be sorted.
- Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.
- Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.
- Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being $O(n \log n)$, it is one of the most respected algorithms.
- Shell sort is a highly efficient sorting algorithm and is based on insertion sort algorithm. This algorithm avoids large shifts as in case of insertion sort, if the smaller value is to the far right and has to be moved to the far left.

KNOWLEDGE CHECK

1. Which of the following is not a stable sorting algorithm?
 - a. Insertion sort
 - b. Selection sort
 - c. Bubble sort
 - d. Merge sort
2. Which of the following is a stable sorting algorithm?
 - a. Merge sort
 - b. Typical in-place quick sort
 - c. Heap sort
 - d. Selection sort
3. Which of the following is not an in-place sorting algorithm?
 - a. Selection sort
 - b. Heap sort
 - c. Quick sort
 - d. Merge sort
4. The complexity of sorting algorithm measures the as a function of the number n of items to be sorted.
 - a. average time
 - b. running time
 - c. average-case complexity
 - d. case-complexity
5. Which of the following is not a limitation of binary search algorithm?
 - a. must use a sorted array
 - b. requirement of sorted array is expensive when a lot of insertion and deletions are needed
 - c. there must be a mechanism to access middle element directly
 - d. binary search algorithm is not efficient when the data elements more than 1500.

REVIEW QUESTIONS

1. What is the linear search?
2. How to count inversions in a sorted array?



3. Write a function which implements linear search. It should take a list and an element as a parameter, and return the position of the element in the list. If the element is not in the list, the function should raise an exception. If the element is in the list multiple times, the function should return the first position.
4. Write a function which implements binary search. You may assume that the input list will be sorted.
5. When does the worst case of Quick Sort occur?

CHECK YOUR RESULTS

1. (b) 2. (a) 3. (d) 4. (b) 5. (d)

REFERENCES

1. <http://www.cs.carleton.edu/faculty/adalal/teaching/f04/117/notes/searchSort.pdf>
2. <http://pages.cs.wisc.edu/~bobh/367/SORTING.html>
3. <http://www.studytonight.com/data-structures/search-algorithms>
4. http://www.idc-online.com/technical_references/pdfs/information_technology/Searching_Algorithms.pdf
5. <http://www.cs.ccsu.edu/~jones/chap13.pdf>

ALGORITHM DESIGN METHODS

CHAPTER

5

LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Describe the backtracking algorithm
2. Explain the branch and bound

INTRODUCTION

An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing. A backtracking algorithm is a recursive method of building up feasible solutions to a combinatorial optimization problem one step at a time. A backtracking algorithm is an exhaustive search, that is, all feasible solutions are considered and it will thus always find the optimal solution. Pruning methods can be used to avoid considering some feasible solutions that are not optimal.

Branch and bound algorithm is a technique used in integer optimization problems – that is optimization problems for which the variables are integers. Often the variables are constrained to have values of either 0 or 1. One example of this is the knapsack problem in

which one is asked to select objects, each with a defined cost and a defined value, from a list, with the aim of reaching the highest total value while keeping total cost under a predefined limit. The variables in this case are binary - is each object selected, or not.

5.1 BACKTRACKING ALGORITHM

Backtracking is an approach to problem solving which is usually applied to constraint satisfaction problems like puzzles. In a backtracking solution, a search path is followed and the **algorithm** backtracks at a particular point (also known as decision point) in the path as soon as it realizes that this path would not lead to a valid solution and then it follows another path starting from a previous decision point. In this way, different paths are repeatedly explored to arrive at the final solution.



Algorithm is a self-contained step-by-step set of operations to be performed.

It is depth first traversal of path in graph where nodes are states of the solution and edge are between two states of solution only if one state can be reached from another state.

Only difference between brute force algorithm and backtracking algorithm is that backtracking discards partial solution as soon as possible without following them till end.

Consider the problem of solving a Sudoku. A naive algorithm will try to find every possible arrangement of numbers and checks if the problem can be solved with a particular arrangement but the backtracking algorithm provides a much better and efficient solution.

Backtracking is basically a refinement of the brute-force approach in which the solution to a problem is found using systematic search rather than trying out possible solutions blindly.

The backtracking algorithm.

Here is the algorithm (in pseudocode) for doing backtracking from a given node n:

```

boolean solve(Node n) {
    if n is a leaf node {
        if the leaf is a goal node, return true
        else return false
    } else {
        for each child c of n {
            if solve(c) succeeds, return true
        }
        return false
    }
}

```

Notice that the algorithm is expressed as a *boolean* function. This is essential to understanding the algorithm. If `solve(n)` is true, that means node `n` is part of a solution--that is, node `n` is one of the nodes on a path from the root to some goal node. We say that `n` is *solvable*. If `solve(n)` is false, then there is *no* path that includes `n` to any goal node.

How does this work?

- If any child of `n` is solvable, then `n` is solvable.
- If no child of `n` is solvable, then `n` is not solvable.

Hence, to decide whether any non-leaf node `n` is solvable (part of a path to a goal node), all you have to do is test whether any child of `n` is solvable. This is done recursively, on each child of `n`. In the above code, this is done by the lines

```

for each child c of n {
    if solve(c) succeeds, return true
}
return false

```

Eventually the recursion will “bottom” out at a leaf node. If the leaf node is a goal node, it is solvable; if the leaf node is not a goal node, it is not solvable. This is our base case. In the above code, this is done by the lines

```

if n is a leaf node {
    if the leaf is a goal node, return true
    else return false
}

```

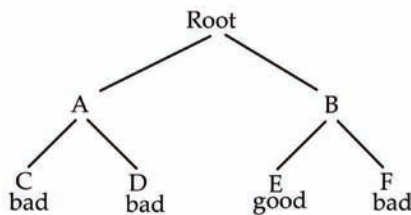
The backtracking algorithm is simple but important. You should understand it thoroughly. Another way of stating it is as follows:

- To search a tree:
 1. If the tree consists of a single leaf, test whether it is a goal node,
 2. Otherwise, search the subtrees until you find one containing a goal node, or until you have searched them all unsuccessfully.

Conceptually, you start at the root of a tree; the tree probably has some good leaves and some bad leaves, though it may be that the leaves are all good or all bad. You want to get to a good leaf. At each node, beginning with the root, you choose one of its children to move to, and you keep this up until you get to a leaf.

For example, suppose you get to a bad leaf. You can *backtrack* to continue the search for a good leaf by revoking your *most recent* choice, and trying out the next option in that set of options. If you run out of options, revoke the choice that got you here, and try another choice at that node. If you end up at the root with no options left, there are no good leaves to be found.

This needs an example.



- Starting at Root, your options are A and B. You choose A.
- At A, your options are C and D. You choose C.
- C is bad. Go back to A.
- At A, you have already tried C, and it failed. Try D.
- D is bad. Go back to A.
- At A, you have no options left to try. Go back to Root.
- At Root, you have already tried A. Try B.
- At B, your options are E and F. Try E.
- E is good. Congratulations!

In this example we drew a picture of a tree. The tree is an abstract model of the possible sequences of choices we could make. There is also a data structure called a tree, but usually we do not have a **data structure** to tell us what choices we have. (If we do have an actual tree data structure, backtracking on it is called *depth-first tree searching*.)

Non-recursive backtracking, using a stack

Backtracking is a rather typical recursive algorithm, and any recursive algorithm can be rewritten as a stack algorithm. In fact, that is how your recursive algorithms are translated into machine or assembly language.

```

boolean solve(Node n) {
    put node n on the stack;
    while the stack is not empty {
        if the node at the top of the stack is a leaf {
            if it is a goal node, return true
            else pop it off the stack
        }
        else {
            if the node at the top of the stack has untried children
                push the next untried child onto the stack
            else pop the node off the stack
        }
    }
    return false
}

```

Starting from the root, the only nodes that can be pushed onto the stack are the children of the node currently on the top of the stack, and these are only pushed on one child at a time; hence, the nodes on the stack at all times describe a valid path in the tree. Nodes are removed from the stack only when it is known that they have no goal nodes among their descendants. Therefore, if the root node gets removed (making the stack empty), there must have been no goal nodes at all, and no solution to the problem.

When the stack algorithm terminates successfully, the nodes on the stack form (in reverse order) a path from the root to a goal node.

```

boolean solve(Node n) {
    if n is a leaf node {
        if the leaf is a goal node {
            print n
            return true
        }
        else return false
    } else {
        for each child c of n {
            if solve(c) succeeds {
                print n
                return true
            }
        }
        return false
    }
}

```

Similarly, when the recursive algorithm finds a goal node, the path information is embodied (in reverse order) in the



Data structure is a particular way of organizing data in a computer so that it can be used efficiently.

sequence of recursive calls. Thus as the recursion unwinds, the path can be recovered one node at a time, by (for instance) printing the node at the current level, or storing it in an array.

Here is the recursive backtracking algorithm, modified slightly to print (in reverse order) the nodes along the successful path:

5.1.1 n-Queen's Problem

The N Queen Problem is one of the best problem used to teach backtracking and of course recursion. Backtracking is a general algorithm which finds all complete solutions to a problem by building over partial solutions. In this process, the problem might reach to a partial solution which may not result into a complete solution. Such partial solutions are effectively rejected by the backtracking algorithm.

In this problem, you are given an empty chess board of dimensions $N * N$. The task is to safely place N queens, one in each row such that they cause no harm to each other. The basic idea is mentioned below:

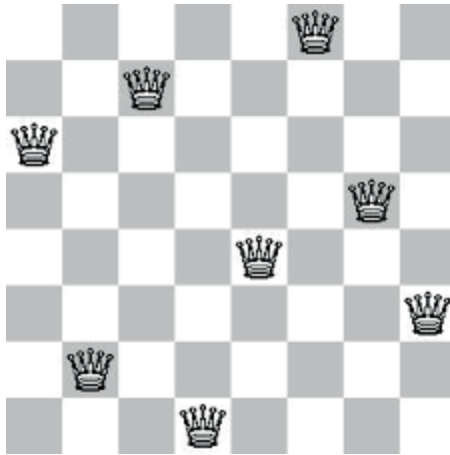
- Start from the first row
- Select a column and if it is empty, well within the limits of the board and in no immediate danger, then place the queen in the cell
- Proceed to the next row
- Two conditions might occur,
all the rows are safely filled
somewhere down the line, one of the queens cannot be placed in a row. Here we need to backtrack, because a decision was wrongly made in the past

Algorithm:

- Place the queens column wise, start from the left most column
- If all queens are placed.
- return true and print the solution matrix.
- Else
 1. Try all the rows in the current column.
 2. Check if queen can be placed here safely if yes mark the current cell in solution matrix as 1 and try to solve the rest of the problem recursively.
 3. If placing the queen in above step leads to the solution return true.
 4. If placing the queen in above step does not lead to the solution, BACKTRACK, mark the current cell in solution **matrix** as 0 and return false.

1. If all the rows are tried and nothing worked, return false and print NO SOLUTION.

Problem: Given a $n \times n$ grid, how would you place n queens so that no two are attacking. Two queens are said to be attacking if they are on same row, column or diagonal. The naive solution of this problem will require to find all the permutations i.e. finding all the $n!$ arrangements. A typical example of n queen problem is 8 queen problem which requires us to find possible placements of 8 queens on a 8×8 chess board. One of the arrangements of 8×8 problem is shown below:



Solution of this problem can be thought as follows:

- We start from first row and proceed to next row till last row.
- if next queen to be placed has same column value as already placed queen has then do not place the queen. This can be checked by traversing from $i=1$ to $i= k-1$ where k is the next row where queen is to be placed.
- if next queen to be placed lies in any of the diagonals of already placed queens then do not place the queen. This can be done by merely checking $(\text{abs}(i-k) == \text{abs}(j-l))$ where (i,j) is the place of any existing queen and (k,l) is the position of the next queen i.e. k th row and l th column.



Matrix is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns.

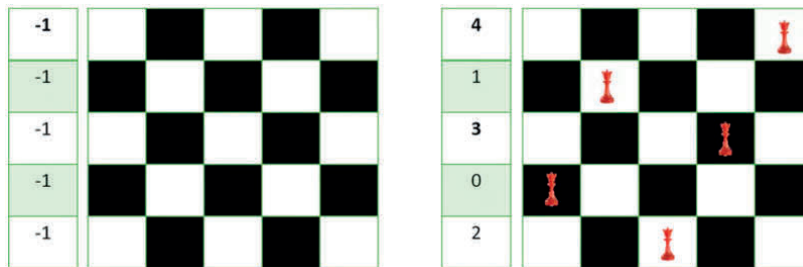
This process requires us to backtrack if next queen cannot be placed and recheck other options. We can use two functions for this. One for placing queen and other for checking whether queen can be placed on k th row and l th column or not.

Algorithm:

- for $i=1$ to n do step 2
- if queen can be placed
then
place the queen
if all queen placed then print the placements
else try next row by recursively calling the function

Formal Solution for N Queen Problem

The trick to the solution is to store the board correctly. You can either use a $N \times N$ array or simply a one dimensional array of length N . As you might have noticed that we only need to place N queens. We can very easily choose to store the position of the queen in each row. For example



The first image shows an empty board, hence the array has -1. In the second image, there are queens in each row, the zeroth row contains a queen at index (0,4) of the matrix. The first row contains a queen at index (1, 1) and so on. Hence, the values in the array are 4, 1, 3, 0 and 2.

The second point in the above algorithm talks about the criteria for accepting a valid cell. A valid cell must have the below properties:

- It must lie within the limits of the board
- It must be empty
- It must not be in immediate danger from the already placed queens (It must not lie in the same column and it must not lie on the major or minor diagonal)

Analysis for N Queen Problem

Space complexity

Space complexity is a measure of the amount of working storage an algorithm needs. That means how much memory, in the worst case, is needed at any point in the algorithm.

For this algorithm it is $O(N)$. The algorithm uses an auxiliary array of length N to store just N positions.

Time complexity

Time complexity of an algorithm signifies the total time required by the program to run to completion. The time complexity of algorithms is most commonly expressed using the big O notation.

Time Complexity is most commonly estimated by counting the number of elementary functions performed by the algorithm. And since the algorithm's performance may vary with different types of input data, hence for an algorithm we usually use the worst-case Time complexity of an algorithm because that is the maximum time taken for any input size.

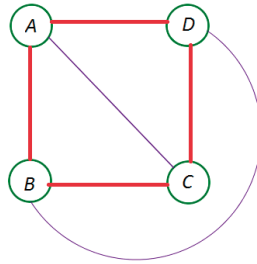
- The isSafe method takes $O(N)$ time as it iterates through our array every time.
- For each invocation of the placeQueen method, there is a loop which runs for $O(N)$ time.
- In each iteration of this loop, there is isSafe invocation which is $O(N)$ and a recursive call with a smaller argument.

If we add all this up and define the run time as $T(N)$. Then $T(N) = O(N^2) + N \cdot T(N-1)$. If you draw a recursion tree using this recurrence, the final term will be something like $n^3 + n!O(1)$. By the definition of Big O , this can be reduced to $O(n!)$ running time.

5.1.2 Hamiltonian Circuit problem

A Hamiltonian circuit, is a circuit (cycle) that visits each of the vertices once and only once and ends on the same vertex as it began.

For example, in this network the Hamiltonian circuit is marked in red.



Other Hamiltonian circuits here include, ABDCA, DBACD and there are many others. If a graph has a Hamiltonian circuit then it automatically has a Hamiltonian path.

Hamiltonian Path in an undirected graph is a path that visits each vertex exactly once. A Hamiltonian cycle (or Hamiltonian circuit) is a Hamiltonian Path such that there is an edge (in graph) from the last vertex to the first vertex of the Hamiltonian Path. Determine whether a given graph contains Hamiltonian Cycle or not. If it contains, then print the path. Following are the input and output of the required function.

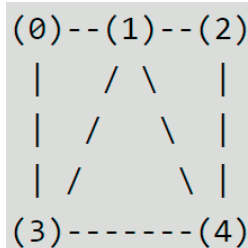
Input

A 2D array $\text{graph}[V][V]$ where V is the number of vertices in graph and $\text{graph}[V][V]$ is adjacency matrix representation of the graph. A value $\text{graph}[i][j]$ is 1 if there is a direct edge from i to j , otherwise $\text{graph}[i][j]$ is 0.

Output

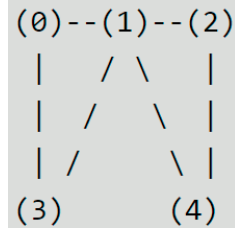
An array $\text{path}[V]$ that should contain the Hamiltonian Path. $\text{path}[i]$ should represent the i th vertex in the Hamiltonian Path. The code should also return false if there is no Hamiltonian Cycle in the graph.

For example, a Hamiltonian Cycle in the following graph is $\{0, 1, 2, 4, 3, 0\}$. There are more Hamiltonian Cycles in the graph like $\{0, 3, 4, 2, 1, 0\}$



And the following graph does not contain any Hamiltonian Cycle.





Naive Algorithm

Generate all possible configurations of vertices and print a configuration that satisfies the given constraints. There will be $n!$ (n factorial) configurations.

```
while there are untried configurations
{
    generate the next configuration
    if ( there are edges between two consecutive vertices of this
        configuration and there is an edge from the last vertex to
        the first ).
    {
        print this configuration;
        break;
    }
}
```

5.1.3 Subset-Sum problem

Subset sum problem is to find subset of elements that are selected from a given set whose sum adds up to a given number K . We are considering the set contains non-negative values. It is assumed that the input set is unique (no duplicates are presented).

if $X = \{8, 6, 7, 5, 3, 10, 9\}$ and $T = 15$, the answer is True, thanks to the subsets $\{8, 7\}$ or $\{7, 5, 3\}$ or $\{6, 9\}$ or $\{5, 10\}$. On the other hand, if $X = \{11, 6, 5, 1, 7, 13, 12\}$ and $T = 15$, the answer is False.



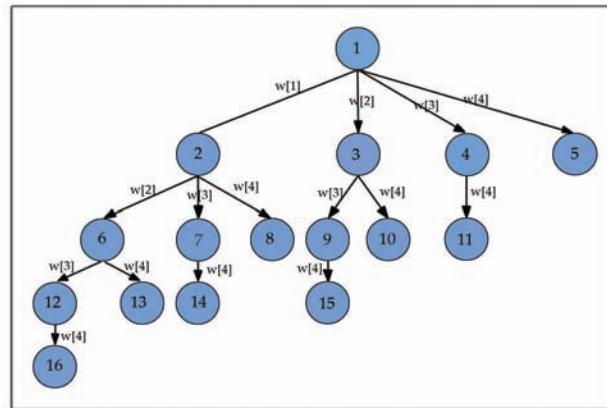
Exhaustive Search Algorithm for Subset Sum

One way to find subsets that sum to K is to consider all possible subsets. A power set contains all those subsets generated from a given set. The size of such a power set is 2^N .

Backtracking Algorithm for Subset Sum

Using exhaustive search we consider all subsets irrespective of whether they satisfy given constraints or not. Backtracking can be used to make a systematic consideration of the elements to be selected.

Assume given set of 4 elements, say $w[1] \dots w[4]$. Tree diagrams can be used to design backtracking algorithms. The following tree diagram depicts approach of generating variable sized tuple.



In the above tree, a node represents function call and a branch represents candidate element. The root node contains 4 children. In other words, root considers every element of the set as different branch. The next level sub-trees correspond to the subsets that includes the parent node. The branches at each level represent tuple element to be considered.

For example, if we are at level 1, $\text{tuple_vector}[1]$ can take any value of four branches generated. If we are at level 2 of left most node, $\text{tuple_vector}[2]$ can take any value of three branches generated, and so on...

For example the left most child of root generates all those subsets that include $w[1]$. Similarly the second child of root generates all those subsets that includes $w[2]$ and excludes $w[1]$.

As we go down along depth of tree we add elements so far, and if the added sum is satisfying explicit constraints, we will continue to generate child nodes further. Whenever the constraints are not met, we stop further generation of sub-trees of that node, and backtrack to previous node to explore the nodes not yet explored. In many scenarios, it saves considerable amount of processing time.

The tree should trigger a clue to implement the backtracking algorithm. It prints all those subsets whose sum add up to given number. We need to explore the nodes

along the breadth and depth of the tree. Generating nodes along breadth is controlled by loop and nodes along the depth are generated using recursion (post order traversal). Pseudo code given below,

```
if(subset is satisfying the constraint)
    print the subset
    exclude the current element and consider next element
else
    generate the nodes of present level along breadth of tree and
    recur for next levels
```

Recurrence relation for subset sum problem

```
isSubsetSum(arr, i, n, sum) = isSubsetSum(A, n-1, S-A[i])
                             if item A[i] can be included in subset
                             isSubsetSum(A, n-1, sum)
                             if item A[i] cannot be included in
subset
```

What would be the base case for **recursive function**? At any given point of time, if required sum is zero, subset is found; return true. Else, if all elements of array are considered and yet required is non zero, there is no subset with given sum; hence, return false.

```
If S == 0 return true
If n == 0 && S != 0 return false
```

Subset sum problem implementation

The subset sum problem is an important problem in complexity theory and cryptography. Complexity of solving subset sum problem using recursion is (2^n). It is because, in worst case, all subsets (of length 1 to N) of set are considered.



Recursive function is a function which either calls itself or is in a potential cycle of function calls.

```

1  #include<stdlib.h>
2  #include<stdio.h>
3
4  #define true 1
5  #define false 0
6  int isSubsetSum(int arr[], int n, int sum ){
7      printf("\nCalled with sum : %d, N = %d", sum, n );
8      if(!sum){
9          return true;
10     }
11     if(sum < 0) return false;
12
13     if(n <= 0 && sum > 0) return false;
14     return isSubsetSum(arr, n-1, sum-arr[n])
15           + isSubsetSum(arr, n-1, sum );
16 }
17
18 /* Driver program */
19 int main(){
20
21     int set[] = {1,3,5,4,6};
22     int n = sizeof(set)/sizeof(set[0]);
23
24     printf("\n Is there subset with given sum : %s",
25           isSubsetSum(set, n-1, 10 ) ? "Yes" : "No");
26     return 0;
27 }

```

5.2 BRANCH AND BOUND

It is a general algorithm for finding optimal solutions of various optimization problems, especially in discrete and combinatorial optimization. A branch and bound algorithm consists of a systematic enumeration of all candidate solutions, where large subsets of fruitless candidates are fathomed, by using upper and lower estimated bounds of the quantity being optimized.

Branch and bound is an algorithm design paradigm for discrete and combinatorial optimization problems, as well as mathematical optimization. A branch-and-bound algorithm consists of a systematic enumeration of candidate solutions by means of state space search: the set of candidate solutions is thought of as forming a rooted tree with the full set at the root. The algorithm explores branches of this tree, which represent subsets of the solution set. Before enumerating the candidate solutions of a branch, the branch is checked against upper and lower estimated bounds on the optimal solution, and is discarded if it cannot produce a better solution than the best one found so far by the algorithm.

The algorithm depends on the efficient estimation of the lower and upper bounds of a region/branch of the search space and approaches exhaustive enumeration as the size (n-dimensional volume) of the region tends to zero



5.2.1 Knapsack problem

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

0/1 Knapsack

Branch and bound is an algorithm design paradigm which is generally used for solving combinatorial optimization problems. These problems typically exponential in terms of time complexity and may require exploring all possible permutations in worst case. Branch and Bound solve these problems relatively quickly.

Let us consider below 0/1 Knapsack problem to understand Branch and Bound.

Given two integer arrays $val[0..n-1]$ and $wt[0..n-1]$ that represent values and weights associated with n items respectively. Find out the maximum value subset of $val[]$ such that sum of the weights of this subset is smaller than or equal to Knapsack capacity W .

Let us explore all approaches for this problem.

- A Greedy approach is to pick the items in decreasing order of value per unit weight. The Greedy approach works only for fractional knapsack problem and may not produce correct result for 0/1 knapsack.
- We can use Dynamic Programming (DP) for 0/1 Knapsack problem. In DP, we use a 2D table of size $n \times W$. The DP Solution does not work if item weights are not integers.
- Since DP solution does not always work, a solution is to use Brute Force. With n items, there are 2^n solutions to be generated, check each to see if they satisfy the constraint, save maximum solution that satisfies constraint. This solution can be expressed as tree.



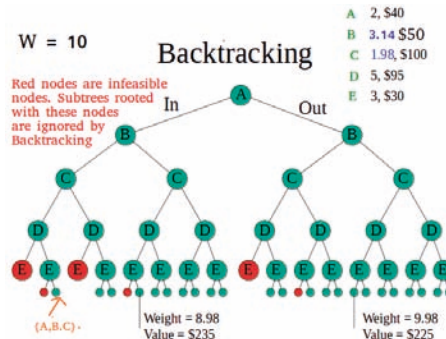
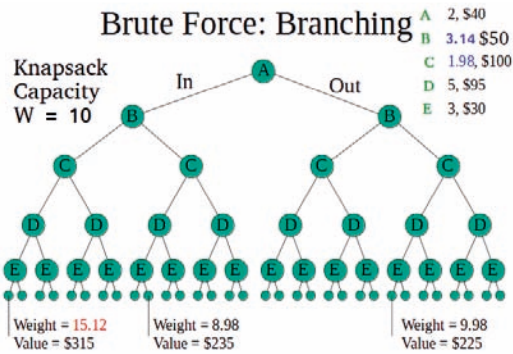
REMEMBER

The algorithm must be rigorously defined: specified in the way it applies in all possible circumstances that could arise. That is, any conditional steps must be systematically dealt with, case-by-case; the criteria for each case must be clear (and computable).

DID YOU KNOW?

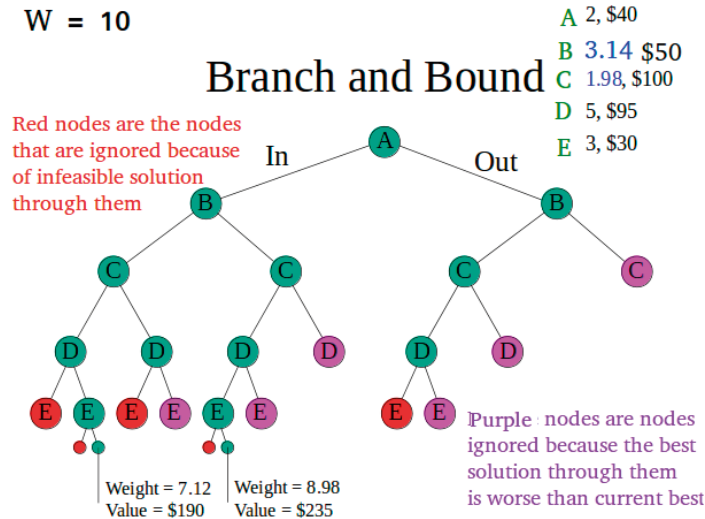
The Branch and Bound (BB or B&B) algorithm is first proposed by A. H. Land and A. G. Doig in 1960 for discrete programming.

- We can use backtracking to optimize the Brute Force solution. In the tree representation, we can do DFS of tree. If we reach a point where a solution no longer is feasible, there is no need to continue exploring. In the given example, backtracking would be much more effective if we had even more items or a smaller knapsack capacity.



The backtracking based solution works better than brute force by ignoring infeasible solutions. We can do better (than backtracking) if we know a bound on best possible solution subtree rooted with every node. If the best in subtree is worse than current best, we can simply ignore this node and its subtrees. So we compute bound (best solution) for every node and compare the bound with current best solution before exploring the node.

Example bounds used in below diagram are, A down can give \$315, B down can \$275, C down can \$225, D down can \$125 and E down can \$30. In the next article, we have discussed the process to get these bounds.



Branch and bound is very useful technique for searching a solution but in worst case, we need to fully calculate the entire tree. At best, we only need to fully calculate one path through the tree and prune the rest of it.

How to find bound for every node for 0/1 Knapsack?

The idea is to use the fact that the Greedy approach provides the best solution for Fractional Knapsack problem.

To check if a particular node can give us a better solution or not, we compute the optimal solution (through the node) using Greedy approach. If the solution computed by Greedy approach itself is more than the best so far, then we cannot get a better solution through the node.

Complete Algorithm:

- Sort all items in decreasing order of ratio of value per unit weight so that an upper bound can be computed using Greedy Approach.
- Initialize maximum profit, $\text{maxProfit} = 0$
- Create an empty queue, Q .
- Create a dummy node of decision tree and enqueue it to Q . Profit and weight of dummy node are 0.
- Do following while Q is not empty.
- Extract an item from Q . Let the extracted item be u .
- Compute profit of next level node. If the profit is more than maxProfit , then update maxProfit .

- Compute bound of next level node. If bound is more than maxProfit, then add next level node to Q.
- Consider the case when next level node is not considered as part of solution and add a node to queue with level as next, but weight and profit without considering next level nodes.

5.2.2 Assignment problem



REMEMBER

The reject procedure should be a Boolean valued function that returns true only if it is certain that no possible extension of c is a valid solution for P . If the procedure cannot reach a definite conclusion, it should return false.

The assignment problem is one of the fundamental combinatorial optimization problems in the branch of optimization or operations research in mathematics. It consists of finding a maximum weight matching (or minimum weight perfect matching) in a weighted bipartite graph.

Let there be N workers and N jobs. Any worker can be assigned to perform any job, incurring some cost that may vary depending on the work-job assignment. It is required to perform all jobs by assigning exactly one worker to each job and exactly one job to each agent in such a way that the total cost of the assignment is minimized.

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Worker A takes 8 units of time to finish job 4.

Finding Optimal Solution using Branch and Bound

The selection rule for the next node in BFS (**breadth-first search**) and DFS (depth-first search) is “blind”. That is the selection rule does not give any preference to a node that has a very good chance of getting the search to an answer node quickly. The search for an optimal solution can often be speeded by using an “intelligent” ranking function, also called an approximate cost function to avoid searching in sub-trees that do not contain an optimal solution. It is similar to



BFS-like search but with one major optimization. Instead of following FIFO (first in first out) order, we choose a live node with least cost. We may not get optimal solution by following node with least promising cost, but it will provide very good chance of getting the search to an answer node quickly.

There are two approaches to calculate the cost function:

- For each worker, we choose job with minimum cost from list of unassigned jobs (take minimum entry from each row).
- For each job, we choose a worker with lowest cost for that job from list of unassigned workers (take minimum entry from each column).

The first approach is followed.

Let's take below example and try to calculate promising cost when Job 2 is assigned to worker A.

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Since Job 2 is assigned to worker A (marked in green), cost becomes 2 and Job 2 and worker A becomes unavailable (marked in red).

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4



Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures.

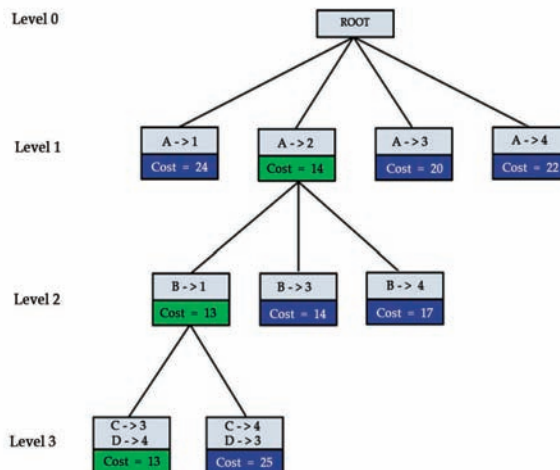
Now we assign job 3 to worker B as it has minimum cost from list of unassigned jobs. Cost becomes $2 + 3 = 5$ and Job 3 and worker B also becomes unavailable.

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Finally, job 1 gets assigned to worker C as it has minimum cost among unassigned jobs and job 4 gets assigned to worker C as it is only Job left. Total cost becomes $2 + 3 + 5 + 4 = 14$.

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Below diagram shows complete search space diagram showing optimal solution path in green.



Complete Algorithm:

```

/* findMinCost uses Least() and Add() to maintain the
list of live nodes

Least() finds a live node with least cost, deletes
it from the list and returns it

Add(x) calculates cost of x and adds it to the list
of live nodes

Implements list of live nodes as a min heap */

// Search Space Tree Node
node
{
    int job_number;
    int worker_number;
    node parent;
    int cost;
}

// Input: Cost Matrix of Job Assignment problem
// Output: Optimal cost and Assignment of Jobs
algorithm findMinCost (costMatrix mat[][])
{
    // Initialize list of live nodes(min-Heap)
    // with root of search tree i.e. a Dummy node
    while (true)
    {
        // Find a live node with least estimated cost
        E = Least();

        // The found node is deleted from the list
        // of live nodes
        if (E is a leaf node)
        {
            printSolution();
            return;
        }

        for each child x of E
        {
            Add(x); // Add x to list of live nodes;
            x->parent = E; // Pointer for path to root
        }
    }
}

```

5.2.3 Traveling salesman problem

Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible tour that visits every city exactly once and returns to the starting point.

For example, consider the graph shown in figure on right side. A TSP tour in the graph is 0-1-3-2-0. The cost of the tour is $10+25+30+15$ which is 80.

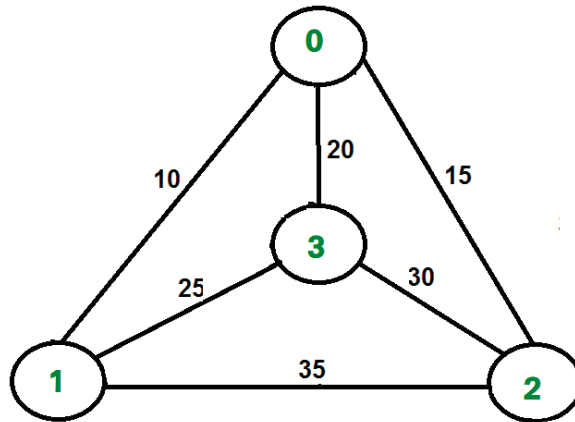


REMEMBER

The programmer must translate the algorithm into a language that the simulator/computer/computer can effectively execute.

We have discussed following solutions

- Naive and Dynamic Programming
- Approximate solution using MST



In Branch and Bound method, for current node in tree, we compute a bound on best possible solution that we can get if we down this node. If the bound on best possible solution itself is worse than current best (best computed so far), then we ignore the subtree rooted with the node.

Note that the cost through a node includes two costs.

- 1) Cost of reaching the node from the root (When we reach a node, we have this cost computed)
- 2) Cost of reaching an answer from current node to a leaf (We compute a bound on this cost to decide whether to ignore sub tree with this node or not).

In cases of a maximization problem, an upper bound tells us the maximum possible solution if we follow the given node. For example in 0/1 knapsack we used

Greedy approach to find an upper bound.

In cases of a minimization problem, a lower bound tells us the minimum possible solution if we follow the given node. *For example*, in Job Assignment Problem, we get a lower bound by assigning least cost job to a worker.

In branch and bound, the challenging part is figuring out a way to compute a bound on best possible solution.

Cost of any tour can be written as below.

```
Cost of a tour T = (1/2) * Σ (Sum of cost of two edges
                             adjacent to u and in the
                             tour T)
                             where u ∈ V
For every vertex u, if we consider two edges through it in T,
and sum their costs. The overall sum for all vertices would
be twice of cost of tour T (We have considered every edge
twice.)

(Sum of two tour edges adjacent to u) >= (sum of minimum weight
two edges adjacent to
u)

Cost of any tour >= 1/2) * Σ (Sum of cost of two minimum
weight edges adjacent to u)
                             where u ∈ V
```

For example, consider the above shown graph. Below are minimum cost two edges adjacent to every node.

Node	Least cost edges	Total cost
0	(0, 1), (0, 2)	25
1	(0, 1), (1, 3)	35
2	(0, 2), (2, 3)	45
3	(0, 3), (1, 3)	45

```
Thus a lower bound on the cost of any tour =
1/2(25 + 35 + 45 + 45)
= 75
```

Refer [this](#) for one more example.

Now we have an idea about computation of lower bound. Let us see how to how to apply it state space search tree. We start enumerating all possible nodes (preferably in lexicographical order)

1. The Root Node: Without loss of generality, we assume we start at vertex "0" for which the lower bound has been calculated above.

Dealing with Level 2: The next level enumerates all possible vertices we can go to (keeping in mind that in any path a vertex has to occur only once) which are, 1, 2, 3... n (Note that the graph is complete). Consider we are calculating for vertex 1, Since we moved from 0 to 1, our tour has now included the edge 0-1. This allows us to make necessary changes in the lower bound of the root.

```
Lower Bound for vertex 1 =
    Old lower bound - ((minimum edge cost of 0 +
                        minimum edge cost of 1) / 2)
                    + (edge cost 0-1)
```

How does it work?

To include edge 0-1, we add the edge cost of 0-1, and subtract an edge weight such that the lower bound remains as tight as possible which would be the sum of the minimum edges of 0 and 1 divided by 2. Clearly, the edge subtracted cannot be smaller than this.

Dealing with other levels: As we move on to the next level, we again enumerate all possible vertices. For the above case going further after 1, we check out for 2, 3, 4,...n.

Consider lower bound for 2 as we moved from 1 to 1, we include the edge 1-2 to the tour and alter the new lower bound for this node.

```
Lower bound(2) =
    Old lower bound - ((second minimum edge cost of 1 +
                        minimum edge cost of 2)/2)
                    + edge cost 1-2)
```

Note: The only change in the formula is that this time we have included second minimum edge cost for 1, because the minimum edge cost has already been subtracted in previous level.

SUMMARY

- An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing
- A backtracking algorithm is a recursive method of building up feasible solutions to a combinatorial optimization problem one step at a time.
- Branch and bound is a technique used in integer optimization problems – that is optimization problems for which the variables are integers.
- The N Queen Problem is one of the best problem used to teach backtracking and of course recursion. Backtracking is a general algorithm which finds all complete solutions to a problem by building over partial solutions.
- A Hamiltonian circuit, is a circuit (cycle) that visits each of the vertices once and only once and ends on the same vertex as it began.
- Subset sum problem is to find subset of elements that are selected from a given set whose sum adds up to a given number K.
- A branch-and-bound algorithm consists of a systematic enumeration of candidate solutions by means of state space search: the set of candidate solutions is thought of as forming a rooted tree with the full set at the root.
- The assignment problem is one of the fundamental combinatorial optimization problems in the branch of optimization or operations research in mathematics.

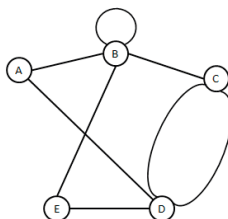
KNOWLEDGE CHECK

1. The quick sort algorithm exploit _____ design technique
 - a. Greedy
 - b. Dynamic programming
 - c. Divide and Conquer
 - d. Backtracking
2. Breadth first traversal is a method to traverse
 - a. All successors of a visited node before any successors of any of those successors
 - b. A single path of the graph as far it can go
 - c. Graph using shortest path
 - d. None of these

3. Which of the following is not a backtracking algorithm?
 - a. Knight tour problem
 - b. N queen problem
 - c. Tower of hanoi
 - d. M coloring problem
4. Which of the following algorithm design technique is used in the quick sort algorithm?
 - a. Dynamic programming
 - b. Backtracking
 - c. Divide and conquer
 - d. Greedy method
5. An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing
 - a. True
 - b. False

REVIEW QUESTIONS

1. What do you understand by backtracking algorithm?
2. Explain the n-Queen's problem.
3. Describe the Hamiltonian circuit problem.
4. Write the short note on "Branch and bound".
5. Is it possible to draw a Hamiltonian circuit on the following diagram?



CHECK YOUR RESULT

1. (c) 2. (a) 3. (c) 4. (c) 5. (a)

REFERENCES

1. Csegeek, http://www.csegeek.com/csegeek/view/tutorials/algorithms/backtrack/backtrack_intro.php
2. <https://www.cis.upenn.edu/~matuszek/cit594-2012/Pages/backtracking.html>
3. programminggeek, <http://www.programminggeek.in/2013/08/solution-of-n-queen-problem-and-8queen-problem-using-backtracking-in-c.html#.WIBMKdyGPcs>
4. <http://techieme.in/solving-the-n-queen-problem/>
5. geeksforgeeks, <http://www.geeksforgeeks.org/backtracking-set-7-hamiltonian-cycle/>
6. <http://algorithmsandme.in/2014/03/subset-sum-problem/>
7. geeksforgeeks, <http://www.geeksforgeeks.org/branch-and-bound-set-1-introduction-with-01-knapsack/>
8. geeksforgeeks, <http://www.geeksforgeeks.org/branch-bound-set-4-job-assignment-problem/>
9. geeksforgeeks, <http://www.geeksforgeeks.org/branch-and-bound-set-2-implementation-of-01-knapsack/>

ALGORITHMS TECHNIQUES AND ANALYSIS

CHAPTER

6

LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Learn about greedy algorithms
2. Define dynamic programming
3. Discuss amortized analysis

INTRODUCTION

Algorithms are the foundation and framework of all computer programs. They are the sequence of precise instructions given to a computer for achieving a desired output. Algorithms are to computer programs what the mind is to the human body. Many algorithmic design patterns or paradigms exist such as greedy algorithms, divide-and-conquer algorithms, dynamic programming algorithms and backtracking algorithms. Each class of algorithms has its own unique design structure.

A technique is a broad word for any kind of general approach that may be used to make progress on problems. Being general, it is usually not specific enough to the details of any given problem to be a one-stop solution. It may also be the case that coming up with an algorithm to a problem requires combining multiple techniques.

For example, someone might write about using a “dynamic programming technique” to solve a problem. The reader can then infer that the problem will be approached by:

- breaking it down into some number of sub-problems whose answers can be expressed in terms of the answers to other sub-problems
- caching the answers to already-solved sub-problems to avoid solving them again

Algorithm analysis is a field of computer science that is dedicated to understanding the complexity of algorithms. Algorithms are generally defined as processes that perform a series of operations to an end. Algorithms can be expressed in many ways, in flow charts, a natural language, and computer programming languages. Algorithms are used in mathematics, computing and linguistics, but a most common use is in computers to do calculations or process data. Algorithm analysis deals with algorithms written in computer programming languages, which are based on mathematical formalism.

More than one algorithm might work to perform the same operation, but some algorithms use more memory and take longer to perform than others. Also, how do we know how well algorithms work in general, given differences between computers and data inputs? This is where algorithm analysis comes in.

One way to test an algorithm is to run a computer program and see how well it works. The problem with this approach is that it only tells us how well the algorithm works with a particular computer and set of inputs. The purpose of algorithm analysis is to test and then draw conclusions about how well a particular algorithm works in general. This would be very difficult and time consuming to do on individual computers, so researchers devise models of computer functioning to test algorithms.

In general, algorithm analysis is most concerned with finding out how much time a program takes to run, and how much memory storage space it needs to execute the program. In particular, computer scientists use algorithm analysis to determine how the data imputed into a program affects its total running time, how much memory space the computer needs for program data, how much space the program’s code takes in the computer, whether an algorithm produces correct calculations, how complex a program is, and how well it deals with unexpected results.

6.1 GREEDY ALGORITHMS

Greedy algorithms are simple and straightforward. They are shortsighted in their approach in the sense that they take decisions on the basis of information at hand without worrying about the effect these decisions may have in the future. They are easy to invent, easy to implement and most of the time quite efficient. Many problems cannot be solved correctly by greedy approach. Greedy algorithms are used to solve optimization problems



In greedy algorithm of optimizing solution, the best solution is chosen at any moment. A greedy algorithm is very easy to apply to complex problems. It decides which step will provide the most accurate solution in the next step.

This algorithm is called greedy because when the optimal solution to the smaller instance is provided, the algorithm does not consider the total program as a whole. Once a solution is considered, the greedy algorithm never considers the same solution again.

A greedy algorithm works recursively creating a group of objects from the smallest possible component parts. Recursion is a procedure to solve a problem in which the solution to a specific problem is dependent on the solution of the smaller instance of that problem.

Greedy Approach

Greedy Algorithm works by making the decision that seems most promising at any moment; it never reconsiders this decision, whatever situation may arise later.

As an example consider the problem of “Making Change”.

Coins available are:

- Dollars (100 cents)
- Quarters (25 cents)
- Dimes (10 cents)
- Nickels (5 cents)
- Pennies (1 cent)

Problem Make a change of a given amount using the smallest possible number of coins.

Informal Algorithm

- Start with nothing.
- at every stage without passing the given amount.
 - add the largest to the coins already chosen.

Formal Algorithm

Make change for n units using the least possible number of coins.

MAKE-CHANGE (n)

$C \leftarrow \{100, 25, 10, 5, 1\}$ // constant.

```

Sol ← {};           // set that will hold the solution set.

Sum ← 0 sum of item in solution set

WHILE sum not = n

    x = largest item in set C such that sum + x ≤ n

    IF no such item THEN

        RETURN  "No Solution"

    S ← S {value of x}

    sum ← sum + x

RETURN S

```

Example: Make a change for 2.89 (289 cents) here $n = 2.89$ and the solution contains 2 dollars, 3 quarters, 1 dime and 4 pennies. The algorithm is greedy because at every stage it chooses the largest coin without worrying about the consequences. Moreover, it never changes its mind in the sense that once a coin has been included in the solution set, it remains there.

Characteristics and Features of Problems solved by Greedy Algorithms

To construct the solution in an optimal way. Algorithm maintains two sets. One contains chosen items and the other contains rejected items.

The greedy algorithm consists of four (4) function.

- A function that checks whether chosen set of items provide a solution.
- A function that checks the feasibility of a set.
- The selection function tells which of the candidates is the most promising.
- An objective function, which does not appear explicitly, gives the value of a solution.

Structure Greedy Algorithm

- Initially the set of chosen items is empty i.e., solution set.
- At each step
 - item will be added in a solution set by using selection function.
 - IF the set would no longer be feasible

- reject items under consideration (and is never consider again).
- ELSE IF set is still feasible THEN
 - add the current item.

6.1.1 Elements of the Greedy Strategy

A **greedy algorithm** obtains an optimal solution to a problem by making a sequence of choices. At each decision point, the algorithm makes choice that seems best at the moment. This heuristic strategy does not always produce an optimal solution, but as we saw in the activity-selection problem, sometimes it does. The general properties of greedy methods.

The process to develop a greedy algorithm was a bit more involved than is typical. We went through the following steps:

- Determine the optimal substructure of the problem.
- Develop a recursive solution. (For the activity-selection problem, we formulated recurrence but we bypassed developing a recursive algorithm based on this recurrence.)
- Show that if we make the greedy choice, then only one sub problem remains.
- Prove that it is always safe to make the greedy choice. (Steps 3 and 4 can occur in either order.)
- Develop a recursive algorithm that implements the greedy strategy.
- Convert the recursive algorithm to an iterative algorithm.

In going through these steps, we saw in great detail the dynamic-programming un-derpinnings of a greedy algorithm. For example, in the activity-selection problem, we first defined the sub problems S_{ij} , where both i and j varied. We then found that if we always made the greedy choice, we could restrict the sub problems to be of the form S_k .

Alternatively, we could have fashioned our optimal substructure with a greedy choice in mind, so that the choice leaves just one sub problem to solve. In the activity-selection problem, we could have started by dropping the second



Greedy algorithm is an algorithmic paradigm that follows the problem solving heuristic of making the locally optimal choice at each stage with the hope of finding a global optimum.

subscript and defining sub problems of the form S_k . Then, we could have proven that a greedy choice (the first activity a_m to finish in S_k), combined with an optimal solution to the remaining set S_m of compatible activities, yields an optimal solution to S_k . More generally, we design greedy algorithms according to the following sequence of steps:

- Cast the optimization problem as one in which we make a choice and are left with one sub problem to solve.
- Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the greedy choice is always safe.
- Demonstrate optimal substructure by showing that, having made the greedy choice, what remains is a sub problem with the property that if we combine an optimal solution to the sub problem with the greedy choice we have made, we arrive at an optimal solution to the original problem.

We shall use this more direct process. Nevertheless, beneath every greedy algorithm, there is almost always a more cumbersome dynamic-programming solution.

How can we tell whether a greedy algorithm will solve a particular optimization problem? No way works all the time, but the greedy-choice property and optimal substructure are the two key ingredients. If we can demonstrate that the problem has these properties, then we are well on the way to developing a greedy algorithm for it.



A greedy strategy for the traveling salesman problem (which is of a high computational complexity) is the following heuristic: “At each stage visit an unvisited city nearest to the current city”. This heuristic need not find a best solution, but terminates in a reasonable number of steps; finding an optimal solution typically requires unreasonably many steps. In mathematical optimization, greedy algorithms solve combinatorial problems having the properties of matroids.

Greedy-Choice Property

The first key ingredient is the greedy-choice property: we can assemble a globally optimal solution by making locally optimal (greedy) choices. In other words, when we are considering which choice to make, we make the choice that looks best

in the current problem, without considering results from sub problems. Here is where greedy algorithms differ from dynamic programming. In dynamic programming, we make a choice at each step, but the choice usually depends on the solutions to sub problems. Consequently, we typically solve dynamic-programming problems in a bottom-up manner, progressing from smaller sub problems to larger sub problems. (Alternatively, we can solve them top down, but memorizing. Of course, even though the code works top down, we still must solve the sub problems before making a choice.) In a greedy algorithm, we make whatever choice seems best at the moment and then solve the sub problem that remains. The choice made by a greedy algorithm may depend on choices so far, but it cannot depend on any future choices or on the solutions to sub problems. Thus, unlike dynamic programming, which solves the sub problems before making the first choice, a greedy algorithm makes its first choice before solving any sub problems. A dynamic- programming algorithm proceeds bottom up, whereas a greedy strategy usually progresses in a top-down fashion, making one greedy choice after another, reducing each given problem instance to a smaller one.

Of course, we must prove that a greedy choice at each step yields a globally optimal solution. The proof examines a globally optimal solution to some sub problem. It then shows how to modify the solution to substitute the greedy choice for some other choice, resulting in one similar, but smaller, sub problem. We can usually make the greedy choice more efficiently than when we have to consider a wider set of choices.

In the activity-selection problem, assuming that we had already sorted the activities in monotonically increasing order of finish times, we needed to examine each activity just once. By preprocessing the input or by using an appropriate data structure (often a priority queue), we often can make greedy choices quickly, thus yielding an efficient algorithm.



Optimal Substructure

A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to sub

problems. This property is a key ingredient of assessing the applicability of dynamic programming as well as greedy algorithms. As an example of optimal substructure, recall how we demonstrated that if an optimal solution to sub problem S_{ij} includes an activity a_k then it must also contain optimal solutions to the sub problems S_{ik} and S_{kj} .

Given this optimal substructure, we argued that if we knew which activity to use as a_k , we could construct an optimal solution to S_{ij} by selecting a_k along with all activities in optimal solutions to the sub problems S_{ik} and S_{kj} . Based on this observation of optimal substructure that described the value of an optimal solution.

We usually use a more direct approach regarding optimal substructure when applying it to greedy algorithms. As mentioned above, we have the luxury of assuming that we arrived at a sub problem by having made the greedy choice in the original problem. All we really need to do is argue that an optimal solution to the sub problem, combined with the greedy choice already made, yields an optimal solution to the original problem. This scheme implicitly uses induction on the sub problems to prove that making the greedy choice at every step produces an optimal solution.

Greedy Versus Dynamic Programming

Because both the greedy and dynamic-programming strategies exploit optimal substructure, you might be tempted to generate a dynamic-programming solution to a problem when a greedy solution suffices or, conversely, you might mistakenly think that a greedy solution works when in fact a dynamic-programming solution is required. To illustrate the subtleties between the two techniques, let us investigate two variants of a classical optimization problem.

The 0-1 knapsack problem is the following. A thief robbing a store finds n items. The i th item is worth v_i dollars and weighs w_i pounds, where v_i and w_i are integers. The thief wants to take as valuable a load as possible, but he can carry at most W pounds in his knapsack, for some integer W . Which items should he take? (We call this the 0-1 knapsack problem because for each item, the thief must either take it or leave it behind; he cannot take a fractional amount of an item or take an item more than once.)

In the fractional knapsack problem, the setup is the same, but the thief can take fractions of items, rather than having to make a binary (0-1) choice for each item. You can think of an item in the 0-1 knapsack problem as being like a gold ingot and an item in the fractional knapsack problem as more like gold dust.

Both knapsack problems exhibit the optimal-substructure property. For the 0-1 problem, consider the most valuable load that weighs at most W pounds. If we remove item j from this load, the remaining load must be the most valuable load weighing at most $W - w_j$ that the thief can take from the $n-1$ original items excluding j . For the comparable fractional problem, consider that if we remove a weight w of one item j from the optimal load, the remaining load must be the most valuable load weighing



at most $W - w$ that the thief can take from the $n - 1$ original items plus $w_j - w$ pounds of item j .

Although the problems are similar, we can solve the fractional knapsack problem by a greedy strategy, but we cannot solve the 0-1 problem by such a strategy. To solve the fractional problem, we first compute the value per pound v_i / w_i for each item. Obeying a greedy strategy, the thief begins by taking as much as possible of the item with the greatest value per pound. If the supply of that item is exhausted and he can still carry more, he takes as much as possible of the item with the next greatest value per pound, and so forth, until he reaches his weight limit W . Thus, by sorting the items by value per pound, the greedy algorithm runs in $O(n \lg n)$ time.

To see that this greedy strategy does not work for the 0-1 knapsack problem, consider the problem instance illustrated in Figure 6.1(a). This example has 3 items and a knapsack that can hold 50 pounds. Item 1 weighs 10 pounds and is worth 60 dollars. Item 2 weighs 20 pounds and is worth 100 dollars. Item 3 weighs 30 pounds and is worth 120 dollars. Thus, the value per pound of item 1 is 6 dollars per pound, which is greater than the value per pound of either item 2 (5 dollars per pound) or item 3 (4 dollars per pound). The greedy strategy, therefore, would take item 1 first. As you can see from the case analysis in Figure 6.2(b), however, the optimal solution takes items 2 and 3, leaving item 1 behind. The two possible solutions that take item 1 are both suboptimal. For the comparable fractional problem, however, the greedy strategy, which takes item 1 first, does yield an optimal solution, as shown in Figure 6.2(c). Taking item 1 doesn't work in the 0-1 problem because the thief is unable to fill his knapsack to capacity, and the empty space lowers the effective value per pound of his load. In the 0-1 problem, when we consider whether to include an item in the knapsack, we must compare the solution to the sub problem that includes the item with the solution to the sub problem that excludes the item before we can make the choice.

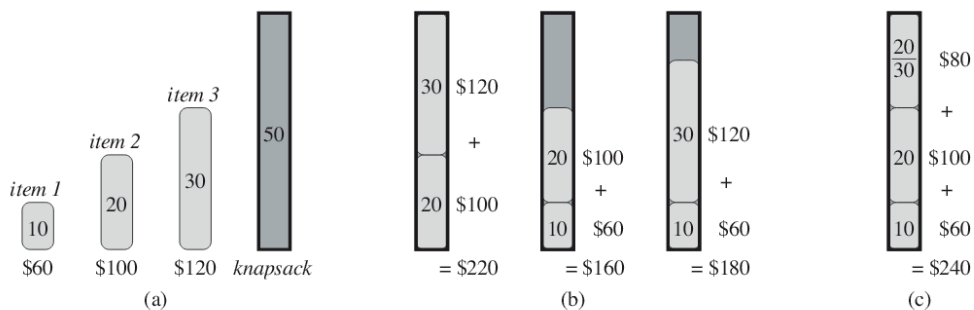


Figure 6.1: Greedy strategy.

6.1.2 Huffman Codes

Huffman codes compress data very effectively: savings of 20% to 90% are typical, depending on the characteristics of the data being compressed. We consider the data to be a sequence of characters. Huffman's greedy algorithm uses a table giving how often each character occurs (i.e., its frequency) to build up an optimal way of representing each character as a binary string.

Suppose we have a 100,000-character data file that we wish to store compactly. We observe that the characters in the file occur with the frequencies given by Figure 6.2. That is, only 6 different characters appear, and the character *a* occurs 45,000 times.

We have many options for how to represent such a file of information. Here, we consider the problem of designing a binary character code (or code for short) in which each character is represented by a unique binary string, which we call a code word. If we use a fixed-length code, we need 3 bits to represent 6 characters: *a* = 000, *b* = 001, ..., *f* = 101. This method requires 300,000 bits to code the entire file. Can we do better?

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Figure 6.2: A character-coding problem. A data file of 100,000 characters contains only the characters *a* – *f*, with the frequencies indicated. If we assign each character a 3-bit code word, we can encode the file in 300,000 bits. Using the variable-length code shown, we can encode the file in only 224,000 bits.

A variable-length code can do considerably better than a fixed-length code, by giving frequent characters short code words and infrequent characters long code- words. Figure 6.2 shows such a code; here the 1-bit string 0 represents *a*, and the 4-bit string 1100 represents *f*. This code requires

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1,000 = 224,000 \text{ bits}$$

to represent the file, a savings of approximately 25%. In fact, this is an optimal character code for this file

Prefix Codes

We consider here only codes in which no code word is also a prefix of some other code word. Such codes are called prefix codes. ³ Although we won't prove it here, a prefix code can always achieve the optimal data compression among any character code, and so we suffer no loss of generality by restricting our attention to prefix codes. Encoding is always simple for any binary character code; we just concatenate the code words representing each character of the file. For example, with the variable- length



prefix code of Figure 6.2, we code the 3-character file abc as 0 101 100 D 0101100 ,where“ ” denotes concatenation. Prefix codes are desirable because they simplify decoding. Since no code word is a prefix of any other, the code word that begins an encoded file is unambiguous. We can simply identify the initial code word, translate it back to the original character, and repeat the decoding process on the remainder of the encoded file. In our example, the string 001011101 parses uniquely as 0 0 101 1101 , which decodes to aabe.

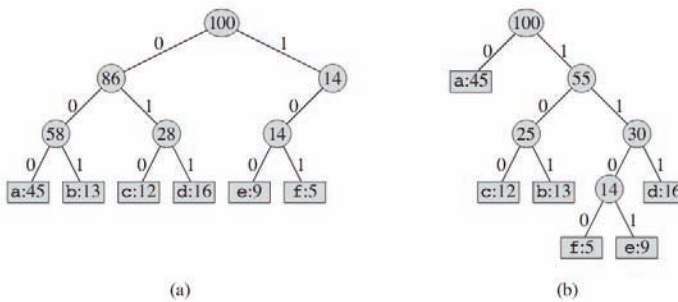


Figure 6.3: Prefix codes.

The decoding process needs a convenient representation for the prefix code so that we can easily pick off the initial code word. A binary tree whose leaves are the given characters provides one such representation. We interpret the binary code word for a character as the simple path from the root to that character, where 0 means “go to the left child” and 1 means “go to the right child.” Figure 6.3 shows the trees for the two codes of our example. Note that these are not binary search trees, since the leaves need not appear in sorted order and internal nodes do not contain character keys.

An optimal code for a file is always represented by a full binary tree in which every non-leaf node has two children. The fixed-length code in our example is not optimal since its tree, shown in Figure 6.3(a), is not a full binary tree: it contains code words beginning 10..., but none beginning 11.... Since we can now restrict our attention to full binary trees, we can say that if C is the alphabet from which the characters are drawn and all character frequencies are positive, then the tree for an optimal prefix code has exactly $|C|$ leaves, one for each letter of the alphabet, and exactly $|C| - 1$ internal nodes.



Prefix code is a type of code system distinguished by its possession of the “prefix property”, which requires that there is no whole code word in the system that is a prefix (initial segment) of any other code word in the system.

Given a tree T corresponding to a prefix code, we can easily compute the number of bits required to encode a file. For each character c in the alphabet C , let the attribute $c.\text{freq}$ denote the frequency of c in the file and let $d_T(c)$ denote the depth of c 's leaf in the tree.

Note that $d_T(c)$ is also the length of the code word for character c . The number of bits required to encode a file is thus

$$B(T) = \sum_{c \in C} c.\text{freq} \cdot d_T(c),$$

which we define as the cost of the tree T .

Constructing a Huffman Code

Huffman invented a greedy algorithm that constructs an optimal prefix code called a

Huffman code. Its proof of correctness relies on the greedy-choice property and optimal substructure. Rather than demonstrating that these properties hold and then developing pseudocode, we present the pseudocode first. Doing so will help clarify how the algorithm makes greedy choices.

In the pseudocode that follows, we assume that C is a set of n characters and that each character $c \in C$ is an object with an attribute $c.\text{freq}$ giving its frequency. The algorithm builds the tree T corresponding to the optimal code in a bottom-up manner. It begins with a set of $|C|$ leaves and performs a sequence of $|C| - 1$ “merging” operations to create the final tree. The algorithm uses a min-priority queue Q , keyed on the freq attribute, to identify the two least-frequent objects to merge together. When we merge two objects, the result is a new object whose frequency is the sum of the frequencies of the two objects that were merged.

```

HUFFMAN( $C$ )
1   $n = |C|$ 
2   $Q = C$ 
3  for  $i = 1$  to  $n - 1$ 
4      allocate a new node  $z$ 
5       $z.\text{left} = x = \text{EXTRACT-MIN}(Q)$ 
6       $z.\text{right} = y = \text{EXTRACT-MIN}(Q)$ 
7       $z.\text{freq} = x.\text{freq} + y.\text{freq}$ 
8       $\text{INSERT}(Q, z)$ 
9  return  $\text{EXTRACT-MIN}(Q)$     // return the root of the tree

```

For our example, Huffman's algorithm proceeds as shown in Figure 6.4. Since the alphabet contains 6 letters, the initial queue size is $n = 6$, and 5 merge steps build the tree. The final tree represents the optimal prefix code. The code word for a letter is the sequence of edge labels on the simple path from the root to the letter. Line 2

initializes the min-priority queue Q with the characters in C . The for loop in lines 3–8 repeatedly extracts the two nodes x and y of lowest frequency names x and y in the proof of correctness. Therefore, we find it convenient to leave them in.

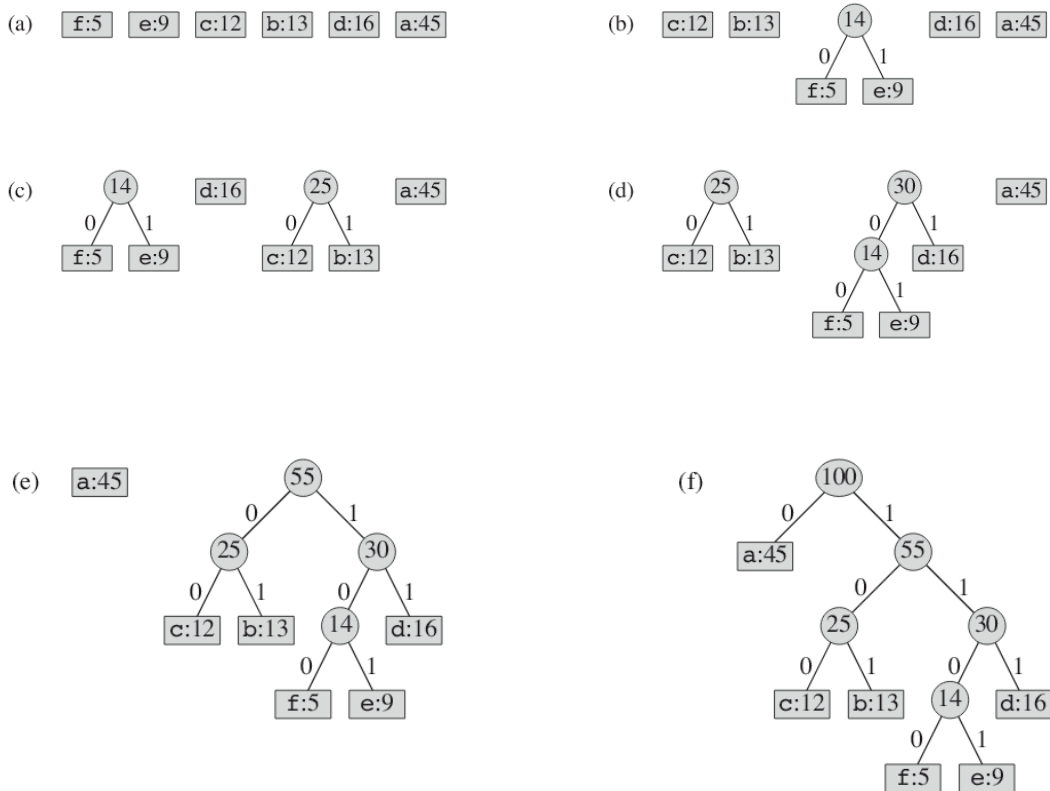


Figure 6.4: steps of Huffman's algorithm.

To analyze the running time of Huffman's algorithm, we assume that Q is implemented as a binary min-heap.

For a set C of n characters, we can initialize Q in line 2 in $O(n)$ time using the BUILD-MIN-HEAP procedure. The for loop in lines 3–8 executes exactly $n-1$ times, and since each heap operation requires time $O(\lg n)$, the loop contributes $O(n \lg n)$ to the running time. Thus, the total running time of HUFFMAN on a set of n characters is $O(n \lg n)$. We can reduce the running time to $O(n \lg \lg n)$ by replacing the binary min-heap with a van Emde Boas tree.

Correctness of Huffman's Algorithm

To prove that the greedy algorithm HUFFMAN is correct, we show that the problem of

determining an optimal prefix code exhibits the greedy-choice and optimal-substructure properties.

Step 1: Show that this problem satisfies the greedy choice property, that is, if a greedy choice is made by Huffman's algorithm, an optimal solution remains possible.

Step 2: Show that this problem has an optimal substructure property, that is, an optimal solution to Huffman's algorithm contains optimal solution to sub problems.

Step 3: Conclude correctness of Huffman's algorithm using step 1 and step 2.

Lemma Greedy Choice Property

Let C be an alphabet in which each character c has frequency $f[c]$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the code words for x and y have the same length and differ only in the last bit.

Proof Idea

Take the tree T representing optimal prefix code and transform T into a tree T' representing another optimal prefix code such that the x characters x and y appear as sibling leaves of maximum depth in T' . If we can do this, then their code words will have same length and differ only in the last bit.

Proof

Let characters b and c are sibling leaves of maximum depth in tree T . Without loss of generality assume that $f[b] \geq f[c]$ and $f[x] \leq f[y]$. Since $f[x]$ and $f[y]$ are lowest leaf frequencies in order and $f[b]$ and $f[c]$ are arbitrary frequencies in order. We have $f[x] \leq f[b]$ and $f[y] \leq f[c]$. As shown in the above figure, exchange the positions of leaves to get first T' and then T'' . By formula, $B(t) = \sum_{c \text{ in } C} f(c)d_T(c)$, the difference in cost between T and T' is

$$\begin{aligned} B(T) - B(T') &= f[x]d_T(x) + f(b)d_T(b) - [f[x]d_{T'}(x) + f[b]d_{T'}(b)] \\ &= (f[b] - f[x]) (d_T(b) - d_T(x)) \\ &= (\text{non-negative})(\text{non-negative}) \\ &\geq 0 \end{aligned}$$

Two Important Points

The reason $f[b] - f[x]$ is non-negative because x is a minimum frequency leaf in tree T and the reason $d_T(b) - d_T(x)$ is non-negative that b is a leaf of maximum depth in T .

Similarly, exchanging y and c does not increase the cost which implies that $B(T) - B(T') \geq 0$. This fact in turn implies that $B(T) \leq B(T')$, and since T is optimal by supposition, which implies $B(T') = B(T)$. Therefore, T'' is optimal in which x and y are sibling leaves of maximum depth, from which greedy choice property follows.

Lemma - Optimal Substructure Property

Let T be a full binary tree representing an optimal prefix code over an alphabet C , where frequency $f[c]$ is define for each character c belongs to set C . Consider any two characters x and y that appear as sibling leaves in the tree T and let z be their parent. Then, considering character z with frequency $f[z] = f[x] + f[y]$, tree $T' = T - \{x, y\}$ represents an optimal code for the alphabet $C' = C - \{x, y\} \cup \{z\}$.

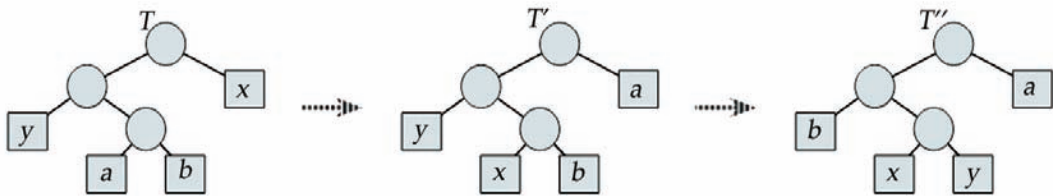


Figure 6.5: Optimal tree.

Proof

We show that the cost $B(T)$ of tree T can be expressed in terms of the cost $B(T')$. By considering the component costs in equation, $B(T) = \sum f(c)d_T(c)$, we show that the cost $B(T)$ of tree T can be expressed in terms of the cost $B(T')$ of the tree T' . For each c belongs to $C - \{x, y\}$, we have $d_T(c) = d_{T'}(c)$

$$\begin{aligned} \sum_{c \in C} f[c]d_T(c) &= \sum_{c \in C - \{x, y\}} f[c]d_{T'}(c) \\ &= f[x](d_{T'}(z) + 1) + f[y](d_{T'}(z) + 1) \\ &= (f[x] + f[y])d_{T'}(z) + f[x] + f[y] \end{aligned}$$

And $B(T) = B(T') + f(x) + f(y)$

If T' is non-optimal prefix code for C' , then there exists a T'' whose leaves are the characters belongs to C' such that $B(T'') < B(T')$. Now, if x and y are added to T'' as a children of z , then we get a prefix code for alphabet C with cost $B(T'') + f[x] + f[y] < B(T)$, Contradicting the optimality of T . Which implies that, tree T' must be optimal for the alphabet C .

6.1.3 Matroids and Greedy Methods

Suppose we are given a set $E = \{e_1, e_2, \dots, e_n\}$ of n objects, each with associated positive integer weights $w(e_1), w(e_2), \dots, w(e_n)$. Let I be a subset of the power set (the set of all subsets) of E with the following two properties:

- For every I_1 in I , if I_2 is a subset of I_1 then I_2 is in I .
- For every pair I_1, I_2 in I such that $|I_2| = |I_1| + 1$, there is an element e in $I_2 - I_1$ which makes $I_1 \cup \{e\}$ in I .

Then the pair (E, I) is called a matroid. Members of I are referred to as independent sets. Independent sets containing the most elements are called maximum cardinality independent sets. The problem of finding a maximum cardinality independent set (I_{max}) which minimizes (or maximizes) the sum of $w(e)$ over all e in I_{max} may be solved as follows (the Greedy Method):

Set $T = \text{emptyset}$

Order all objects in E by increasing (decreasing) weight

Repeat the following until E is empty

Let e be the lowest (highest) weight element in E

Remove e from E

If $T \cup \{e\}$ is in I then add e to T

Output T

Next we present two examples: the Maximum Spanning Network Problem and the Integer Deadline Scheduling Problem with unit processing times.

First consider the problem of finding a Minimum Spanning Tree (network) in a weighted graph G . Let E be the set of edges in the graph G . Let I be the set of all forests of G (that is, all graphs using edges of G which have no cycles). Then (E, I) is a matroid.

To see this consider the two properties which define a matroid. Property (1) is satisfied since removing any edge from a forest results in another forest. Property (2) is satisfied by the following argument: Start with any I_1, I_2 in I with $|I_2| = |I_1| + 1$. Consider three possibilities.

- There is an edge e of I_2 which has at most one vertex in common with a vertex that is an endpoint of an edge of I_1 . In this case $I_1 \cup \{e\}$ is a forest.
- There is an edge e of I_2 which has one vertex common to a vertex in one connected component of I_1 and the other vertex common to a vertex in another connected component of I_1 . In this case, since both connected components are trees and there are no other edges between the components, $I_1 \cup \{e\}$ is a forest.
- For each edge $\langle v, w \rangle$ of I_2 , v and w are in the same connected component of I_1 . This case is impossible since it would require that at least one connected



component of I_1 contain a number of edges of I_2 greater than the number of edges of I_1 within that component. This implies that there is a cycle among edges of I_2 in that component and this implies I_2 is not a forest (contrary to hypothesis). Therefore, we can always choose an e from $I_2 - I_1$ such that $I_1 \cup \{e\}$ is in I .

Now, the maximum cardinality forest of G is a spanning tree of G . Also, the weight of a spanning network in the Minimum Spanning Network problem is the sum of the edges in the network (a tree). That is, the weight of a maximum cardinality independent set is the sum of weights of objects in that set. Hence, we may use the greedy method above to solve the Minimum Spanning Network problem (which is what we have been doing all along). The only part that has to be filled in is how to check whether adding a particular edge to a forest T keeps a forest. But, this is easy, as we saw.

The Integer Deadline Scheduling problem is stated as follows:

Integer Deadline Scheduling

Instance: A set $A = \{a_1, a_2, \dots, a_n\}$ of n jobs, each requiring unit time to complete, and associated with each job a is an integer deadline $d(a)$ and an integer profit $p(a)$.

Produce: A schedule $[a_{o1}, a_{o2}, \dots, a_{on}]$ for a single processor such that the sum of the profits of all jobs which are processed before their deadline is maximum over all schedules.

The solution here is trickier than for the **Minimum Spanning Tree** problem since, first, we can order jobs based on their deadlines or their profits so we must decide on which to base the ordering and, second, we need a schedule (that is, a list) as output and not a subset. As you might suspect, ordering on profit is correct (since we are trying to maximize profit). We use T to hold a subset of jobs which *can* be scheduled in such a way that all jobs in the subset can be completed before their deadlines. The final solution is obtained by finding that schedule for T (easily done by ordering the jobs in T by increasing deadline) and tacking onto that schedule all jobs not in T in any order.



REMEMBER

Besides serving a packaging role, the shell and the shell membranes also have a biological function; namely to regulate evaporation and air circulation, but also to prevent penetration of microorganisms.

6.1.4 A Task-Scheduling Problem as a Matroid

An interesting problem that can be solved using matroids is the problem of optimally scheduling unit-time tasks on a single processor, where each task has a deadline and a penalty that must be paid if the deadline is missed. The problem looks complicated, but it can be solved in a surprisingly simple manner using a greedy algorithm.

A *unit-time task* is a job, such as a program to be run on a computer, that requires exactly one unit of time to complete. Given a finite set S of unit-time tasks, a *schedule* for S is a permutation of S specifying the order in which these tasks are to be performed. The first task in the schedule begins at time 0 and finishes at time 1, the second task begins at time 1 and finishes at time 2, and so on.



KEYWORD

Minimum spanning tree (MST) is a subset of the edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycles and with the minimum possible total edge weight.

The problem of scheduling unit-time tasks with deadlines and penalties for a single *processor* has the following inputs:

- a set $S = \{1, 2, \dots, n\}$ of n unit-time tasks;
- a set of n integer *deadlines* $d_1, d_2, \dots, d_n$, such that each d_i satisfies $1 \leq d_i \leq n$ and task i is supposed to finish by time d_i ; and
- a set of n nonnegative weights or *penalties* $w_1, w_2, \dots, w_n$, such that a penalty w_i is incurred if task i is not finished by time d_i and no penalty is incurred if a task finishes by its deadline.

We are asked to find a schedule for S that minimizes the total penalty incurred for missed deadlines.

Consider a given schedule. We say that a task is *late* in this schedule if it finishes after its deadline. Otherwise, the task is *early* in the schedule. An arbitrary schedule can always be put into *early-first form*, in which the early tasks precede the late tasks. To see this, note that if some early task x follows some late task y , then we can switch the positions of x and y without affecting x being early or y being late.

We similarly claim that an arbitrary schedule can always be put into *canonical form*, in which the early tasks precede the late tasks and the early tasks are scheduled in order of nondecreasing deadlines. To do so, we put the schedule into early-first form. Then, as long as there are two early tasks i and j finishing at respective times k and $k + 1$ in the schedule

such that $d_j < d_i$, we swap the positions of i and j . Since task j is early before the swap, $k + 1 \leq d_j$. Therefore, $k + 1 < d_i$, and so task i is still early after the swap. Task j is moved earlier in the schedule, so it also still early after the swap.

The search for an optimal schedule thus reduces to finding a set A of tasks that are to be early in the optimal schedule. Once A is determined, we can create the actual schedule by listing the elements of A in order of non-decreasing deadline, then listing the late tasks (i.e., $S - A$) in any order, producing a canonical ordering of the optimal schedule.

We say that a set A of tasks is *independent* if there exists a schedule for these tasks such that no tasks are late. Clearly, the set of early tasks for a schedule forms an independent set of tasks. Let I denote the set of all independent sets of tasks.

Consider the problem of determining whether a given set A of tasks is independent. For $t = 1, 2, \dots, n$, let $N_t(A)$ denote the number of tasks in A whose deadline is t or earlier.

Lemma

For any set of tasks A , the following statements are equivalent.

- The set A is independent.
- For $t = 1, 2, \dots, n$, we have $N_t(A) \leq t$.
- If the tasks in A are scheduled in order of non-decreasing deadlines, then no task is late.

Proof

Clearly, if $N_t(A) > t$ for some t , then there is no way to make a schedule with no late tasks for set A , because there are more than t tasks to finish before time t . Therefore, (1) implies (2). If (2) holds, then (3) must follow: there is no way to “get stuck” when scheduling the tasks in order of non-decreasing deadlines, since (2) implies that the i th largest deadline is at most i . Finally, (3) trivially implies (1).

Using property 2 of Lemma, we can easily compute whether or not a given set of tasks is independent. The problem of minimizing the sum of the penalties of the late tasks is the same as the problem of maximizing the sum of the penalties of the early tasks. The following theorem thus ensures that we can use the greedy algorithm to find an independent set A of tasks with the maximum total penalty.

Theorem

If S is a set of unit-time tasks with deadlines, and I is the set of all independent sets of tasks, then the corresponding system (S, I) is a matroid.

Proof



REMEMBER

The search for an optimal schedule thus reduces to finding a set A of tasks that are to be early in the optimal schedule. Once A is determined, we can create the actual schedule by listing the elements of A in order of monotonically increasing deadline, then listing the late tasks (i.e., $S - A$) in any order, producing a canonical ordering of the optimal schedule.

Every subset of an independent set of tasks is certainly independent. To prove the exchange property, suppose that B and A are independent sets of tasks and that $|B| > |A|$. Let k be the largest t such that $N_t(B) \leq N_t(A)$. Since $N_n(B) = |B|$ and $N_n(A) = |A|$, but $|B| > |A|$, we must have that $k < n$ and that $N_j(B) > N_j(A)$ for all j in the range $k + 1 \leq j \leq n$. Therefore, B contains more tasks with deadline $k + 1$ than A does. Let x be a task in $B - A$ with deadline $k + 1$. Let $A' = A \cup \{x\}$.

We now show that A' must be independent by using property 2 of Lemma 17.11. For $1 \leq t \leq k$, we have $N_t(A') = N_t(A) \leq t$, since A is independent. For $k < t \leq n$, we have $N_t(A') \leq N_t(B) \leq t$, since B is independent. Therefore, A' is independent, completing our proof that $(S, \mathcal{I})$ is a matroid.

By Theorem, we can use a greedy algorithm to find a maximum-weight independent set of tasks A . We can then create an optimal schedule having the tasks in A as its early tasks. This method is an efficient algorithm for scheduling unit-time tasks with deadlines and penalties for a single processor. The running time is $O(n^2)$ using GREEDY, since each of the $O(n)$ independence checks made by that algorithm takes time $O(n)$. A faster implementation is given in Problem

Task							
1	2	3	4	5	6	7	

d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

An example of a problem of scheduling unit-time tasks with deadlines and penalties for a single processor. In this example, the greedy algorithm selects tasks 1, 2, 3, and 4, then rejects tasks 5 and 6, and finally accepts task 7. The final optimal schedule is

(2, 4, 1, 3, 7, 5, 6) ,

which has a total penalty incurred of $w_5 + w_6 = 50$.



6.2 DYNAMIC PROGRAMMING

Dynamic programming is a very powerful technique to solve a particular class of problems. It demands very elegant formulation of the approach and simple thinking and the coding part is very easy. The idea is very simple, If you have solved a problem with the given input, then save the result for future reference, so as to avoid solving the same problem again. Shortly 'Remember your Past' :). If the given problem can be broken up in to smaller sub-problems and these smaller sub problems are in turn divided in to still-smaller ones, and in this process, if you observe some over-lapping sub problems, then its a big hint for DP. Also, the optimal solutions to the sub problems contribute to the optimal solution of the given problem.

Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems. But unlike, divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.

Dynamic programming is used where we have problems, which can be divided into similar sub-problems, so that their results can be re-used. Mostly, these algorithms are used for optimization. Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems. The solutions of sub-problems are combined in order to achieve the best solution.

So we can say that:

- The problem should be able to be divided into smaller overlapping sub-problem.
- An optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- Dynamic algorithms use memorization.

Comparison

In contrast to greedy algorithms, where local optimization is addressed, dynamic algorithms are motivated for an overall optimization of the problem.

In contrast to divide and conquer algorithms, where solutions are combined to achieve an overall solution, dynamic algorithms use the output of a smaller sub-problem and then try to optimize a bigger sub-problem. Dynamic algorithms use memorization to remember the output of already solved sub-problems.

Example

The following computer problems can be solved using dynamic programming approach –

- Fibonacci number series

- Knapsack problem
- Tower of Hanoi
- All pair shortest path by Floyd-Warshall
- Shortest path by Dijkstra
- Project scheduling

Dynamic programming can be used in both top-down and bottom-up manner. And of course, most of the times, referring to the previous solution output is cheaper than re-computing in terms of CPU cycles.

6.2.1 Overlapping Sub Problems

A problem is said to have overlapping sub problems if it can be broken down into sub problems which are reused multiple times. This is closely related to recursion. To see the difference consider the factorial function, defined as follows (in Python):

```
def factorial(n):
    if n == 0: return 1
    return n*factorial(n-1)
```

Thus the problem of calculating factorial(n) depends on calculating the sub problem factorial(n-1). This problem does not exhibit *overlapping* sub problems since factorial is called exactly once for each positive integer less than n.

Fibonacci Numbers

The problem of calculating the nth **Fibonacci number** does, however, exhibit overlapping sub problems. The naïve recursive implementation would be

```
def fib(n):
    if n == 0: return 0
    if n == 1: return 1
    return fib(n-1) + fib(n-2)
```

The problem of calculating fib(n) thus depends on both fib(n-1) and fib(n-2). To see how these subproblems overlap look at how many times fib is called and with what arguments when we try to calculate fib(5):



KEYWORD

Fibonacci number is the numbers in the following integer sequence, called the Fibonacci sequence, and characterized by the fact that every number after the first two is the sum of the two preceding ones.

```

fib(5)
fib(4) + fib(3)
fib(3) + fib(2) + fib(2) + fib(1)
fib(2) + fib(1) + fib(1) + fib(0) + fib(1) +
fib(0) + fib(1)
fib(1) + fib(0) + fib(1) + fib(1) + fib(0) +
fib(1) + fib(0) + fib(1)

```

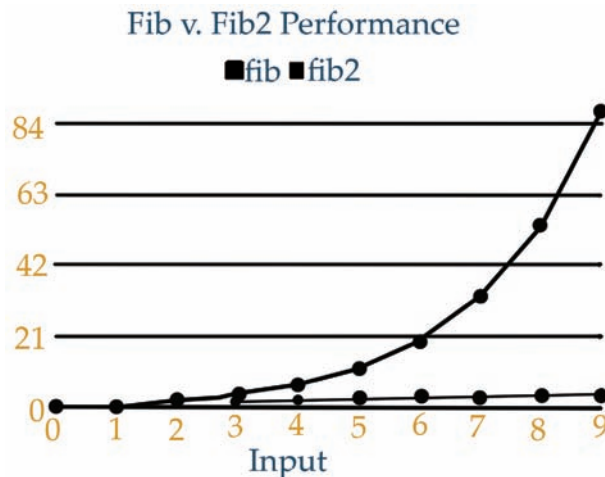
At the k^{th} stage we only need to know the values of $\text{fib}(k-1)$ and $\text{fib}(k-2)$, but we wind up calling each multiple times. Starting from the bottom and going up we can calculate the numbers we need for the next step, removing the massive redundancy.

```

def fib2(n):
    n2, n1 = 0, 1
    for i in range(n-2):
        n2, n1 = n1, n1 + n2
    return n2+n1

```

In Big-O notation the `fib` function takes $O(c^n)$ time, i.e., exponential in n , while the `fib2` function takes $O(n)$ time. If this is all too abstract take a look at this graph comparing the runtime (in microseconds) of `fib` and `fib2` versus the input parameter.



The above problem is pretty easy and for most Programmers is one of the first examples of the performance issues surrounding recursion versus iteration. In fact, I've seen many instances where the Fibonacci example leads people to believe that recursion is inherently slow. This is not true, but in cases where we can define a problem with overlapping sub problems recursively using the above technique will always reduce the execution time. Now, for the second characteristic of dynamic Programming: optimal substructure.

DID YOU KNOW?

The term dynamic programming was originally used in the 1940s by Richard Bellman to describe the process of solving problems where one needs to find the best decisions one after another.

6.2.2 Optimal Substructure

A problem is said to have optimal substructure if the globally optimal solution can be constructed from locally optimal solutions to sub problems. The general form of problems in which optimal substructure plays a roll goes something like this. Let's say we have a collection of objects called A . For each object o in A we have a "cost," $c(o)$. Now find the subset of A with the maximum (or minimum) cost, perhaps subject to certain constraints.

The brute-force method would be to generate every subset of A , calculate the cost, and then find the maximum (or minimum) among those values. But if A has n elements in it we are looking at a search space of size 2^n if there are no constraints on A . Oftentimes n is huge making a brute-force method computationally infeasible. Let's take a look at an example.

Maximum Subarray Sum

Let's say we're given an array of integers. What (contiguous) subarray has the largest sum? For example, if our array is $[1, 2, -5, 4, 7, -2]$ then the subarray with the largest sum is $[4, 7]$ with a sum of 11. One might think at first that this problem reduces to finding the subarray with all positive entries, if one exists, that maximizes the sum. But consider the array $[1, 5, -3, 4, -2, 1]$. The subarray with the largest sum is $[1, 5, -3, 4]$ with a sum of 7. First, the brute-force solution. Because of the constraints on the problem, namely that the subsets under consideration are contiguous, we only have to check $O(n^2)$ subarrays (why?). Here it is, in Python:

```
def msum(a):
    return max([(sum(a[j:i]), (j,i)) for i in
range(1,len(a)+1) for j in range(i)])
```

This returns both the sum and the offsets of the subarray. Let's see if we can't find an optimal substructure to exploit.

We are given an input array a . I'm going to use Python notation so that $a[0:k]$ is the subarray starting at 0 and including every element up to and including $k-1$. Let's say we know the subarray of $a[0:i]$ with the largest sum (and that sum). Using just this information can we find the subarray of $a[0:i+1]$ with

the largest sum?

Let $a[j:k+1]$ be the optimal subarray, t the sum of $a[j:i]$, and s the optimal sum. If $t+a[i]$ is greater than s then set $a[j:i+1]$ as the optimal array and set $s = t$. If $t + a[i]$ is negative, however, the contiguity constraint means that we cannot include $a[j:i+1]$ in our subarray since any such subarray will have a smaller sum than a subarray without it. So, if $t+a[i]$ is negative set $t = 0$ and set the left-hand bound of the optimal subarray to $i+1$.

To visualize consider the array $[1,2,-5,4,7,-2]$.

Set $s = -\text{infinity}$, $t = 0$, $j = 0$, bounds = $(0,0)$

$(1\ 2\ -5\ 4\ 7\ -2)$

$(1)\ | \ 2\ -5\ 4\ 7\ -2$ (set $t=1$. Since $t > s$, set $s=1$ and bounds = $(0,1)$)

$(1\ 2)\ | \ -5\ 4\ 7\ -2$ (set $t=3$. Since $t > s$, set $s=3$, and bounds = $(0,2)$)

$1\ 2\ -5\ (\ | \ 4\ 7\ -2$ (set $t=-2$. Since $t < 0$, set $t=0$ and $j = 3$)

$1\ 2\ -5\ (4)\ | \ 7\ -2$ (set $t=4$. Since $t > s$, set $s=4$ and bounds = $(3,4)$)

$1\ 2\ -5\ (4\ 7)\ | \ -2$ (set $t=11$. Since $t > s$, set $s=11$ and bounds = $(3,5)$)

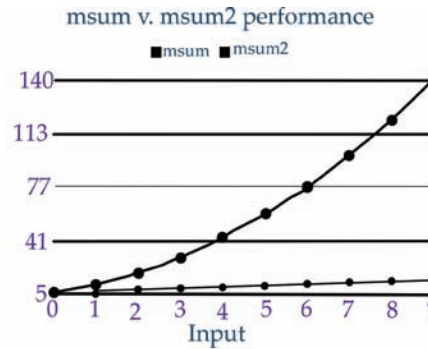
$1\ 2\ -5\ (4\ 7)\ -2\ |$ (set $t=9$. Nothing happens since $t < s$)

This requires only one pass through the array and at each step we're only keeping track of three variables: the current sum from the left-hand edge of the bounds to the current point (t), the maximal sum (s), and the bounds of the current optimal subarray (bounds). In Python:

```
def msum2(a):
    bounds, s, t, j = (0,0), -float('infinity'), 0
    for i in range(len(a)):
        t = t + a[i]
        if t > s: bounds, s = (j, i+1), t
        if t < 0: t, j = 0, i+1
    return (s, bounds)
```

In this problem the “globally optimal” solution corresponds to a subarray with a globally maximal sum, but at each each step we only make a decision relative to what we have already seen.

That is, at each step we know the best solution *thus far*, but might change our decision later based on our previous information and the current information. This is the sense in the problem has optimal substructure. Because we can make decisions locally we only need to traverse the list once, reducing the run-time of the solution to $O(n)$ from $O(n^2)$. Again, a graph:



6.2.3 Steps in Dynamic Programming

- Initialization
- Matrix fill (scoring)
- Traceback (alignment)

Initialization Step

The first step in the global alignment dynamic Programming approach is to create a matrix with $M + 1$ columns and $N + 1$ rows where M and N correspond to the size of the sequences to be aligned.

Since this example assumes there is no gap opening or gap extension penalty, the first row and first column of the matrix can be initially filled with 0.

		G	A	A	T	T	C	A	G	T	T	A
	0	0	0	0	0	0	0	0	0	0	0	0
G	0											
G	0											
A	0											
T	0											
C	0											
G	0											
A	0											

Matrix Fill Step

One possible (inefficient) solution of the matrix fill step finds the maximum global alignment score by starting in the upper left hand corner in the matrix and finding the maximal score $M_{i,j}$ for each position in the matrix. In order to find $M_{i,j}$ for any i,j it is minimal to know the score for the matrix positions to the left, above and diagonal to i, j . In terms of matrix positions, it is necessary to know $M_{i-1,j}$, $M_{i,j-1}$ and $M_{i-1,j-1}$.

For each position, $M_{i,j}$ is defined to be the maximum score at position i,j ; i.e.

$$M_{i,j} = \text{MAXIMUM} \begin{bmatrix} M_{i-1,j-1} + S_{ij} \text{ (match/mismatch in the diagonal),} \\ M_{i,j-1} + w \text{ (gap in sequence \#1),} \\ M_{i-1,j} + w \text{ (gap in sequence \#2)} \end{bmatrix}$$

Note that in the example, $M_{i-1,j-1}$ will be red, $M_{i,j-1}$ will be green and $M_{i-1,j}$ will be blue.

Using this information, the score at position 1,1 in the matrix can be calculated. Since the first residue in both sequences is a G, $S_{1,1} = 1$, and by the assumptions stated at the beginning, $w = 0$. Thus, $M_{1,1} = \text{MAX}[M_{0,0} + 1, M_{1,0} + 0, M_{0,1} + 0] = \text{MAX}[1, 0, 0] = 1$.

A value of 1 is then placed in position 1,1 of the scoring matrix.

		G	A	A	T	T	C	A	G	T	T	A
G	0	0	0	0	0	0	0	0	0	0	0	0
G	0	1										
A	0											
T	0											
C	0											
G	0											
A	0											

Since the gap penalty (w) is 0, the rest of row 1 and column 1 can be filled in with the value 1. Take the example of row 1. At column 2, the value is the max of 0 (for a mismatch), 0 (for a vertical gap) or 1 (horizontal gap).

The rest of row 1 can be filled out similarly until we get to column 8. At this point, there is a G in both sequences (light blue).

Thus, the value for the cell at row 1 column 8 is the maximum of 1 (for a match), 0 (for a vertical gap) or 1 (horizontal gap). The value will again be 1. The rest of row 1 and column 1 can be filled with 1 using the above reasoning.

		G	A	A	T	T	C	A	G	T	T	A
		0	0	0	0	0	0	0	0	0	0	0
G	0	1	1	1	1	1	1	1	1	1	1	1
G	0	1	1									
A	0	1	2									
T	0	1	2									
C	0	1	2									
G	0	1	2									
A	0	1	2									

Now let's look at column 2. The location at row 2 will be assigned the value of the maximum of 1(mismatch), 1(horizontal gap) or 1 (vertical gap). So its value is 1. At the position column 2 row 3, there is an A in both sequences. Thus, its value will be the maximum of 2(match), 1 (horizontal gap), 1 (vertical gap) so its value is 2.

Moving along to position column 2 row 4, its value will be the maximum of 1 (mismatch), 1 (horizontal gap), 2 (vertical gap) so its value is 2.

Note that for all of the remaining positions except the last one in column 2, the choices for the value will be the exact same as in row 4 since there are no matches.

The final row will contain the value 2 since it is the maximum of 2 (match), 1 (horizontal gap) and 2(vertical gap).

		G	A	A	T	T	T	C	G	T	T	A
		0	0	0	0	0	0	0	0	0	0	0
G	0	1	1	1	1	1	1	1	1	1	1	1
G	0	1										
A	0	1										
T	0	1										
C	0	1										
G	0	1										
A	0	1										

Using the same techniques as described for column 2, we can fill in column 3.

		G	A	A	T	T	C	A	G	T	T	A
		0	0	0	0	0	0	0	0	0	0	0
G	0	1	1	1	1	1	1	1	1	1	1	1
G	0	1	1	1								
A	0	1	2	2								
T	0	1	2	2								
C	0	1	2	2								
G	0	1	2	2								
A	0	1	2	3								

After filling in all of the values the score matrix is as follows:]

		G	A	A	T	T	C	A	G	T	T	A
		0	0	0	0	0	0	0	0	0	0	0
G		0	1	1	1	1	1	1	1	1	1	1
G		0	1	1	1	1	1	1	2	2	2	2
A		0	1	1	2	2	2	2	2	2	2	3
T		0	1	2	2	3	3	3	3	3	3	3
C		0	1	2	2	3	3	4	4	4	4	4
G		0	1	2	2	3	3	4	4	5	5	5
A		0	1	2	3	3	3	4	5	5	5	6

Traceback Step

After the matrix fill step, the maximum alignment score for the two test sequences is 6. The traceback step determines the actual alignment(s) that result in the maximum score. Note that with a simple scoring algorithm such as one that is used here, there are likely to be multiple maximal alignments.

The traceback step begins in the M,J position in the matrix, i.e. the position that leads to the maximal score. In this case, there is a 6 in that location. Traceback takes the current cell and looks to the Neighbour cells that could be direct predecessors. This means it looks to the Neighbour to the left (gap in sequence #2), the diagonal Neighbour (match/mismatch), and the Neighbour above it (gap in sequence #1). The algorithm for traceback chooses as the next cell in the sequence one of the possible predecessors. In this case, the Neighbours are marked in red. They are all also equal to 5.

		G	A	A	T	T	C	A	G	T	T	A
		0	0	0	0	0	0	0	0	0	0	0
G		0	1	1	1	1	1	1	1	1	1	1
G		0	1	1	1	1	1	1	2	2	2	2
A		0	1	2	2	2	2	2	2	2	2	3
T		0	1	2	2	3	3	3	3	3	3	3
C		0	1	2	2	3	3	3	4	4	4	4
G		0	1	2	2	3	3	3	4	4	5	5
A		0	1	2	3	3	3	3	4	5	5	6

Since the current cell has a value of 6 and the scores are 1 for a match and 0 for anything else, the only possible predecessor is the diagonal match/mismatch Neighbour. If more than one possible predecessor exists, any can be chosen.

This gives us a current alignment of,

(Seq #1) A

(Seq #2) A

So now we look at the current cell and determine which cell is its direct predecessor. In this case, it is the cell with the red 5.

		G	A	A	T	T	C	A	G	T	T	A
	0	0	0	0	0	0	0	0	0	0		
G	0	1	1	1	1	1	1	1	1	1		
G	0	1	1	1	1	1	1	1	2	2		
A	0	1	2	2	2	2	2	2	2	2		
T	0	1	2	2	3	3	3	3	3	3		
C	0	1	2	2	3	3	4	4	4	4		
G	0	1	2	2	3	3	4	4	5	5	5	
A												6

The alignment as described in the above step adds a gap to sequence #2, so the current alignment is

(Seq #1) T A

(Seq #2) _ A

Once again, the direct predecessor produces a gap in sequence #2.

		G	A	A	T	T	C	A	G	T	T	A
	0	0	0	0	0	0	0	0	0	0		
G	0	1	1	1	1	1	1	1	1	1		
G	0	1	1	1	1	1	1	1	2	2	2	
A	0	1	2	2	2	2	2	2	2	2	2	
T	0	1	2	2	3	3	3	3	3	3	3	
C	0	1	2	2	3	3	4	4	4	4	4	
G	0	1	2	2	3	3	4	4	5	5	5	
A												6

After this step, the current alignment is,

(Seq #1) T T A

_ _ A

Continuing on with the traceback step, we eventually get to a position in column 0 row 0 which tells us that traceback is completed. One possible maximum alignment is:

If the available Capital x_i is	Then the Optimal Proposal is	And the Revenue is for Stage1 is
0	1	0
1	2	5
2	3	6
3	3	6
4	3	6
5	3	6

Giving an alignment of:

```

G   A   A   T   T   C A G T T A
|       |       |   |   |
G   G   A   _   T   C   _ G _ _ A

```

Example

Let's begin with a simple capital budgeting problem. A corporation has \$5 million to allocate to its three plants for possible expansion. Each plant has submitted a number of proposals on how it intends to spend the money. Each proposal gives the cost of the expansion (c) and the total revenue expected (r). The following table 6.1 gives the proposals generated:

Table 6.1: Investment Possibilities

Proposal	Plant 1.		Plant2.		Plant3	
	c_1	r_1	c_2	r_2	c_3	r_3
1	0	0	0	0	0	0
2	1	5	2	8	1	4
3	2	6	3	9	–	–
4	–	–	4	12	–	–

Each plant will only be permitted to enact one of its proposals. The goal is to maximize the firm's revenues resulting from the allocation of the \$5 million. We will assume that any of the \$5 million we don't spend is lost (you can work out how a more reasonable assumption will change the problem as an exercise). A straightforward way to solve this is to try all possibilities and choose the best. In this case, there are only $3 \times 4 \times 2 = 24$ ways of allocating the money. Many of these are infeasible (for instance, proposals 3, 4, and 1 for the three plants costs \$6 million). Other proposals are feasible, but very poor (like proposals 1, 1, and 2, which is feasible but returns only \$4 million).

Here are some disadvantages of total enumeration:

- For larger problems the enumeration of all possible solutions may not be computationally feasible.
- Infeasible combinations cannot be detected *a priori*, leading to inefficiency.
- Information about previously investigated combinations is not used to eliminate inferior, or infeasible, combinations.

Note also that this problem cannot be formulated as a linear Programme, for the revenues returned are not linear functions.

One method of calculating the solution is as follows:

Let's break the problem into three *stages*: each stage represents the money allocated to a single plant. So stage 1 represents the money allocated to plant 1, stage 2 the money to plant 2, and stage 3 the money to plant 3.

We will artificially place an ordering on the stages, saying that we will first allocate to plant 1, then plant 2, then plant 3.

Each stage is divided into *states*. A state encompasses the information required to go from one stage to the next. In this case the states for stages 1, 2, and 3 are

- $\{0,1,2,3,4,5\}$: The amount of money spent on plant 1, represented as x_1 ,
- $\{0,1,2,3,4,5\}$: The amount of money spent on plants 1 and 2 (x_2), and
- $\{5\}$: The amount of money spent on plants 1, 2, and 3 (x_3).

Unlike linear Programming, the x_i do not represent decision variables: they are simply representations of a generic state in the stage.

Associated with each state is a revenue: Note that to make a decision at stage 3, it is only necessary to know how much was spent on plants 1 and 2, not how it was spent. Also notice that we will want x_3 to be 5. Let's try to figure out the revenues associated with each state. The only easy possibility is in stage 1, the states x_1 . Table 6.2 gives the revenue associated with x_1 .

Table 6.2: Stage 1 Computations.

If the available Capital x_1 is	Then the Optimal Porosal is Proposal is for Stage1 is	And the Revenue
0	1	0
1	2	5
2	3	6
3	3	6
4	3	6
5	3	6

We are now ready to tackle the computations for stage 2. In this case, we want to find the best solution for both plants 1 and 2. If we want to calculate the best revenue for a given x_2 , we simply go through all the plant 2 proposals, allocate the given amount of funds to plant 2, and use the above table to see how plant 1 will spend the remainder.

For instance, suppose we want to determine the best allocation for state $x_2 = 4$.

In stage 2 we can do one of the following proposals:

- Proposal 1 gives revenue of 0, leaves 4 for stage 1, which returns 6. Total: 6.
- Proposal 2 gives revenue of 8, leaves 2 for stage 1, which returns 6. Total: 14.
- Proposal 3 gives revenue of 9, leaves 1 for stage 1, which returns 5. Total: 14.
- Proposal 4 gives revenue of 12, leaves 0 for stage 1, which returns 0. Total: 12.

The best thing to do with four units is proposal 1 for plant 2 and proposal 2 for plant 1, returning 14, or proposal 2 for plant 2 and proposal 1 for plant 1, also returning 14. In either case, the revenue for being in state $x_2 = 4$ is 14.

Table 6.3: Stage 2 Computations.

If the available Capital x_1 is	Then the Optimal Proposal is	And the Revenue for Stage 1 is
0	1	0
1	1	5
2	2	8
3	2	13
4	2 or 3	14
5	4	17

We can now go on to stage 3. The only value we are interested in is $x_3 = 5$. Once again, we go through all the proposals for this stage, determine the amount of money remaining and use Table to decide the value for the previous stages.

So here we can do the following at plant 3:

- Proposal 1 gives revenue 0, leaves 5. Previous stages give 17. Total: 17.
- Proposal 2 gives revenue 4, leaves 4. Previous stages give 14. Total: 18.

Therefore, the optimal solution is to implement proposal 2 at plant 3, proposal 2 or 3 at plant 2, and proposal 3 or 2 (respectively) at plant 1. This gives a revenue of 18.

If you study this procedure, you will find that the calculations are done *recursively*. Stage 2 calculations are based on stage 1, stage 3 only on stage 2. Indeed, given you are at a state, all future decisions are made independent of how you got to the state. This is the *principle of optimality* and all of **dynamic Programming** rests on this assumption.

We can sum up these calculations in the following formulas:

Denote by $r(k_j)$ the revenue for proposal k_j at stage j , and by $f_j(x_j)$ the corresponding cost. Let x_j be the revenue of state x_j in stage j . Then we have the following calculations

$$f_1(x_1) = \max_{k_1: r(k_1) \leq x_1} \{r(k_1)\}$$

and

$$f_j(x_j) = \max_{k_j: c(k_j) \leq x_j} \{r(k_j) + f_{j-1}(x_j - c(k_j))\} \text{ for } j = 2, 3$$

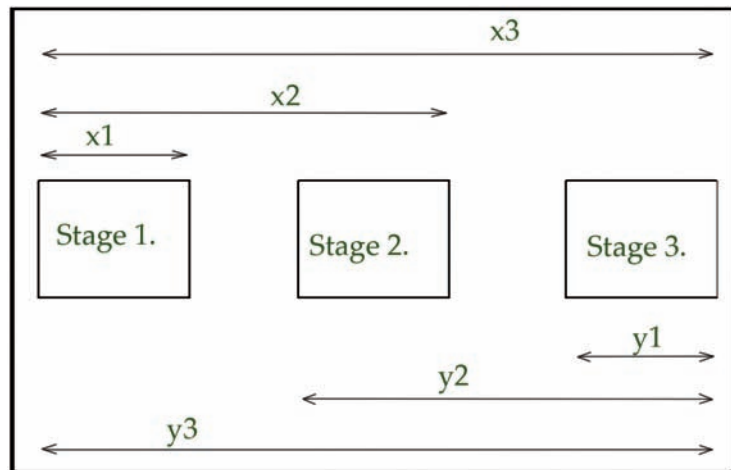
All we were doing with the above calculations was determining these functions.

The computations were carried out in a forward procedure. It was also possible to calculate things from the “last” stage back to the first stage.

We could define,

- y_1 = amount allocated to stages 1, 2, and 3,
- y_2 = amount allocated to stages 2 and 3, and
- y_3 = amount allocated to stage 3.

This defines a *backward recursion*. Graphically, this is illustrated in Figure.



Dynamic

programming is a method for solving a complex problem by breaking it down into a collection of simpler sub problems, solving each of those sub problems just once, and storing their solutions ideally, using a memory-based data structure.

Corresponding formulas are:

- Let $f_3(y_3)$ be the optimal revenue for stage 3, given y_3 ,
- $f_2(y_2)$ be the optimal revenue for stages 2 and 3, given y_3 , and
- $f_1(y_1)$ be the optimal revenue for stages 1, 2, and 3, given y_1 .

The recursion formulas are:

$$f_3(y_3) = \max_{k_3: c(k_3) \leq y_3} \{r(k_3)\}$$

$$f_j(y_j) = \max_{k_j: c(k_j) \leq y_j} \{r(k_j) + f_{j+1}(y_j - c(k_j))\}$$

If you carry out the calculations, you will come up with the same answer. Therefore, in the future, I will be presenting only the backward recursion, except in cases where I wish to contrast the two recursions.

COMMON CHARACTERISTICS

There are a number of characteristics that are common to these two problems and to all dynamic Programming problems.

These are:

The problem can be divided into *stages* with a *decision* required at each stage.

In the capital budgeting problem the stages were the allocations to a single plant. The decision was how much to spend. In the shortest path problem, they were defined by the structure of the graph. The decision was where to go next.

Each stage has a number of states associated with it. The states for the capital budgeting problem corresponded to the amount spent at that point in time. The states for the shortest path problem was the node reached.

- The decision at one stage transforms one state into a state in the next stage.

The decision of how much to spend gave a total amount spent for the next stage. The decision of where to go next defined where you arrived in the next stage.

- Given the current state, the optimal decision for each of the remaining states does not depend on the previous states or decisions. In the budgeting problem, it is not necessary to know how the money was spent in previous stages, only how much was spent. In the path problem, it was not necessary to know how you got to a node, only that you did.
- There exists a recursive relationship that identifies the optimal decision for stage j , given that stage $j+1$ has already been solved.

The final stage must be solvable by itself.

The last two properties are tied up in the recursive relationships given above.

The big skill in dynamic Programming, and the art involved, is to take a problem and determine stages and states so that all of the above hold. If you can, then the recursive relationship makes finding the values relatively easy. Because of the difficulty in identifying stages and states, we will do a fair number of examples.

6.2.4 Knapsack Problem

The knapsack problem is a particular type of integer Programme with just one constraint. Each item that can go into the knapsack has a size and a benefit. The knapsack has a certain capacity. What should go into the knapsack so as to maximize the total benefit? As an example, suppose we have three items as shown in Table 6.4, and suppose the capacity of the knapsack is 5.

Table 6.4: Knapsack Items

Item (j)	Weight (w_j)	Benefit (b_j)
1	2	65
2	3	80
3	1	30

The stages represent the items: we have three stages $j=1,2,3$. The state y_j at stage j represents the total weight of items j and all following items in the knapsack. The decision at stage j is how many items j to place in the knapsack.

Call this value k_j . This leads to the following recursive formulas: Let $f_j(y_j)$ be the value of using units of capacity for items j and following.

Let y_j represent the largest integer less than or equal to a .

$$f_3(y_3) = 30y_3$$

$$f_j(y_j) = \max k_j \leq [y_j / w_j] \{b_j k_j + f_{j+1}(y_j - w_j k_j)\}$$

Let's apply what we've learned so far to a slightly more interesting problem. You are an art thief who has found a way to break into the impressionist wing at the Art Institute of Chicago. Obviously you can't take everything. In particular, you're constrained to take only what your knapsack can hold — let's say it can only hold W pounds. You also know the market value for each painting. Given that you can only carry W pounds what paintings should you steal in order to maximize your profit?

First let's see how this problem exhibits both overlapping sub problems and optimal substructure. Say there are n paintings with weights $w_1, \dots, w_n$ and market values $v_1, \dots, v_n$. Define $A(i, j)$ as the maximum value that can be attained from considering only the

first i items weighting at most j pounds as follows. Obviously $A(0,j) = 0$ and $A(i,0) = 0$ for any $i \leq n$ and $j \leq W$. If $w_i > j$ then $A(i,j) = A(i-1, j)$ since we cannot include the i^{th} item. If, however, $w_i \leq j$ then $A(i,j)$ then we have a choice: include the i^{th} item or not. If we do not include it then the value will be $A(i-1, j)$. If we do include it, however, the value will be $v_i + A(i-1, j - w_i)$. Which choice should we make? Well, whichever is larger, i.e., the maximum of the two.

This problem exhibits both overlapping sub problems and optimal substructure and is therefore a good candidate for dynamic Programming. The sub problems overlap because at any stage (i,j) we might need to calculate $A(k,l)$ for several $k < i$ and $l < j$. We have optimal substructure since at any point we only need information about the choices we have already made.

The recursive solution is not hard to write:

```
def A(w, v, i,j):
    if i == 0 or j == 0: return 0
    if w[i-1] > j: return A(w, v, i-1, j)
    if w[i-1] ≤ j: return max(A(w,v, i-1, j), v[i-1] + A(w,v, i-1, j - w[i-1]))
```

Remember we need to calculate $A(n,W)$. To do so we're going to need to create an n -by- W table whose entry at (i,j) contains the value of $A(i,j)$. The first time we calculate the value of $A(i,j)$ we store it in the table at the appropriate location. This technique is called memoization and is one way to exploit overlapping sub problems. There's also a Ruby module called memoize which does it for Ruby.

To exploit the optimal substructure we iterate over all $i \leq n$ and $j \leq W$, at each step applying the recursion formula to generate the $A(i,j)$ entry by using the memoized table rather than calling $A()$ again. This gives an algorithm which takes $O(nW)$ time using $O(nW)$ space and our desired result is stored in the $A(n,W)$ entry in the table.

The following is an example of global sequence alignment using Needleman/Wunsch techniques. For this example, the two sequences to be globally aligned are:

G A A T T C A G T T A (sequence #1)

G G A T C G A (sequence #2)

So $M = 11$ and $N = 7$ (the length of sequence #1 and sequence #2, respectively)

A simple scoring scheme is assumed where

- $S_{i,j} = 1$ if the residue at position i of sequence #1 is the same as the residue at position j of sequence #2 (match score); otherwise
- $S_{i,j} = 0$ (mismatch score)
- $w = 0$ (gap penalty)

6.3 AMORTIZED ANALYSIS



REMEMBER

Amortized Analysis is used for algorithms where an occasional operation is very slow, but most of the other operations are faster. In Amortized Analysis, we analyze a sequence of operations and guarantee a worst case average time which is lower than the worst case time of a particular expensive operation.

In an amortized analysis, the time required to perform a sequence of data-structure operations is averaged over all the operations performed. Amortized analysis can be used to show that the average cost of an operation is small, if one averages over a sequence of operations, even though a single operation might be expensive. Amortized analysis differs from average-case analysis in that probability is not involved; an amortized analysis guarantees the average performance of each operation in the worst case.

Amortized analysis is a worst-case analysis of a *sequence* of operations — to obtain a tighter bound on the overall or average cost per operation in the sequence than is obtained by separately analyzing each operation in the sequence. For instance, when we considered the union and find operations for the disjoint set data abstraction earlier in the semester, we were able to bound the running time of individual operations by $O(\log n)$. However, for a sequence of n operations, it is possible to obtain tighter than an $O(n \log n)$ bound. Here we will consider a simplified version of the hash table problem above, and show that a sequence of n insert operations has overall time $O(n)$.

There are three main techniques used for amortized analysis:

- The aggregate method, where the total running time for a sequence of operations is analyzed.
- The accounting (or banker's) method, where we impose an extra charge on inexpensive operations and use it to pay for expensive operations later on.
- The potential (or physicist's) method, in which we derive a *potential function* characterizing the amount of extra work we can do in each step. This potential either increases or decreases with each successive operation, but cannot be negative.

Consider an extensible array that can store an arbitrary number of integers, like an ArrayList or Vector in Java. These are implemented in terms of ordinary (non-extensible) arrays. Each add operation inserts a new element after all the elements previously inserted. If there are no empty cells left, a new array of double the size is allocated, and all the data from the old



array is copied to the corresponding entries in the new array. For instance, consider the following sequence of insertions, starting with an array of length 1:

	+++
Insert 11	11
	+++
	+++++
Insert 12	11 12
	+++++
	+++++
Insert 13	11 12 13
	+++++
	+++++
Insert 14	11 12 13 14
	+++++
	+++++
Insert 15	11 12 13 14 15
	+++++

The table is doubled in the second, third, and fifth steps. As each insertion takes $O(n)$ time in the worst case, a simple analysis would yield a bound of $O(n^2)$ time for n insertions. But it is not this bad. Let's analyze a sequence of n operations using the three methods.

6.3.1 Aggregate Method

Let c_i be the cost of the i -th insertion:

$c_i = i$ if $i-1$ is a power of 2

1 otherwise

Let's consider the size of the table s_i and the cost c_i for the first few insertions in a sequence:

i	1	2	3	4	5	6	7	8	9	10
s_i	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1

Alternatively we can see that $c_i = 1 + d_i$ where d_i is the cost of doubling the table size. That is

$d_i = i-1$ if $i-1$ is a power of 2

0 otherwise

Then summing over the entire sequence, all the 1's sum to $O(n)$, and all the d_i also sum to

$O(n)$. That is,

where $m = \log(n - 1)$. Both terms on the right hand side of the inequality are $O(n)$, so the total running time of n insertions is $O(n)$.

6.3.2 Accounting (Banker's) Method

The aggregate method directly seeks a bound on the overall running time of a sequence of operations. In contrast, the accounting method seeks to find a *payment* of a number of extra time units charged to each individual operation such that the sum of the payments is an upper bound on the total actual cost. Intuitively, one can think of maintaining a bank account. Low-cost operations are charged a little bit more than their true cost, and the surplus is deposited into the bank account for later use. High-cost operations can then be charged less than their true cost, and the deficit is paid for by the savings in the bank account. In that way we spread the cost of high-cost operations over the entire sequence. The charges to each operation must be set large enough that the balance in the bank account always remains positive, but small enough that no one operation is charged significantly more than its actual cost.

We emphasize that the extra time charged to an operation does not mean that the operation really takes that much time. It is just a method of accounting that makes the analysis easier.

If we let c'_i be the charge for the i -th operation and c_i be the true cost, then we would like

$$\sum_{1 \leq i \leq n} c_i \leq \sum_{1 \leq i \leq n} c'_i$$

for all n , which says that the amortized time $\sum_{1 \leq i \leq n} c'_i$ for that sequence of n operations is a bound on the true time $\sum_{1 \leq i \leq n} c_i$.

Back to the example of the extensible array. Say it costs 1 unit to insert an element and 1 unit to move it when the table is doubled. Clearly a charge of 1 unit per insertion is not enough, because there is nothing left over to pay for the moving. A charge of 2 units per insertion again is not enough, but a charge of 3 appears to be:

i	1	2	3	4	5	6	7	8	9	10
s_i	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
c'_i	3	3	3	3	3	3	3	3	3	3
b_i	2	3	3	5	3	5	7	9	3	4

where b_i is the balance after the i -th insertion.

In fact, this is enough in general. Let m refer to the m -th element inserted. The three units charged to m are spent as follows:

- One unit is used to insert m immediately into the table.
- One unit is used to move m the first time the table is grown after m is inserted.
- One unit is donated to element $m - 2^k$, where 2^k is the largest power of 2 not exceeding m , and is used to move that element the first time the table is grown after m is inserted.

Now whenever an element is moved, the move is already paid for. The first time an element is moved, it is paid for by one of its own time units that was charged to it when it was inserted; and all subsequent moves are paid for by donations from elements inserted later.

In fact, we can do slightly better, by charging just 1 for the first insertion and then 3 for each insertion after that, because for the first insertion there are no elements to copy. This will yield a zero balance after the first insertion and then a positive one thereafter.

6.3.3 Potential (Physicist's) Method

Above we saw the aggregate method and the banker's method for dealing with extensible arrays. Now let us look at the physicist's method.

Suppose we can define a potential function Φ (read "Phi") on states of a data structure with the following properties:

- $\Phi(h_0) = 0$, where h_0 is the initial state of the data structure.
- $\Phi(h_i) \geq 0$ for all states h_i of the data structure occurring during the course of the computation.

Intuitively, the potential function will keep track of the precharged time at any point in the computation. It measures how much saved-up time is available to pay for expensive operations. It is analogous to the bank balance in the banker's method. But interestingly, it depends only on the current state of the data structure, irrespective of the history of the computation that got it into that state.

We then define the amortized time of an operation as

$$c + \Phi(h') - \Phi(h),$$

where c is the actual cost of the operation and h and h' are the states of the data structure before and after the operation, respectively. Thus the amortized time is the actual time plus the change in potential. Ideally, Φ should be defined so that the amortized time of each operation is small. Thus the change in potential should be positive for low-cost operations and negative for high-cost operations.

Now consider a sequence of n operations taking actual times $c_0, c_1, c_2, \dots, c_{n-1}$ and producing data structures $h_1, h_2, \dots, h_n$ starting from h_0 . The total amortized time is the sum of the individual amortized times:

$$\begin{aligned}
& (c_0 + \Phi(h_1) - \Phi(h_0)) + (c_1 + \Phi(h_2) - \Phi(h_1)) + \dots + (c_{n-1} + \Phi(h_n) - \Phi(h_{n-1})) \\
& = c_0 + c_1 + \dots + c_{n-1} + \Phi(h_n) - \Phi(h_0) \\
& = c_0 + c_1 + \dots + c_{n-1} + \Phi(h_n).
\end{aligned}$$

Therefore the amortized time for a sequence of operations overestimates of the actual time by $\Phi(h_n)$, which by assumption is always positive. Thus the total amortized time is always an upper bound on the actual time.

For dynamically resizable arrays with resizing by doubling, we can use the potential function

$$\Phi(h) = 2n - m,$$

where n is the current number of elements and m is the current length of the array. If we start with an array of length 0 and allocate an array of length 1 when the first element is added, and thereafter double the array size whenever we need more space, we have $\Phi(h_0) = 0$ and $\Phi(h_t) \geq 0$ for all t . The latter inequality holds because the number of elements is always at least half the size of the array.

Now we would like to show that adding an element takes amortized constant time. There are two cases.

- If $n < m$, then the actual cost is 1, n increases by 1, and m does not change. Then the potential increases by 2, so the amortized time is $1 + 2 = 3$.
- If $n = m$, then the array is doubled, so the actual time is $n + 1$. But the potential drops from n to 2, so amortized time is $n + 1 + (2 - n) = 3$.

In both cases, the amortized time is $O(1)$.

The key to amortized analysis with the physicist's method is to define the right potential function. The potential function needs to save up enough time to be used later when it is needed. But it cannot save so much time that it causes the amortized time of the current operation to be too high.

SUMMARY

- Algorithms are to computer programs what the mind is to the human body. Many algorithmic design patterns or paradigms exist such as greedy algorithms, divide-and-conquer algorithms, dynamic programming algorithms and backtracking algorithms.
- Algorithm analysis is a field of computer science that is dedicated to understanding the complexity of algorithms. Algorithms are generally defined as processes that perform a series of operations to an end.
- Greedy algorithms are simple and straightforward. They are shortsighted in their approach in the sense that they take decisions on the basis of information at hand without worrying about the effect these decisions may have in the future.
- Huffman codes compress data very effectively: savings of 20% to 90% are typical, depending on the characteristics of the data being compressed. We consider the data to be a sequence of characters.
- Huffman invented a greedy algorithm that constructs an optimal prefix code called a
- Huffman code. Its proof of correctness relies on the greedy-choice property and optimal substructure.
- Dynamic programming is a very powerful technique to solve a particular class of problems. It demands very elegant formulation of the approach and simple thinking and the coding part is very easy.
- Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems. But unlike, divide and conquer, these sub-problems are not solved independently.
- The first step in the global alignment dynamic Programming approach is to create a matrix with $M + 1$ columns and $N + 1$ rows where M and N correspond to the size of the sequences to be aligned.
- Amortized analysis is a worst-case analysis of a *sequence* of operations — to obtain a tighter bound on the overall or average cost per operation in the sequence than is obtained by separately analyzing each operation in the sequence.

KNOWLEDGE CHECK

1. Indicate the runtime of Dijkstra's algorithm when the implementation is based on a binary heap. (E = edges; V = vertices)
 - a $O(E \log V)$

- b $O(V^2)$
 - c $O(E + V \log V)$
 - d $O(E + V)$
2. What is the worst case run-time complexity of binary search algorithm?
- a A - $O(n^2)$
 - b B - $O(n^{\log n})$
 - c C - $O(n^3)$
 - d D - $O(n)$
3. Which of the below mentioned sorting algorithms are not stable?
- a A - Selection Sort
 - b B - Bubble Sort
 - c C - Merge Sort
 - d D - Insertion Sort
4. Program with highest run-time complexity is
- a A - Tower of Hanoi
 - b B - Fibonacci Series
 - c C - Prime Number Series
 - d D - None of the above
5. Which of the following standard algorithms is not a Greedy algorithm?
- a Dijkstra's shortest path algorithm
 - b Prim's algorithm
 - c Kruskal algorithm
 - d Huffman Coding
 - e Bellman Ford Shortest path algorithm

REVIEW QUESTIONS

1. Describe the elements of the greedy strategy.
2. Define the greedy versus dynamic programming



3. What are Huffman codes?
4. Discuss on matroids and greedy methods
5. Explain the steps in dynamic programming.
6. Define the knapsack problem
7. What is differences between aggregate method and accounting (Banker's) method?

CHECK YOUR RESULT

1. (a) 2. (d) 3. (a) 4. (a) 5. (d)

REFERENCES

1. Wisegeek <http://www.wisegeek.com/what-is-algorithm-analysis.htm>
2. ehow http://www.ehow.com/how_8699537_design-algorithms.html
3. <https://www.quora.com/What-is-the-difference-between-algorithm-technique-and-algorithmic-technique>
4. <http://gauss.ececs.uc.edu/Courses/c472/lectures/matroids.html>
5. <http://staff.ustc.edu.cn/~csl/graduate/algorithms/book6/chap17.htm>
6. Tutorialspoint https://www.tutorialspoint.com/data_structures_algorithms/dynamic_programming.htm
7. <https://www.codechef.com/wiki/tutorial-dynamic-programming>
8. <http://www.cs.cornell.edu/courses/cs3110/2011sp/Lectures/lec20-amortized/amortized.htm>
9. <http://www.personal.kent.edu/~rmuhamma/Algorithms/algorithm.html>
10. Tutorialspoint https://www.tutorialspoint.com/parallel_algorithm/design_techniques.htm

DIVIDE AND CONQUER

CHAPTER

7

LEARNING OBJECTIVES

After studying this chapter, you will be able to:

1. Analyze divide and conquer algorithms
2. Explain strassen's algorithm

INTRODUCTION

In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently. When we keep on dividing the sub-problems into even smaller sub-problems, we may eventually reach a stage where no more division is possible. Those “atomic” smallest possible sub-problem (fractions) are solved. The solution of all sub-problems is finally merged in order to obtain the solution of an original problem. Divide and conquer (D&C) is an algorithm design paradigm based on multi-branched recursion. A divide and conquer algorithm works by recursively breaking down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly. The

solutions to the sub-problems are then combined to give a solution to the original problem. This divide and conquer technique is the basis of efficient algorithms for all kinds of problems, such as sorting (e.g., quicksort, merge sort), multiplying large numbers (e.g. the Karatsuba algorithm), finding the closest pair of points, syntactic analysis (e.g., top-down parsers), and computing the discrete Fourier transform (FFTs). Understanding and designing D&C algorithms is a complex skill that requires a good understanding of the nature of the underlying problem to be solved.



As when proving a theorem by induction, it is often necessary to replace the original problem with a more general or complicated problem in order to initialize the recursion, and there is no systematic method for finding the proper generalization. These D&C complications are seen when optimizing the calculation of a Fibonacci number with efficient double recursion. The correctness of a divide and conquer algorithm is usually proved by mathematical induction, and its computational cost is often determined by solving recurrence relations.



The name “divide and conquer” is sometimes applied also to algorithms that reduce each problem to only one sub-problem, such as the binary search algorithm for finding a record in a sorted list (or its analog in numerical computing, the bisection algorithm for root finding). These algorithms can be implemented more efficiently than general divide-and-conquer algorithms; in particular, if they use tail recursion, they can be converted into simple loops. Under this broad definition, however, every algorithm

that uses recursion or loops could be regarded as a “divide and conquer algorithm”. Therefore, some authors consider that the name “divide and conquer” should be used only when each problem may generate two or more sub-problems. The name decrease and conquer has been proposed instead for the single-sub-problem class.

An important application of divide and conquer is in optimization, where if the search space is reduced (“pruned”) by a constant factor at each step, the overall algorithm has the same asymptotic complexity as the pruning step, with the constant depending on the pruning factor (by summing the geometric series); this is known as prune and search.



REMEMBER

An important application of divide and conquer is in optimization, where if the search space is reduced (“pruned”) by a constant factor at each step, the overall algorithm has the same asymptotic complexity as the pruning step, with the constant depending on the pruning factor (by summing the geometric series); this is known as prune and search.

7.1 ANALYSIS OF DIVIDE AND CONQUER ALGORITHMS

Divide-and-conquer algorithms are typically recursive, since the conquer part involves invoking the same technique on a smaller sub problem. Analyzing the running times of recursive programs is rather tricky, but we will show that there is an elegant mathematical concept, called a recurrence, which is useful for analyzing the sort of recursive programs that naturally arise in divide-and-conquer solutions. For the next couple of lectures we will discuss some examples of divide-and-conquer algorithms, and how to analyze those using recurrences.

The divide-and-conquer strategy solves a problem by:

- Breaking it into sub problems that are themselves smaller instances of the same type of problem
- Recursively solving these sub problems
- Appropriately combining their answers

The real work is done piecemeal, in three different places: in the partitioning of problems into sub problems; at the very tail end of the recursion, when the sub problems are so small that they are solved outright; and in the gluing together of partial answers. These are held together and coordinated by the algorithm’s core recursive structure.

7.1.1 Multiplication

The mathematician Carl Friedrich Gauss once noticed that although the product of two complex numbers

$$(a + bi)(c + di) = ac - bd + (bc + ad)i$$

seems to involve four real-number multiplications, it can in fact be done with just three: ac , bd , and $(a + b)(c + d)$, since

$$bc + ad = (a + b)(c + d) - ac - bd$$

In our big-O way of thinking, reducing the number of multiplications from four to three seems wasted ingenuity. But this modest improvement becomes very significant when applied recursively.

$$\begin{aligned} x &= \boxed{x_L} \boxed{x_R} = 2^{n/2}x_L + x_R \\ y &= \boxed{y_L} \boxed{y_R} = 2^{n/2}y_L + y_R. \end{aligned}$$

For instance, if $x = 10110110_2$ (the subscript 2 means binary) then $x_L = 1011_2$, $x_R = 0110_2$, and $x = 1011_2 \times 24 + 0110_2$. The product of x and y can then be rewritten as

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) = 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$

We will compute xy via the expression on the right. The additions take linear time, as do the multiplications by powers of 2 (which are merely left-shifts). The significant operations are the four $n=2$ -bit multiplications, $x_L y_L$; $x_L y_R$; $x_R y_L$; $x_R y_R$; these we can handle by four recursive calls. Thus our method for multiplying n -bit numbers starts by making recursive calls to multiply these four pairs of $n=2$ -bit numbers (four subproblems of half the size), and then evaluates the preceding expression in $O(n)$ time. Writing $T(n)$ for the overall running time on n -bit inputs, we get the **recurrence relation**.

$$T(n) = 4T(n/2) + O(n)$$

We will soon see general strategies for solving such equations. In the meantime, this particular one works out to $O(n^2)$, the same running time as the traditional grade-school multiplication technique. So we have a radically new algorithm, but we haven't yet made any progress in efficiency. How can our method be sped up?

This is where Gauss's trick comes to mind. Although the expression for xy seems to demand four $n=2$ -bit multiplications, as before just three will do: $x_L y_L$; $x_R y_R$; and



$(x_L+x_R)(y_L+y_R)$, since $x_L y_R + x_R y_L = (x_L+x_R)(y_L+y_R) - x_L y_L - x_R y_R$. The resulting algorithm, shown in Figure 7.1, has an improved running time of

$$T(n) = 3T(n/2) + O(n)$$

The point is that now the constant factor improvement, from 4 to 3, occurs at every level of the recursion, and this compounding effect leads to a dramatically lower time bound of $O(n^{1.59})$.

```
function multiply(x,y)
Input:  Positive integers x and y, in binary
Output: Their product

n = max(size of x, size of y)
if n = 1: return xy

xL, xR = leftmost ⌊n/2⌋, rightmost ⌊n/2⌋ bits of x
yL, yR = leftmost ⌊n/2⌋, rightmost ⌊n/2⌋ bits of y

P1 = multiply(xL, yL)
P2 = multiply(xR, yR)
P3 = multiply(xL + xR, yL + yR)
return P1 × 2n + (P3 - P1 - P2) × 2n/2 + P2
```

Figure 7.1: A divide-and-conquer algorithm for integer multiplication

This running time can be derived by looking at the algorithm's pattern of recursive calls, which form a tree structure, as in Figure 7.2. Let's try to understand the shape of this tree. At each successive level of recursion the subproblems get halved in size. At the $(\log_2 n)$ th level, the subproblems get down to size 1, and so the recursion ends. Therefore, the height of the tree is $\log_2 n$. The branching factor is 3 each problem recursively produces three smaller ones with the result that at depth k in the tree there are 3^k subproblems, each of size $n=2^k$. For each subproblem, a linear amount of work is done in identifying further subproblems and combining their answers. Therefore the total time spent at depth k in the tree is

$$3^k \times O\left(\frac{n}{2^k}\right) = \left(\frac{3}{2}\right)^k \times O(n)$$

At the very top level, when $k = 0$, this works out to $O(n)$.



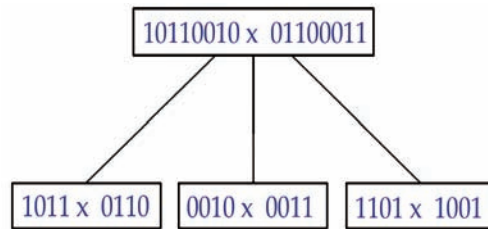
Recurrence relation

is an equation that recursively defines a sequence or multidimensional array of values, once one or more initial terms are given: each further term of the sequence or array is defined as a function of the preceding terms.

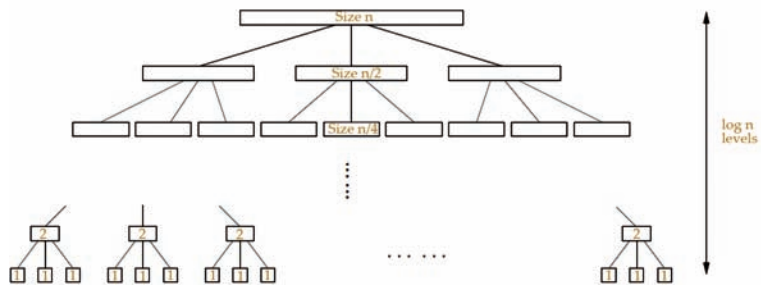
**REMEMBER**

In divide-and-conquer algorithms, the number of subproblems translates into the branching factor of the recursion tree; small changes in this coefficient can have a big impact on running time.

At the bottom, when $k = \log_2 n$, it is $O(3^{\log_2 n})$, which can be rewritten as $O(n^{\log_2 3})$ (do you see why?). Between these two endpoints, the work done increases geometrically from $O(n)$ to $O(n^{\log_2 3})$, by a factor of $3=2$ per level. The sum of any increasing geometric series is, within a constant factor, simply the last term of the series: such is the rapidity of the increase.



(a)



(b)

Figure 7.2 Divide-and-conquer integer multiplication. (a) Each problem is divided into three subproblems. (b) The levels of recursion.

Therefore the overall running time is $O(n^{\log_2 3})$, which is about $O(n^{1.59})$. In the absence of Gauss's trick, the recursion tree would have the same height, but the branching factor would be 4. There would be $4^{\log_2 n} = n^2$ leaves, and therefore the running time would be at least this much.

7.1.2 Recurrence Relations

Divide-and-conquer algorithms often follow a generic pattern: they tackle a problem of size n by recursively solving, say, a subproblems of size n/b and then combining these answers



in $O(n^d)$ time, for some a ; b ; $d > 0$ (in the multiplication algorithm, $a = 3$, $b = 2$, and $d = 1$). Their running time can therefore be captured by the equation $T(n) = aT(\lfloor n/b \rfloor) + O(n^d)$. We next derive a closed-form solution to this general recurrence so that we no longer have to solve it explicitly in each new instance.

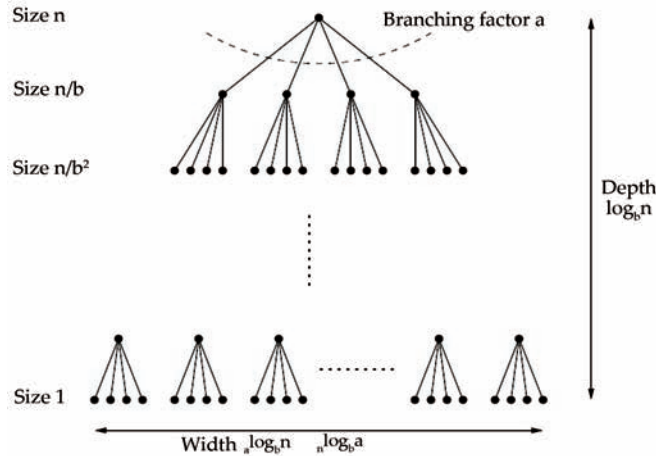


Figure 7.3: Each problem of size n is divided into a subproblems of size n/b .

Master theorem: If $T(n) = aT(\lfloor n/b \rfloor) + O(n^d)$ for some constants $a > 0$, $b > 1$, and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

This single theorem tells us the running times of most of the divide-and-conquer procedures we are likely to use.

Proof. To prove the claim, let's start by assuming for the sake of convenience that n is a power of b . This will not influence the final bound in any important way after all, n is at most a multiplicative factor of b away from some power of b and it will allow us to ignore the rounding effect in $\lfloor n/b \rfloor$. Next, notice that the size of the subproblems decreases by a factor of b with each level of recursion, and therefore reaches the base case after $\log_b n$ levels. This is the height of the recursion tree. Its branching factor is a , so the k th level of the tree is made up of a^k subproblems, each of size n/b^k (Figure 7.3). The total work done at this level is

$$a^k \times O\left(\frac{n}{b^k}\right)^d = O(n^d) \times \left(\frac{a}{b^d}\right)^k$$

As k goes from 0 (the root) to $\log_b n$ (the leaves), these numbers form a geometric series with ratio $a=b^d$. Finding the sum of such a series in big-O notation is easy, and comes down to three cases.

7.1.3 Merge sort

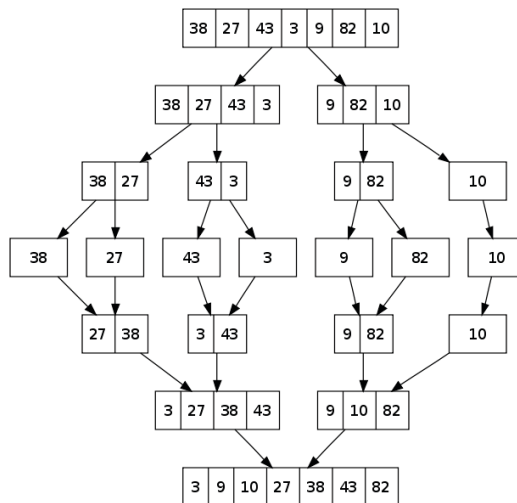
Like Quick Sort, Merge Sort is a Divide and Conquer algorithm. It divides input array in two halves, calls itself for the two halves and then merges the two sorted halves. The `merge()` function is used for merging two halves. The `merge(arr, l, m, r)` is key process that assumes that `arr[l..m]` and `arr[m+1..r]` are sorted and merges the two sorted sub-arrays into one. See following C implementation for details.

```

MergeSort(arr[], l, r)
If r > l
    1. Find the middle point to divide the array into two halves:
        middle m = (l+r)/2
    2. Call mergeSort for first half:
        Call mergeSort(arr, l, m)
    3. Call mergeSort for second half:
        Call mergeSort(arr, m+1, r)
    4. Merge the two halves sorted in step 2 and 3:
        Call merge(arr, l, m, r)

```

The following diagram from shows the complete merge sort process for an example array {38, 27, 43, 3, 9, 82, 10}. If we take a closer look at the diagram, we can see that the array is recursively divided in two halves till the size becomes 1. Once the size becomes 1, the merge processes comes into action and starts merging arrays back till the complete array is merged.



Applications of Merge Sort

Merge Sort is useful for sorting linked lists in $O(n \log n)$ time. In case of linked lists the case is different mainly due to difference in memory allocation of arrays and linked lists. Unlike arrays, linked list nodes may not be adjacent in memory. Unlike array, in linked list, we can insert items in the middle in $O(1)$ extra space and $O(1)$ time. Therefore merge operation of merge sort can be implemented without extra space for linked lists.

In arrays, we can do random access as elements are continuous in memory. Let us say we have an integer (4-byte) array A and let the address of A[0] be x then to access A[i], we can directly access the memory at $(x + i \cdot 4)$. Unlike arrays, we cannot do random access in linked list. **Quick Sort** requires a lot of this kind of access. In linked list to access i'th index, we have to travel each and every node from the head to i'th node as we don't have continuous block of memory. Therefore, the overhead increases for quick sort. Merge sort accesses data sequentially and the need of random access is low.



REMEMBER

Merge sort is an efficient, general-purpose, comparison-based sorting algorithm. Most implementations produce a stable sort, which means that the implementation preserves the input order of equal elements in the sorted output.

How Merge Sort Works?

Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being $O(n \log n)$, it is one of the most respected algorithms. Merge sort first divides the array into equal halves and then combines them in a sorted manner.

To understand merge sort, we take an unsorted array as the following



We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.



We further divide these arrays and we achieve atomic value which can no more be divided.



Now, we combine them in exactly the same manner as they were broken down. And note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.



After the final merging, the list should look like this –



Now we should learn some programming aspects of merge sorting.

Algorithm

Merge sort keeps on dividing the list into equal halves until it can no more be divided. By definition, if it is only one element in the list, it is sorted. Then, merge sort combines the smaller sorted lists keeping the new list sorted too.

Step 1 – if it is only one element in the list it is already sorted, return.

Step 2 – divide the list recursively into two halves until it can no more be divided.

Step 3 – merge the smaller lists into new list in sorted order.

Pseudocode

We shall now see the pseudocodes for merge sort functions. As our algorithms point

out two main functions – divide & merge. Merge sort works with recursion and we shall see our implementation in the same way.

```
procedure mergesort( var a as array )
```

```
    if ( n == 1 ) return a
```

```
    var l1 as array = a[0] ... a[n/2]
```

```
    var l2 as array = a[n/2+1] ... a[n]
```

```
    l1 = mergesort( l1 )
```

```
    l2 = mergesort( l2 )
```

```
    return merge( l1, l2 )
```

```
end procedure
```

```
procedure merge( var a as array, var b as array )
```

```
    var c as array
```

```
    while ( a and b have elements )
```

```
        if ( a[0] > b[0] )
```

```
            add b[0] to the end of c
```

```
            remove b[0] from b
```

```
        else
```

```
            add a[0] to the end of c
```

```
            remove a[0] from a
```

```
        end if
```

```
    end while
```

```
    while ( a has elements )
```

```
        add a[0] to the end of c
```

```
        remove a[0] from a
```

```

end while

while ( b has elements )

    add b[0] to the end of c

    remove b[0] from b

end while

return c

end procedure

```

7.1.4 Median

The median of a list of numbers is its 50th percentile: half the numbers are bigger than it, and half are smaller. For instance, the median of [45, 1, 10, 30, 25] is 25, since this is the middle element when the numbers are arranged in order. If the list has even length, there are two choices for what the middle element could be, in which case we pick the smaller of the two, say.

The purpose of the median is to summarize a set of numbers by a single, typical value. The mean, or average, is also very commonly used for this, but the median is in a sense more typical of the data: it is always one of the data values, unlike the mean, and it is less sensitive to outliers.



The median of a list of a hundred 1's is (rightly) 1, as is the mean. However, if just one of these numbers gets accidentally corrupted to 10,000, the mean shoots up above 100, while the median is unaffected.

Computing the median of n numbers is easy: just sort them. The drawback is that this takes $O(n \log n)$ time, whereas we would ideally like something linear. We have reason to be hopeful, because sorting is doing far more work than we really need we just want the middle element and don't care about the relative ordering of the rest of them. When looking for a recursive solution, it is paradoxically often easier to work with a more general version of the problem for the simple reason

that this gives a more powerful step to recurse upon. In our case, the generalization we will consider is selection.

SELECTION

Input: A list of numbers S ; an integer k

Output: The k th smallest element of S

For instance, if $k = 1$, the minimum of S is sought, whereas if $k = \lfloor |S|/2 \rfloor$, it is the median.

A randomized divide-and-conquer algorithm for selection

Here's a divide-and-conquer approach to selection. For any number v , imagine splitting list S into three categories: elements smaller than v , those equal to v (there might be duplicates), and those greater than v . Call these S_L , S_v and S_R respectively. For instance, if the array

S :

2	36	5	21	8	13	11	20	5	4	1
---	----	---	----	---	----	----	----	---	---	---

is split on $v = 5$, the three subarrays generated are

S_L :

2	4	1
---	---	---

 S_v :

5	5
---	---

 S_R :

36	21	8	13	11	20
----	----	---	----	----	----

The search can instantly be narrowed down to one of these sublists. If we want, say, the eighth-smallest element of S , we know it must be the third-smallest element of S_R since $|S_L| + |S_v| = 5$. That is, $\text{selection}(S, 8) = \text{selection}(S_R, 3)$. More generally, by checking k against the sizes of the subarrays, we can quickly determine which of them holds the desired element:

$$\text{selection}(S, k) = \begin{cases} \text{selection}(S_L, k) & \text{if } k \leq |S_L| \\ v & \text{if } |S_L| < k \leq |S_L| + |S_v| \\ \text{selection}(S_R, k - |S_L| - |S_v|) & \text{if } k > |S_L| + |S_v|. \end{cases}$$

The three sublists S_L , S_v and S_R can be computed from S in linear time; in fact, this computation can even be done in place, that is, without allocating new memory. We then recurse on the appropriate sublist. The effect of the split is thus to shrink the number of elements from $|S|$ to at most $\max\{|S_L|, |S_R|\}$.

Our divide-and-conquer algorithm for selection is now fully specified, except for the crucial detail of how to choose v . It should be picked quickly, and it should shrink the array substantially, the ideal situation being $|S_L|, |S_R| \approx 1/2 |S|$. If we could always guarantee this situation, we would get a running time of

$$T(n) = T(n/2) + O(n)$$

which is linear as desired. But this requires picking v to be the median, which is our ultimate goal! Instead, we follow a much simpler alternative: we pick v randomly from S .

Efficiency analysis

Naturally, the running time of our algorithm depends on the random choices of v . It is possible that due to persistent bad luck we keep picking v to be the largest element of the array (or the smallest element), and thereby shrink the array by only one element each time. In the earlier example, we might first pick $v = 36$, then $v = 21$, and so on. This worst-case scenario would force our selection algorithm to perform

$$n + (n - 1) + (n - 2) + \cdots + \frac{n}{2} = \Theta(n^2)$$



An early two-subproblem D&C algorithm that was specifically developed for computers and properly analyzed is the merge sort algorithm, invented by John von Neumann in 1945.

operations (when computing the median), but it is extremely unlikely to occur. Equally unlikely is the best possible case we discussed before, in which each randomly chosen v just happens to split the array perfectly in half, resulting in a running time of $O(n)$. Where, in this spectrum from $O(n)$ to $\Theta(n^2)$, does the average running time lie? Fortunately, it lies very close to the best-case time.

To distinguish between lucky and unlucky choices of v , we will call v good if it lies within the 25th to 75th percentile of the array that it is chosen from. We like these choices of v because they ensure that the sublists S_L and S_R have size at most three-fourths that of S (do you see why?), so that the array shrinks substantially. Fortunately, good v 's are abundant: half the elements of any list must fall between the 25th to 75th percentile!

7.2 STRASSEN'S ALGORITHM

The Strassen's method of matrix multiplication is a typical divide and conquer algorithm. We've seen so far some divide and conquer algorithms like merge sort and the Karatsuba's fast multiplication of large numbers. However let's get again on what's behind the divide and conquer approach. Unlike the dynamic programming where we "expand" the solutions

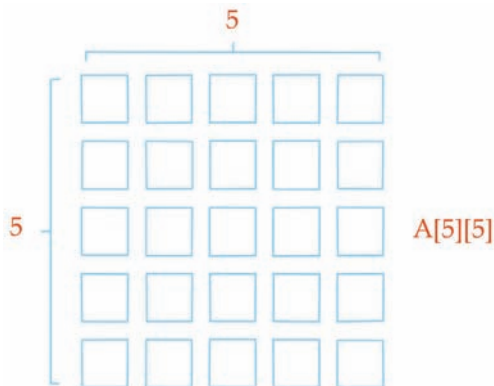
of sub-problems in order to get the final solution, here we are talking more on joining sub-solutions together. These solutions of some sub-problems of the general problem are equal and their merge is somehow well defined.

A typical example is the merge sort algorithm. In merge sort we have two sorted arrays and all we want is to get the array representing their union again sorted. Of course, the tricky part in merge sort is the merging itself. That's because we've to pass through the two arrays, A and B, and we've to compare each "pair" of items representing an item from A and from B. A bit off topic, but this is the weak point of merge sort and although its worst-case **time complexity** is $O(n \log(n))$, quicksort is often preferred in practice because there's no "merge". Quicksort just concatenates the two sub-arrays. Note that in quicksort the sub-arrays aren't with an equal length in general and although its worst-case time complexity is $O(n^2)$ it often outperforms merge sort. This simple example from the paragraph above shows us how sometimes merging the solutions of two sub-problems actually isn't a trivial task to do. Thus we must be careful when applying any divide and conquer approach. The general algorithm on multiplying two matrices $A[N \times N]$ and $B[N \times N]$ is fairly simple. Although it's more difficult than multiplying two numbers and also it is not commutative it's still very simple – but slow. Let's first define what's a matrix $A[N \times N]$. As we speak about matrices $N \times N$ we usually think of a square grid with N rows and N columns. In each row and column $A[i][j]$ we've a value.



Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the string representing the input.

Square Matrix



Any two dimensional array can be interpreted as a matrix. An array $A[n][n]$ is also a $N \times N$ matrix. Of course, as developers, we can think of a matrix as a two-dimensional array.

```
// PHP two-dimensional array
$a = array(
    0 => array($v1, $v2, $v3, $v4),
    1 => array($v5, $v6, $v7, $v8),
    2 => array($v9, $v10, $v11, $v12),
);
```

7.2.1 Karatsuba Fast Multiplication



Algorithm is an effective method that can be expressed within a finite amount of space and time

Typically multiplying two n -digit numbers require n^2 multiplications. That is actually how we, humans, multiply numbers. Let's take a look of an example in case we've to multiply two 2-digit numbers.

$$12 \times 15 = ?$$

OK, we know that the answer is 180 and there are lots of intuitive methods that help us get the right answer. Indeed 12×15 it's just a bit more difficult to calculate than 10×15 , because multiplying by 10 it really easy – we just add one 0 at the end of the number. Thus 15×10 equals 150. But now again on 12×15 – we know that this equals 10×15 (which is 150) and 2×15 , which is also very easy to calculate and it is 30. The result of 12×15 will be $150 + 30$, which fortunately isn't difficult to get and equals to 180.

That was easy but in some cases the calculations are a bit more difficult and we need a structured algorithm to get the right answer. What about 65×97 ? That is not so easy as 12×15 , right?

The **algorithm** we know from the primary school, described on the diagram below, is well structured and help us multiply two numbers.

$$\begin{array}{r}
 65 \\
 \times 97 \\
 \hline
 455 \\
 5850 \\
 \hline
 6305
 \end{array}$$

We see that even for two-digit numbers this is quite difficult – we have 4 multiplications and some additions.

$$\begin{array}{r}
 65 \\
 \times 97 \\
 \hline
 455 \\
 5850 \\
 \hline
 6305
 \end{array}$$

However so far we know how to multiply numbers, the only problem is that our task becomes very difficult as the numbers grow. If multiplying 65 by 97 was somehow easy, what about

374773294776321

x

222384759707982

It seems almost impossible.

Basically Karatsuba stated that if we have to multiply two n -digit numbers x and y , this can be done with the following operations, assuming that B is the base of and $m < n$.

First both numbers x and y can be represented as x_1, x_2 and y_1, y_2 with the following formula.

$$x = x_1 * B^m + x_2$$

$$y = y_1 * B^m + y_2$$

Obviously now xy will become as the following product.

$$xy = (x_1 * B^m + x_2)(y_1 * B^m + y_2) \Rightarrow$$

$$a = x_1 * y_1$$

$$b = x_1 * y_2 + x_2 * y_1$$

$$c = x_2 * y_2$$

Finally xy will become:

$$xy = a * B^{2m} + b * B^m + c$$

However a , b and c can be computed at least with four multiplication, which isn't a big optimization. That is why Karatsuba came up with the brilliant idea to calculate b with the following formula:

$$b = (x_1 + x_2)(y_1 + y_2) - a - c$$

That make use of only three multiplications to get xy .

Let's see this formula by example.

$$47 \times 78$$

$$x = 47$$

$$x = 4 * 10 + 7$$

$$x_1 = 4$$

$$x_2 = 7$$

$$y = 78$$

$$y = 7 * 10 + 8$$

$$y_1 = 7$$

$$y_2 = 8$$

$$a = x_1 * y_1 = 4 * 7 = 28$$

$$c = x_2 * y_2 = 7 * 8 = 56$$

$$b = (x_1 + x_2)(y_1 + y_2) - a - c = 11 * 15 - 28 - 56$$

Now the thing is that $11 * 15$ it's again a multiplication between 2-digit numbers, but fortunately we can apply the same rules two them. This makes the algorithm of Karatsuba a perfect example of the "divide and conquer" algorithm.



REMEMBER

The basic step of Karatsuba's algorithm is a formula that allows one to compute the product of two large numbers x and y using three multiplications of smaller numbers, each with about half as many digits as x and y , plus some additions and digit shifts.

Implementation

Typically the standard implementation of multiplication of n-digit numbers require n^2 multiplications as you can see from the following PHP implementation.

```

    $x = array(1,2,3,4);

    $y = array(5,6,7,8);

function multiply($x, $y)
{
    $len_x = count($x);
    $len_y = count($y);
    $half_x = ceil($len_x / 2);
    $half_y = ceil($len_y / 2);
    $base = 10;

    // bottom of the recursion
    if ($len_x == 1 && $len_y == 1) {
        return $x[0] * $y[0];
    }

    $x_chunks = array_chunk($x, $half_x);
    $y_chunks = array_chunk($y, $half_y);

    // predefine aliases
    $x1 = $x_chunks[0];
    $x2 = $x_chunks[1];
    $y1 = $y_chunks[0];
    $y2 = $y_chunks[1];

    return multiply($x1, $y1) * pow($base, $half_x * 2)
// a

```

```

    + (multiply($x1, $y2) + multiply($x2, $y1)) * pow($base, $half_x) // b
    + multiply($x2, $y2); // c}

// 7 006 652

echo multiply($x, $y);

```

Karatsuba Multiplication

Karatsuba replaces two of the multiplications – this of $x_1 * y_2 + x_2 * y_1$ with only one – $(x_1 + x_2)(y_1 + y_2)$ and this makes the algorithm faster.

```

$x = array(1,2,3,4);
$y = array(5,6,7,8);

function karatsuba($x, $y)
{
    $len_x = count($x);
    $len_y = count($y);

    // bottom of the recursion
    if ($len_x == 1 && $len_y == 1) {
        return $x[0] * $y[0];
    }

    if ($len_x == 1 || $len_y == 1) {
        $t1 = implode("", $x);
        $t2 = implode("", $y);
        return (int)$t1 * $t2;
    }

    $a = array_chunk($x, ceil($len_x/2));
    $b = array_chunk($y, ceil($len_y/2));

```



```

$deg = floor($len_x/2);

$x1 = $a[0];    // 1
$x2 = $a[1];    // 2
$y1 = $b[0];    // 1
$y2 = $b[1];    // 2

return ($a = karatsuba($x1, $y1)) * pow(10, 2 * $deg)
        + ($c = karatsuba($x2, $y2))
        + (karatsuba(sum($x1, $x2), sum($y1, $y2)) - $a - $c) * pow(10, $deg);
}

// 7 006 652

echo karatsuba($x, $y);

```

7.2.2 Finding the Closest Pair of Points

Problem: Given n ordered pairs $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, find the distance between the two points in the set that are closest together.

The brute force algorithm is as follows:

Iterate through all possible pairs of points, calculating the distance between each of these pairs. Any time you see a distance shorter than the shortest distance seen, update the shortest distance seen.

Since computing the distance between two points takes $O(1)$ time, and there are

a total of $\frac{n(n-1)}{2} = q(n^2)$ distinct pairs of points, it follows that the running time of this algorithm is $q(n^2)$.

Can we do better?

Here's the idea:

- 1) Split the set of n points into two halves by a vertical line. (We can do this by sorting all the points by their x -coordinate and then picking the middle point and drawing a vertical line just to the left or right of it.)
- 2) Recursively call the function to solve the problem on both sets of points.

3) Return the smaller of the two values.

What's the problem with this idea?

The problem is that the actual shortest distance between any two of the original points MIGHT BE between a point from the first set and a point in the second set! Consider this situation:

Here we have four points separated into groups A and B. Let the points be labeled 1, 2, 3 and 4 from left to right. Notice that the minimum of the distances between points 1 and 2 and then points 3 and 4 is NOT the actual shortest possible distance between points, since points 2 and 3 are much closer to one another.

Thus, we must adapt our approach:

In step 3, we can “save” the smaller of the two values, (we’ll call this d), then, we have to check to see if there are points (one in each group) that are closer than d apart.

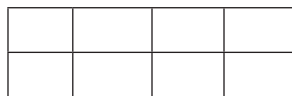
Do we need to search through all possible pairs of points from the two different sides?

Probably not. We must only consider points that are within a distance of d to our dividing line. Still, however, one could construct a case where ALL the points on each side are within d of the vertical line:

So, technically speaking, this case is as bad as our original idea where we’d have to compare each pair of points to one another from the different groups.

But, wait, is it really necessary to compare each point on one side with every other point on every other side???

Consider the following rectangle around the dividing line that is constructed by eight $d/2 \times d/2$ squares. Note that the diagonal of each square is $d/\sqrt{2}$, which is less than d . Since each square lies on a single side of the dividing line, at most one point lies in each box, because if two points were within a single box the distance between those two points would be less than d , contradicting the definition of d . This means, that there are at MOST 7 other points that could possibly be a distance of less than d apart from a given point, that have a greater y coordinate than that point. (We assume that our point is on the bottom row of this grid; we draw the grid that way.)



Now, we come to the issue of how do we know WHICH 7 points to compare a given point with??? The idea is as follows: as you are processing the points recursively, SORT them based on the y -coordinate. Then, for a given point within the strip, you

only need to compare with the next seven points! Here is a **pseudocode** version of the algorithm.

```

closest_pair(p) {
    mergesort(p, 1, n) // n is number of points
    return rec_cl_pair(p, 1, 2)
}

rec_cl_pair(p, i, j) {

    if (j - i < 3) { \ \ If there are three points or less...
        mergesort(p, i, j)
        return shortest_distance(p[i], p[i+1], p[i+2])
    }

    xval = p[(i+j)/2].x
    deltaL = rec_cl_pair(p, i, (i+j)/2)
    deltaR = rec_cl_pair(p, (i+j)/2+1, j)
    delta = min(deltaL, deltaR)
    merge(p, i, k, j)
    v = vert_strip(p, xval, delta)
    for k=1 to size(v)-1
        for s = (k+1) to min(t, k+7)
            delta = min(delta, dist(v[k], v[s]))
    return delta
}

```

Let's trace through an example with the following 9 points:



Pseudocode is an informal high-level description of the operating principle of a computer program or other algorithm.

(0, 0), (1, 6), (2, 8), (2, 3), (3, 4), (5, 1), (6, 7), (7, 4) and (8, 0).

These are already sorted these by the x-coordinate. We will split the points in half, with the first four in the first half and the last five in the last half.

So, we must recursively call `rec_cl_pair` on the set

{(0, 0), (1, 6), (2, 8), (2, 3)}

This results in two more recursive calls, to `rec_cl_pair` on the sets {(0, 0), (1, 6)} and {(2, 8), (2, 3)}. The first call simply returns the distance $\sqrt{37}$ and sorts the points by y-coordinate which leaves them unchanged. The second call returns 5 and swaps the order of the points (2, 3), (2, 8) in the array p.

Then, delta is set to 5, and the points are all merged by y coordinate: (0, 0), (2, 3), (1, 6), (2, 8). (All of these are then copied into the v array since all points are within a distance of 5 of the line $x=1$.)

Since we have less than 7 points, all points are compared and it is discovered that (1, 6) and (2, 8) are a distance of $\sqrt{5}$ apart.

Now, let's call `rec_cl_pair` on the set

{(3, 4), (5, 1), (6, 7), (7, 4), (8, 0)}

We first make two recursive calls to the sets: {(3, 4), (5, 1)} and {(6, 7), (7, 4), (8, 0)}.

The first will result in deltaL being set to $\sqrt{13}$ and the points getting sorted by y: {(5, 1), (3, 4)}.

The second results in delta being set to $\sqrt{10}$ and the points getting sorted by y: {(8, 0), (7, 4), (6, 7)}. We then merge all of these points: {(8, 0), (5, 1), (3, 4), (7, 4), (6, 7)}. All of these points get copied into v.

Then, we discover that NO pair of points beats the original deltaR of $\sqrt{10}$. This is then the closest distance between any pair of points within the second set of data.

Finally, we must merge ALL the points together by y-coordinate:

(0, 0), (8, 0), (5, 1), (2, 3), (3, 4), (7, 4), (1, 6), (6, 7), (2, 8)

this time, we only pick those points that are within $\sqrt{10}$ of the line $x=2$ to copy into v. These points are:

(0, 0), (5, 1), (2, 3), (3, 4), (1, 6), (2, 8)

Now, we scan through all pairs to discover that the shortest distance between any of the two points is $\sqrt{2}$.

7.2.3 Strassen's algorithm: Matrix multiplication

The standard method of matrix multiplication of two $n \times n$ matrices takes $T(n) = O(n^3)$.

The following algorithm multiplies $n \times n$ matrices A and B:

```
// Initialize C.
```

```
for i = 1 to n
```

```
  for j = 1 to n
```

```
    for k = 1 to n
```

```
      C[i, j] += A[i, k] * B[k, j];
```

Strassen's algorithm is a Divide-and-Conquer algorithm that beats the bound. The usual multiplication of two $n \times n$ matrices takes

$$A = \begin{pmatrix} a_1 & a_2 \\ a_1 & a_2 \end{pmatrix} \quad B = \begin{pmatrix} b_1 & b_2 \\ b_1 & b_2 \end{pmatrix} \quad C = \begin{pmatrix} c_1 & c_2 \\ c_1 & c_2 \end{pmatrix}$$

if $C = AB$, then we have the following:

$$c_1 = a_1 b_1 + a_2 b_1$$

$$c_2 = a_1 b_2 + a_2 b_2$$

$$c_1 = a_1 b_1 + a_2 b_1$$

$$c_2 = a_1 b_2 + a_2 b_2$$

8 $n/2 \times n/2$ matrix multiplications plus

4 $n/2 \times n/2$ matrix additions

$$T(n) = 8T(n/2) + O(n^2)$$

$$\text{Plug in } a = 8, b = 2, k = 2 \rightarrow \log_b a = 3 \rightarrow T(n) = O(n^3)$$

Strassen showed how two matrices can be multiplied using only 7 multiplications and 18 additions:



Data is a set of values of qualitative or quantitative variables.

Consider calculating the following 7 products:

$$q_1 = (a_{11} + a_{22}) * (b_{11} + b_{22})$$

$$q_2 = (a_{21} + a_{22}) * b_{11}$$

$$q_3 = a_{11} * (b_{12} - b_{22})$$

$$q_4 = a_{22} * (b_{21} - b_{11})$$

$$q_5 = (a_{11} + a_{12}) * b_{22}$$

$$q_6 = (a_{21} - a_{11}) * (b_{11} + b_{12})$$

$$q_7 = (a_{12} - a_{22}) * (b_{21} + b_{22})$$

It turns out that

$$c_{11} = q_1 + q_4 - q_5 + q_7$$

$$c_{12} = q_3 + q_5$$

$$c_{21} = q_2 + q_4$$

$$c_{22} = q_1 + q_3 - q_2 + q_6$$

Let's verify a couple of these:

$$\begin{aligned} q_1 + q_4 - q_5 + q_7 &= (a_{11} + a_{22}) * (b_{11} + b_{22}) + a_{22} * (b_{21} - b_{11}) \\ &\quad - (a_{11} + a_{12}) * b_{22} + (a_{12} - a_{22}) * (b_{21} + b_{22}) \\ &= a_{11}b_{11} + a_{11}b_{22} + a_{22}b_{11} + a_{22}b_{22} + a_{22}b_{21} - a_{22}b_{11} - a_{11}b_{22} - a_{12}b_{22} \\ &\quad + a_{12}b_{21} + a_{12}b_{22} - a_{22}b_{21} - a_{22}b_{22} \\ &= a_{11}b_{11} + a_{12}b_{21}, \text{ since everything else cancels out.} \end{aligned}$$

$$\begin{aligned} q_3 + q_5 &= a_{11} * (b_{12} - b_{22}) + (a_{11} + a_{12}) * b_{22} \\ &= a_{11}b_{12} - a_{11}b_{22} + a_{11}b_{22} + a_{12}b_{22} \\ &= a_{11}b_{12} + a_{12}b_{22} \end{aligned}$$

Strassen came up with these combinations. He probably realized that he wanted to determine each element in the product using less than 8 multiplications. From there, he probably just played around with it. In the algorithms text by Cormen, Leiserson and Rivest, they show how one could derive these products.



If we let $T(n)$ be the running time of Strassen's algorithm, then it satisfies the following recurrence relation:

$$T(n) = 7T(n/2) + O(n^2)$$

It's important to note that the hidden constant in the $O(n^2)$ term is larger than the corresponding constant for the standard divide and conquer algorithm for this problem. However, for large matrices this algorithm yields an improvement over the standard one with respect to time.



ROLE MODEL

VOLKER STRASSEN: IMPORTANT CONTRIBUTIONS TO THE ANALYSIS OF THE STRASSEN ALGORITHM.

BIOGRAPHY AND EARLY LIFE

Volker Strassen (born April 29, 1936) is a German mathematician, a professor emeritus in the department of mathematics and statistics at the University of Konstanz. For important contributions in analysis of algorithms he has received many awards, including the Cantor medal, the Konrad Zuse Medal, the Paris Kanellakis Award for work on randomized primality testing, the Knuth Prize for “seminal and influential contributions to the design and analysis of efficient algorithms.”

Strassen was born on April 29, 1936, in Düsseldorf-Gerresheim. After studying music, philosophy, physics, and mathematics at several German universities, he received his Ph.D. in mathematics in 1962 from the University of Göttingen under the supervision of Konrad Jacobs (de). He then took a position in the department of statistics at the University of California, Berkeley while performing his habilitation at the University of Erlangen-Nuremberg, where Jacobs had since moved. In 1968, Strassen moved to the Institute of Applied Mathematics at the University of Zurich, where he remained for twenty years before moving to the University of Konstanz in 1988. He retired in 1998.

Research

Strassen began his researches as a probabilist; his 1964 paper An Invariance Principle for the Law of the Iterated Logarithm defined a functional form of the law of the iterated logarithm, showing a form of scale invariance in random walks. This result, now known as Strassen’s invariance principle or as Strassen’s law of the iterated logarithm, has been highly cited and led to a 1966 presentation at the International Congress of Mathematicians.

In 1969, Strassen shifted his research efforts towards the analysis of algorithms with a paper on Gaussian elimination, introducing Strassen’s algorithm, the first algorithm for performing matrix multiplication faster than the $O(n^3)$ time bound that would result from a naive algorithm. In the same paper he also presented an asymptotically fast algorithm to perform matrix inversion, based on the fast matrix multiplication algorithm. This result was an important theoretical breakthrough, leading to much additional research on fast matrix multiplication, and despite later theoretical improvements it



remains a practical method for multiplication of dense matrices of moderate to large sizes. In 1971 Strassen published another paper together with Arnold Schönhage on asymptotically fast integer multiplication based on the fast Fourier transform; see the Schönhage–Strassen algorithm. Strassen is also known for his 1977 work with Robert M. Solovay on the Solovay–Strassen primality test, the first method to show that testing whether a number is prime can be performed in randomized polynomial time and one of the first results to show the power of randomized algorithms more generally.

In 1999 Strassen was awarded the Cantor medal, and in 2003 he was co-recipient of the Paris Kanellakis Award with Robert Solovay, Gary Miller, and Michael Rabin for their work on randomized primality testing. In 2008 he was awarded the Knuth Prize for “seminal and influential contributions to the design and analysis of efficient algorithms.” In 2011 he won the Konrad Zuse Medal of the Gesellschaft für Informatik. In 2012 he became a fellow of the American Mathematical Society.

SUMMARY

- In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently.
- The name “divide and conquer” is sometimes applied also to algorithms that reduce each problem to only one sub-problem, such as the binary search algorithm for finding a record in a sorted list (or its analog in numerical computing, the bisection algorithm for root finding).
- Divide-and-conquer algorithms are typically recursive, since the conquer part involves invoking the same technique on a smaller sub problem.
- Merge Sort is useful for sorting linked lists in $O(n \log n)$ time. In case of linked lists the case is different mainly due to difference in memory allocation of arrays and linked lists.
- The median of a list of numbers is its 50th percentile: half the numbers are bigger than it, and half are smaller
- The purpose of the median is to summarize a set of numbers by a single, typical value.

KNOWLEDGE CHECK

1. Which of the following sorting algorithm is of divide and conquer type?
 - a. Bubble sort
 - b. Insertion sort
 - c. Merge sort
 - d. Selection sort
2. Running merge sort on an array of size n which is already sorted is
 - a. $O(n)$
 - b. $O(n \log n)$
 - c. $O(n^2)$
 - d. None
3. Which of the following algorithm design technique is used in the quick sort algorithm?
 - a. Dynamic programming
 - b. Backtracking
 - c. Divide-and-conquer
 - d. Greedy method



4. **Merge sort uses**
 - a. Divide-and-conquer
 - b. Backtracking
 - c. Heuristic approach
 - d. Greedy approach
5. **Which of the following algorithms is NOT a divide & conquer algorithm by nature?**
 - a. Euclidean algorithm to compute the greatest common divisor
 - b. Heap Sort
 - c. Cooley-Tukey fast Fourier transform
 - d. Quick Sort

REVIEW QUESTIONS

1. What is the multiplication process of divide and conquer algorithms?
2. Discuss on recurrence relations.
3. What do you understand by merge sort? Explain.
4. How to find the closest pair of points in strassen's lgorithm?
5. Define the matrix multiplication in strassen's algorithm.

CHECK YOUR RESULT

1. (d) 2. (b) 3. (c) 4. (a) 5. (b)

REFERENCES

1. Tutorialspoint: https://www.tutorialspoint.com/data_structures_algorithms/merge_sort_algorithm.htm
2. <http://quiz.geeksforgeeks.org/merge-sort/>
3. <https://brilliant.org/wiki/median-finding-algorithm/>
4. <https://people.eecs.berkeley.edu/~vazirani/algorithms/chap2.pdf>
5. <http://ce.bonabu.ac.ir/uploads/30/CMS/user/file/115/EBook/Introduction.to.Algorithms.3rd.Edition.Sep.2010.pdf>

INDEX

A

ADT (Abstract Data Type) 57, 115
Algorithm 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14, 15, 16, 17, 19, 22, 25, 27, 28, 30
Algorithm design 161, 174, 175, 185, 186
Array of length 168, 169
Asymptotic analysis 10, 11, 13, 14

B

Backtracking algorithm 161, 162, 163, 166, 172,
185, 186
Binary search 121, 123, 124, 157
Binary search trees (BSTs) 88
Breadth-First Search (BFS) 107
Brute-force approach 162
Bubble sort 133, 135, 157, 158

C

Computer system 7, 28

D

Data Encryption Standard (DES) 2
data structure 127, 143, 157
Depth-First Search (DFS) 107
Depth first traversal 162
Divide and conquer 235, 236, 237, 243, 248, 249,

252, 261, 264, 265

F

First-in first-out (FIFO) 104
Fourier transform (FFTs) 236
Framework 31, 51

H

Hashing 127
Hash Table 127, 157

I

Interpolation search 124, 125, 157

L

Last-in first-out (LIFO) 100
linear probing 129, 130, 131
Linear search 120, 124, 157
Logarithm function 10, 14, 15

M

Matrix multiplication 248, 259, 262, 265
Maximum contiguous subarrays (MCS) 25
Median 246, 247, 248, 264, 265
Merge sort 145, 146, 157, 158
Merge Sort 242, 243, 264

Q

Quick Sort 242, 243, 265

R

Recurrence relation 238, 261

S

search algorithm 120, 121, 124, 125, 157, 158

Selection sort 142, 157, 158

Shell sort 148, 149, 157

sorting algorithm 119, 133, 134, 135, 139, 142,
148, 152, 157, 158

Sorting algorithms 133

T

Termination argument 19

Introduction to Algorithm Design

Problem solving is an essential part of every scientific discipline. It has two components; problem identification and formulation, and solution of the formulated problem. One can solve a problem on its own using ad hoc techniques or follow those techniques that have produced efficient solutions to similar problems. This requires the understanding of various algorithm design techniques, how and when to use them to formulate solutions and the context appropriate for each of them. Algorithm design is identified and incorporated into many solution theories of operation research, such as dynamic programming and divide-and-conquer. Techniques for designing and implementing algorithm designs are algorithm design patterns, such as template method pattern and decorator pattern, and uses of data structures, and name and sort lists. Some current day uses of algorithm design can be found in internet retrieval processes of web crawling, packet routing and caching.

Key features:

- The book sponsors the study of algorithm design techniques by presenting most of the useful algorithm design techniques and illustrating them through numerous examples.
- Topics are logically arranged according to the course outline of the subject.
- The use of simple language to facilitate a better understanding of each chapter.
- Illustrative examples of varying difficulties, wherever needed, are presented to enhance the interest of students.
- The provision of knowledge check test for each chapter is given at the end of chapter to strengthen the learning process of students.
- Case Study, Role Model, Keywords, and Examples are also provided to enrich the knowledge of students.

